



دانشکده مهندسی کامپیوتر

شبکه‌های کامپیوتری



دانشگاه صنعتی امیرکبیر
(پلی‌تکنیک تهران)

مسعود صبائی

دانشکده مهندسی کامپیوتر

دانشگاه صنعتی امیرکبیر

لایه کاربرد

فهرست مطالب:

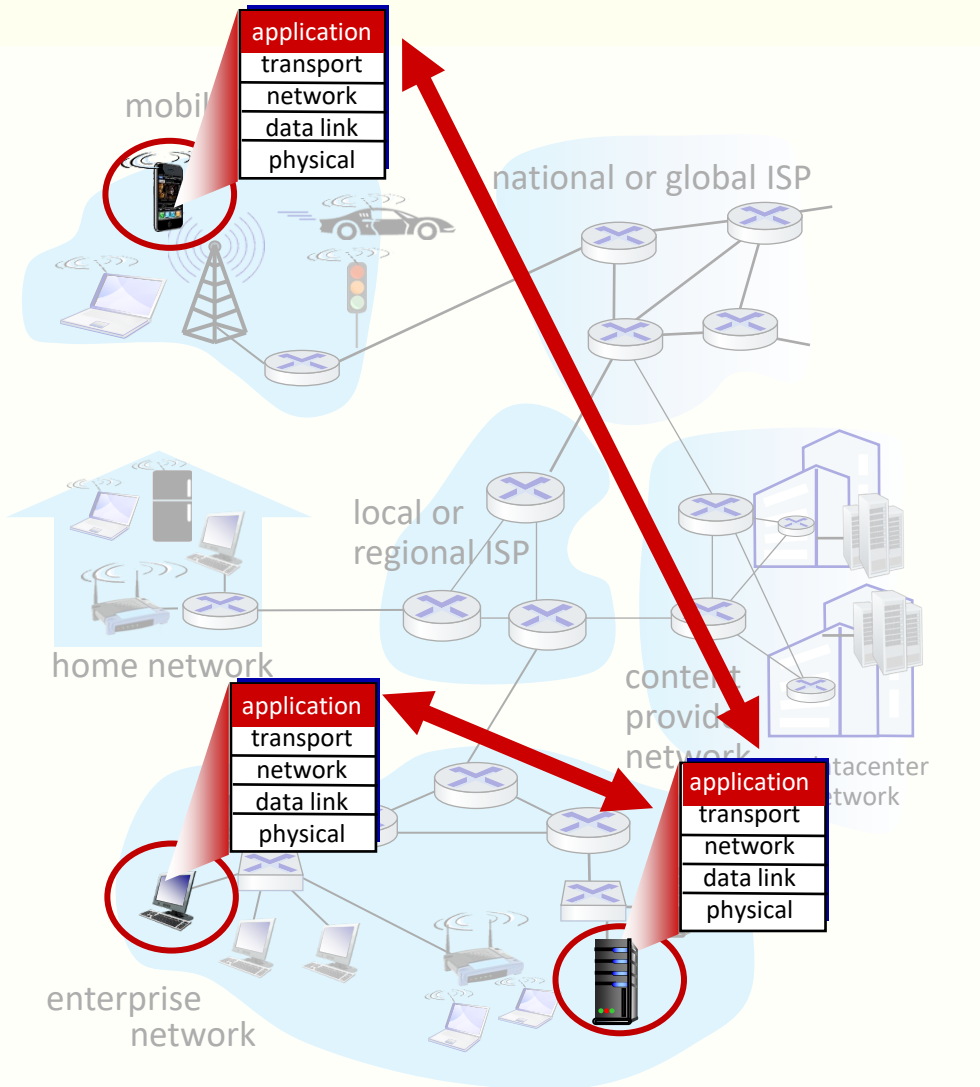
- اصول کاربردهای شبکه‌ای
- الگو سرویس‌دهنده-سرویس‌گیرنده
- الگو نظیر به نظیر
- پروتکل لایه کاربرد و نیازمندی‌های کیفیت سرویس‌دهی
- وب و پروتکل HTTP
- پست الکترونیکی و پروتکل SMTP
- سرویس دایرکتوری در اینترنت و پروتکل DNS
- انتقال فایل و پروتکل FTP
- معماری نظیر به نظیر
- جریان‌سازی ویدیو و شبکه‌های توزیع محتوا
- برنامه‌نویسی سوکت

برخی از برنامه‌های کاربردی:

- شبکه‌سازی اجتماعی (اینستاگرام، ایکس، لینکدین...)
- وب
- پیام‌رسان متنی
- پست الکترونیکی
- بازهای تحت شبکه چند کاربره
- جریان‌سازی ویدیو ذخیره‌شده (یوتیوب، نتفلیکس، ...)
- به اشتراک‌گذاری فایل نظیر به نظیر (بیت تورنت، ...)
- تلفن اینترنتی (اسکایپ، ...)
- کنفرانس زنده ویدیویی
- دسترسی از راه دور (remote Login)
- ...

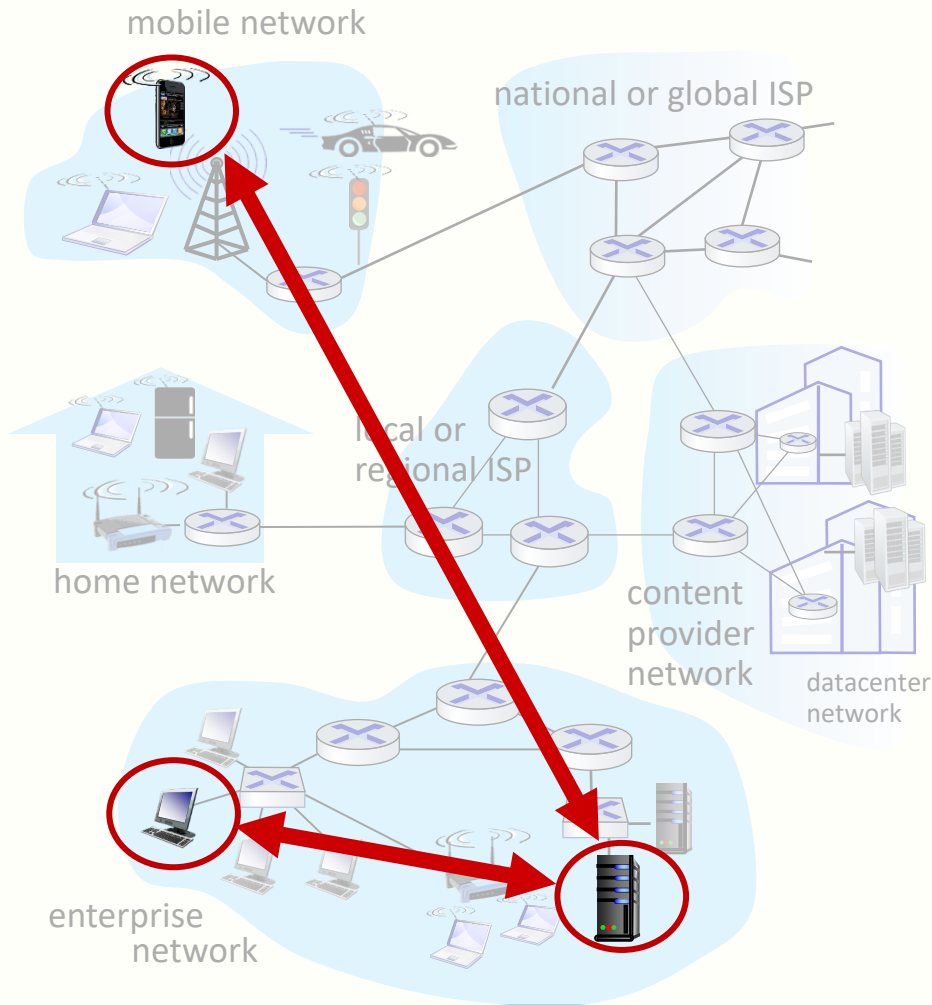
ایجاد یک برنامه کاربردی:

- اجرای برنامه‌های کاربردی بر روی سیستم‌های انتهایی (مختلف)
- تبادل پیام‌ها بین برنامه‌های کاربردی از طریق شبکه
- به عنوان مثال: تبادل پیام بین برنامه کاربردی مرورگر وب با برنامه کاربردی سرویس‌دهنده وب
- برنامه‌های کاربردی فقط در سیستم‌های انتهایی اجرا می‌شوند.
- دستگاه‌های (گره‌های) میانی برنامه‌های کاربردی را اجرا نمی‌کنند.
- الگوهای ایجاد برنامه‌های کامپیوتری:
- الگو سرویس‌دهنده-سرویس‌گیرنده (Client-Server Paradigm)
- الگو نظیر به نظیر (Peer-to-Peer Paradigm)



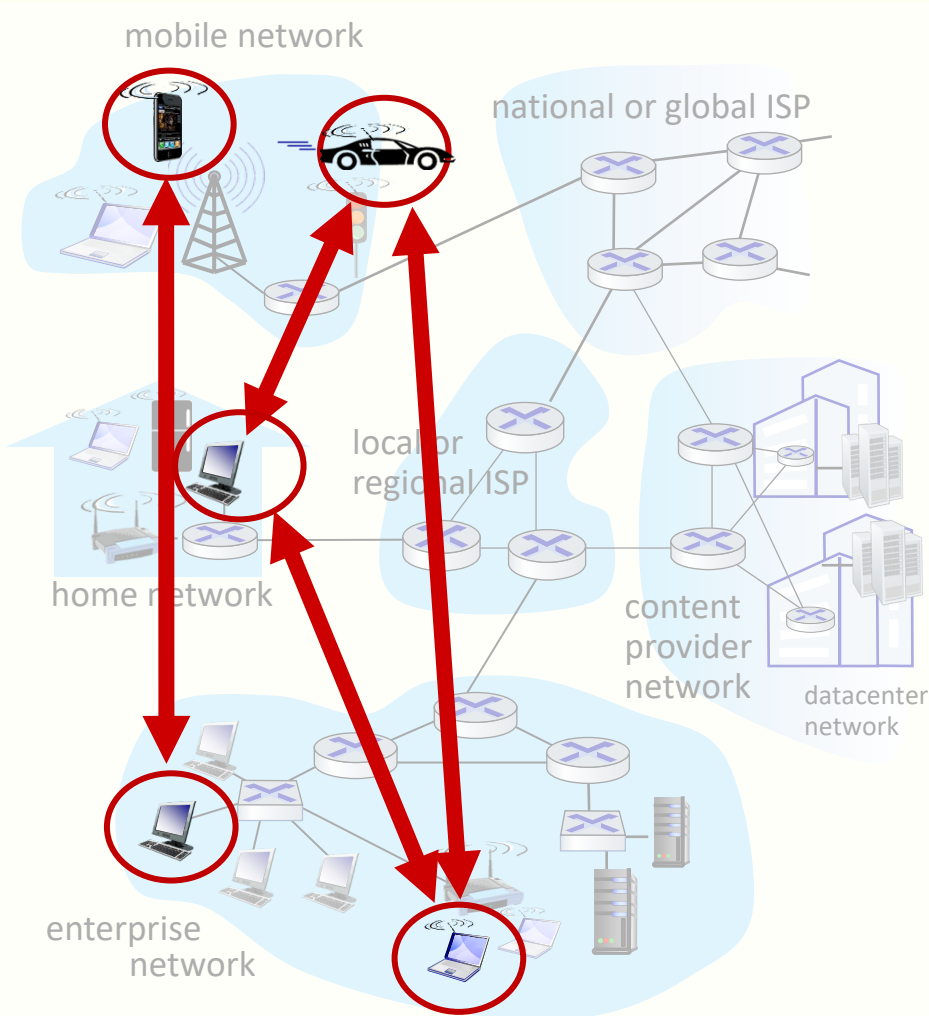
الگو سرویس دهنده-سرویس گیرنده (Client-Server Paradigm)

- برنامه‌های کاربردی فقط یکی از دو نقش **سرویس دهنده** یا **سرویس گیرنده** را دارند.
- **سرویس دهنده‌ها (Servers)**
 - همیشه در حال اجرا و آماده سرویس دهی هستند.
 - آدرس IP ثابت
 - اتصال دائمی به اینترنت دارد.
 - غالباً در مراکز داده (برای مقیاس پذیر بودن و کاهش هزینه‌های جاری)
- **سرویس گیرنده‌ها (Clients)**
 - باید از قبل آدرس IP و شماره پورت سرویس دهنده را داشته باشد
 - با ارسال پیام به سرویس دهنده درخواست دریافت سرویس خود را اعلام می‌کند.
 - ممکن آدرس IP ثابت نداشته باشد.
 - ممکن اتصال دائمی به اینترنت نداشته باشد.



الگو نظیر به نظیر (Peer-to-Peer Paradigm)

- برنامه‌های کاربردی **نظیر** هم هستند.
- هم نقش سرویس‌دهنده و هم نقش سرویس‌گیرنده را دارند.
- هر نظیر درخواست دریافت سرویس را از بقیه نظیرها دارد و همزمان به درخواست نظیرهای دیگر پاسخ می‌دهد.
- **مقیاس‌پذیری خود به خودی** دارد، هر نظیر جدید تقاضا دریافت سرویس را افزایش می‌دهد و به طور همزمان ظرفیت سرویس‌دهی را نیز افزایش می‌دهد.
- نظیرها به صورت متناوب متصل هستند و آدرس IP نیز ممکن است تغییر کند.
- **پیچیدگی مدیریت**
- مثال: به اشتراک‌گذاری فایل نظیر به نظیر (P2P file Sharing)



تبادل داده‌ها بین فرایندها:

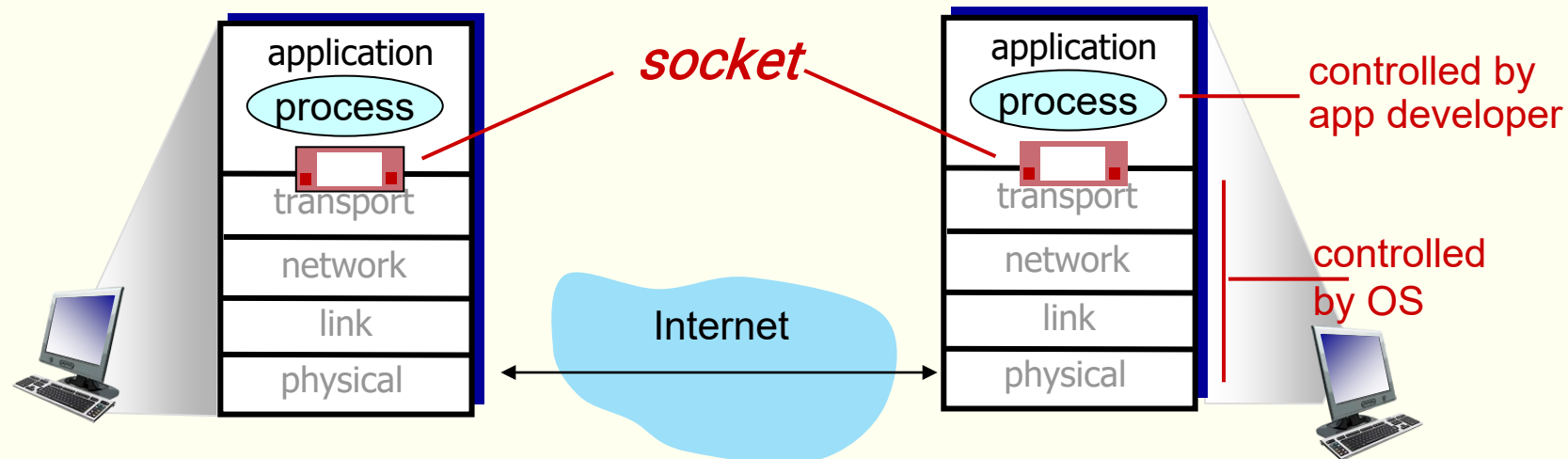
- **فرایند:** برنامه در حال اجرا در یک میزبان
- تبادل داده بین دو فرایند درون یک میزبان از طریق **رد و بدل اطلاعات بین فرایندی** (تعریف شده توسط سیستم عامل) انجام می‌شود.
- تبادل داده بین دو فرایند در میزبان‌های مختلف از طریق **رد و بدل کردن پیام** انجام می‌شود.
- **فرایند سرویس‌گیرنده:** فرایندی است که تبادل داده را شروع می‌کند.
- **فرایند سرویس‌دهنده:** فرایندی است که منتظر است تا با آن تماس گرفته شود. ...

توجه:

در معماری نظیر به نظیر، برنامه‌های کاربردی هم فرایند سرویس‌گیرنده و هم فرایند سرویس‌دهنده دارند.

سوکت‌ها (Sockets):

- **سوکت:** واسطه ارتباطی فرایند برای ارسال یا دریافت پیام (شبهه درب ورود و خروج)
- فرایند ارسال‌کننده از طریق سوکت پیام را به بیرون می‌فرستد (ارسال می‌کند)
- فرایند ارسال‌کننده، برای ارسال پیام، به زیرساخت انتقال داده‌ها وابسته است تا پیام ارسالی از طریق سوکت در مقصد به گیرنده تحویل داده شود.
- برای تبادل پیام، دو سوکت لازم است: یکی در هر طرف



آدرس‌دهی فرایندها:

- برای دریافت پیام‌ها، هر فرایند باید یک **شناسه منحصر بفرد** داشته باشد.
- مثال:
 - HTTP server: ۸۰
 - mail server: ۲۵
- هر میزبان یک **آدرس IP ۳۲ بیتی** منحصر بفرد دارد.
- برای تفکیک فرایندهای در حال اجرا در یک میزبان، هر سوکت یک شناسه به نام **شماره پورت** دارد.
- هر فرایند با یک شناسه منحصر بفرد **دوتایی (آدرس IP میزبان و شماره پورت)** در میزبان معرفی می‌شود.
- برای ارسال پیام به سرویس‌دهنده وب www.aut.ac.ir
 - آدرس IP: ۱۸۵.۲۱۱.۸۸.۱۳۱
 - شماره پورت: ۸۰

لایه کاربرد

موارد زیر در پروتکل لایه کاربرد تعریف می‌شود:

• نوع پیام:

- به عنوان مثال درخواست یا پاسخ

• فرمت پیام:

- چه فیلدهایی در پیام وجود دارد و چگونه این فیلدهای تشخیص داده می‌شوند.

• مفهوم پیام:

- معنی و مفهوم اطلاعات هر فیلد

• عملیات:

- چه پردازش‌هایی باید انجام شود و چه زمانی فرایندها باید پیام‌های درخواست یا پاسخ را ارسال کنند.

• پروتکل‌های عمومی (باز):

- در RFCها تعریف شده و هر فردی به آن‌ها دسترسی دارد.
- امکان همکاری بین برنامه‌های کاربردی توسعه یافته توسط تأمین‌کننده‌های مختلف را فراهم می‌کند.
- به عنوان مثال: پروتکل‌های HTTP و SMTP

• پروتکل‌های اختصاصی:

- به عنوان مثال: اسکایپ

نیازمندی‌های کیفیت سرویس‌دهی:

• درستی داده‌ها (Data Integrity):

- بعضی از برنامه‌های کاربردی مانند ایمیل، انتقال فایل و وب نیاز به قابلیت اطمینان صد در صدی برای انتقال داده‌ها دارند.
- بعضی برنامه‌های کاربردی دیگر مانند انتقال صوت یا ویدیو تا اندازه‌ای از دست دادن داده‌ها را می‌توانند تحمل کنند.

• تأخیر و زمان‌بندی (Delay & Timing):

- بعضی از برنامه‌های کاربردی مانند تلفن اینترنتی و بازی‌های تعاملی یک حداکثر تأخیری را می‌توانند تحمل کند.
- در بعضی از برنامه‌های کاربردی مانند جریان‌سازی ویدیو داده‌ها دریافتی باید با همان زمان‌بندی ارسال، بازپخش شوند.

• گذردهی (Throughput):

- بعضی از برنامه‌های کاربردی مانند برنامه‌های چندرسانه‌ای نیاز به یک حداقل گذردهی دارند.
- بعضی برنامه‌های کاربردی دیگر مانند انتقال فایل با هر میزان گذردهی (در دسترس) کار می‌کنند.

• امنیت (Security):

• سه اصل امنیت اطلاعات:

- محرمانگی (Confidentiality)
- درستی داده‌ها (Integrity)
- در دسترس بودن (Availability)

• راه‌حل‌های مقابله با تهدیدات امنیتی:

- رمزنگاری (Encryption)، تصدیق هویت (Authentication)، و ...
- سطح امنیت مورد انتظار به کاربرد بستگی دارد.

لایه کاربرد

نیازمندی کیفیت انتقال پیام‌ها:

برنامه کاربردی	تحمل‌پذیری از دست‌دادن داده‌ها	گذردهی	حساسیت به تأخیر
انتقال فایل	۱۰۰% مطمئن	انعطاف‌پذیر	خیر
ایمیل	۱۰۰% مطمئن	انعطاف‌پذیر	خیر
وب	۱۰۰% مطمئن	انعطاف‌پذیر	خیر
صوت یا ویدیو real time	تا حدودی	صوت: 5 Kbps تا 1 Mbps ویدیو: 10 Kbps تا 5 Mbps	بله، در حد چند ده میلی‌ثانیه
جریان‌سازی صوت یا ویدیو	تا حدودی	صوت: 5 Kbps تا 1 Mbps ویدیو: 10 Kbps تا 5 Mbps	بله، در حد چند ثانیه
بازی‌های تعاملی	تا حدودی	چند صد کیلو بیت بر ثانیه	بله، در حد چند ده میلی‌ثانیه
پیام‌رسان متنی	۱۰۰% مطمئن	انعطاف‌پذیر	بله و خیر

لایه کاربرد

کیفیت سرویس دهی:

- مسئول تأمین کیفیت سرویس دهی پروتکل لایه کاربرد است.
- پروتکل لایه کاربرد برای ارسال پیام های خود از سرویس لایه انتقال استفاده می کند.
- پروتکل TCP یا پروتکل UDP
- کیفیت سرویس دهی وابسته به کارایی شبکه دارد.
- تأخیر انتها به انتها
- قابلیت اطمینان
- گذردهی

آیا سرویس لایه انتقال می توانند همه نیازمندی های کیفیت سرویس دهی لایه کاربرد را تأمین کند؟

سرویس پروتکل‌های انتقال اینترنت:

• سرویس TCP:

• اتصال‌گرا (Connection-oriented)

• برقراری ارتباط بین ارسال‌کننده و دریافت‌کننده

• انتقال مطمئن (Reliable)

• انتقال بدون خطا (مطمئن) بین ارسال‌کننده و دریافت‌کننده

• کنترل جریان (Flow Control)

• کنترل نرخ ارسال فرستنده توسط گیرنده برای از بین نرفتن داده‌ها در گیرنده

• کنترل ازدحام (Congestion Control)

• کنترل نرخ ارسال فرستنده برای از بین بردن ازدحام در شبکه

• عدم ارائه:

• عدم تضمین حداقل گذردهی، حداقل تأخیر (زمان‌بندی) و

امنیت

• سرویس UDP:

• بدون اتصال (Connectionless)

• ارسال پیام بدون برقراری ارتباط بین ارسال‌کننده و دریافت‌کننده

• انتقال غیرمطمئن (Unreliable)

• انتقال غیرمطمئن (با احتمال از دست‌دادن) بین ارسال‌کننده و دریافت‌کننده

• عدم ارائه:

• عدم انجام کنترل خطا (ارسال مطمئن)، کنترل جریان، کنترل ازدحام و ایجاد ارتباط و همچنین عدم تضمین حداقل گذردهی، حداقل تأخیر (زمان‌بندی) و امنیت

• سؤال:

• پروتکل UDP به چه دردی می‌خورد؟!

لایه کاربرد

سرویس پروتکل‌های انتقال اینترنت:

برنامه کاربردی	پروتکل لایه کاربرد	پروتکل لایه انتقال
انتقال فایل	FTP [RFC 959]	TCP
ایمیل	SMTP [RFC 5321]	TCP
وب	HTTP 1.1 [RFC 7320]	TCP
تلفن اینترنتی	SIP [RFC 3261], RTP [RFC 3550], or proprietary	TCP یا UDP
جریان‌سازی صوت یا ویدیو	HTTP [RFC 7320], DASH	TCP
بازی‌های تعاملی	WOW, FPS (proprietary)	TCP یا UDP

امن سازی پروتکل TCP (Securing TCP):

• سوکت های TCP و UDP:

• بدون رمزنگاری

• گذرواژه ها متن معلوم (Cleartext) لایه کاربرد به صورت متن معلوم در سوکت از شبکه اینترنت عبور می کند.

• امنیت لایه انتقال (TLS - Transport Layer Security)

• اتصال TCP رمز شده

• تصدیق هویت

• پیاده سازی TLS در لایه کاربرد:

• استفاده از کتابخانه TLS توسط برنامه های کاربردی

• استفاده از TCP توسط TLS

• API سوکت TLS:

• عبور متن معلوم به صورت رمز شده از شبکه اینترنت

سرویس وب و پروتکل HTTP:

بررسی سریع اولیه:

- یک صفحه وب حاوی تعدادی شیء وب است که می‌توانند در سرویس‌دهنده‌های وب مختلف ذخیره شده باشند.
- هر شیء وب می‌تواند یک فایل HTML، تصویر JPEG، جاوا اپلت، فایل صوتی و ... باشد.
- هر صفحه وب شامل یک فایل HTML پایه (base HTML) است که به شیء‌های وب دیگر داخل آن صفحه وب ارجاع می‌دهد. هر شیء وب توسط URL آدرس‌دهی می‌شود. به عنوان مثال:

www.someschool.edu/someDept/pic.gif

host name

path name

HTML: HyperText Markup Language

URL: Uniform Resource Locator

لایه کاربرد

مثال: یک صفحه وب با شیءهای متعدد

The screenshot displays the IRNA (Islamic Republic News Agency) website. The header includes the IRNA logo, the text 'The Islamic Republic News Agency', and a navigation bar with categories like 'سیاست' (Politics), 'اقتصاد' (Economy), 'فرهنگ' (Culture), 'علم و آموزش' (Science and Education), 'ورزش' (Sports), 'تهران بزرگ' (Greater Tehran), 'استان‌ها' (Provinces), 'جهان' (World), 'پژوهش' (Research), 'عکس' (Photos), 'فیلم' (Movies), 'زندگی' (Life), 'گرافیک' (Graphics), and 'سایر' (Others). The main content area features a large banner image of a green combine harvester in a golden field, with the headline 'خرید گندم در کشور از ۱۰ میلیون تن گذشت' (Wheat purchase in the country exceeded 10 million tons). Below the banner, there are several smaller news items, including 'آسیای غربی | بیش از ۸۰۰۰ شهید در غزه / حملات موشکی به تل آویو و کمین مبارزان فلسطینی در بیت‌لاهی' (Western Asia | More than 8,000 martyrs in Gaza / Missile attacks on Tel Aviv and Palestinian fighters in Beit Lahia), 'سیاست | پرویز قنات رئیس ستاد اجرایی فرمان امام شد' (Politics | Proiz Qanat became the head of the Executive Office of the Imam's Order), and 'جهان | اعتراف وزیر امنیت رژیم اسرائیل به خیالی بودن بازدارندگی در برابر مقاومت' (World | Israeli Minister of Defense admits the deterrence against resistance is imaginary). A sidebar on the left titled 'اخبار برگزیده' (Selected News) lists various topics such as 'ایران جزو کشورهای برتر در صنعت گاو شیری در دنیا است' (Iran is among the top countries in the world in the dairy industry), 'جنگ غزه باعث اختلاف در احزاب چپ در اروپا' (Gaza war causes differences in left-wing parties in Europe), and 'کاهش قیمت انواع سکه و طلا' (Decrease in the price of various coins and gold). The bottom of the page shows a URL: 'https://www.irna.ir/news/852740717'. The footer contains the text 'خرید گندم در کشور از ۱۰ میلیون تن گذشت' (Wheat purchase in the country exceeded 10 million tons) and 'شهادت ۱۱۶ نفر از کارکنان پزشکی غزه' (Martyrdom of 116 medical workers in Gaza).

مرور کلی پروتکل HTTP:

Hypertext Transfer Protocol

- پروتکل لایه کاربرد برای ارائه سرویس وب

- مدل Client/Server

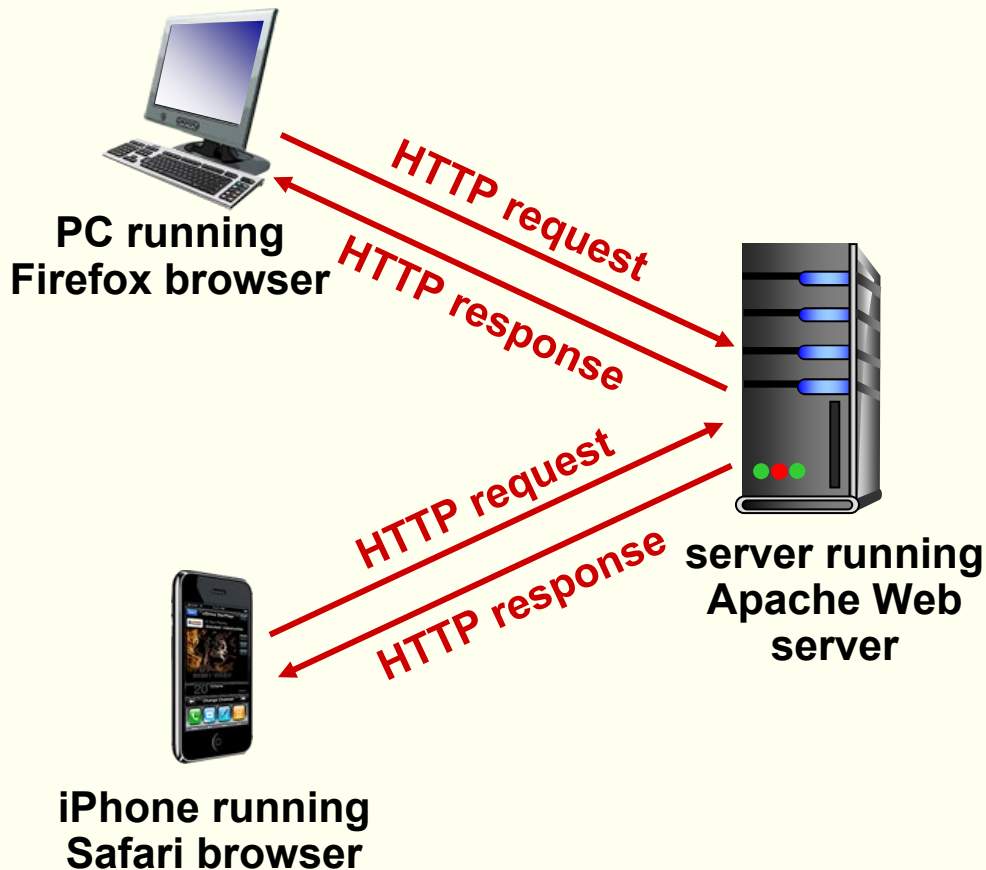
- سرویس گیرنده (Client):

- مرورگر وب با استفاده از پروتکل HTTP درخواست دریافت شیءهای وب را می دهد.

- مرورگر وب شیءهای دریافتی در پاسخ به درخواستهایش را نمایش می دهد.

- سرویس دهنده (Server):

- سرویس دهنده وب با استفاده از پروتکل HTTP در پاسخ به درخواستهای سرویس گیرنده، شیءهای وب را ارسال می کند.



ادامه مرور کلی پروتکل HTTP:

HTTP برای ارسال پیام‌هایش از TCP استفاده می‌کند.

- سرویس‌گیرنده (Client) شروع‌کننده اتصال به سرویس‌دهنده (Server) است.

- پورت قراردادی سرویس‌دهنده پورت ۸۰ است.

- سرویس‌دهنده درخواست برقراری اتصال با سرویس‌گیرنده را می‌پذیرد.

- پیام‌های HTTP (پیام‌های پروتکل لایه کاربرد) بعد از ایجاد اتصال بین مرورگر وب (سرویس‌گیرنده HTTP) و سرویس‌دهنده وب (سرویس‌دهنده HTTP) رد و بدل می‌شود.

- در خاتمه اتصال TCP بسته می‌شود.

HTTP بدون حالت (Stateless) است.

- سرویس‌دهنده (Server) هیچ اطلاعاتی از تاریخچه درخواست‌های سرویس‌گیرنده (Client) نگهداری نمی‌کند.

توضیح

پروتکل‌هایی که حالت (State) را نگهداری می‌کنند، پیچیده هستند!

- تاریخچه گذشته (حالت) باید نگهداری شود (هزینه نگهداری بالا و مقیاس‌پذیر نبودن)

- اگر سرویس‌گیرنده یا سرویس‌دهنده از کار بیفتند، ممکن است "حالت" در نظر گرفته شده توسط آن‌ها یکسان نباشد و نیاز به تطبیق (یکسان‌سازی) داشته باشند.

دو نوع اتصالات HTTP:

اتصال HTTP غیرمداوم (Non-Persistent HTTP)

۱- باز شدن اتصال TCP با سرویس دهنده

۲- درخواست دریافت فقط یک شیء و دریافت آن

۳- بسته شدن اتصال TCP

– دریافت چند شیء نیاز به چند اتصال TCP دارد.

– اتصال های TCP می توانند به صورت موازی ایجاد شوند.

– تعداد اتصال های TCP موازی با توجه به منابع سیستم و همچنین محدودیت سرویس دهنده محدود است.

اتصال HTTP مداوم (Persistent HTTP)

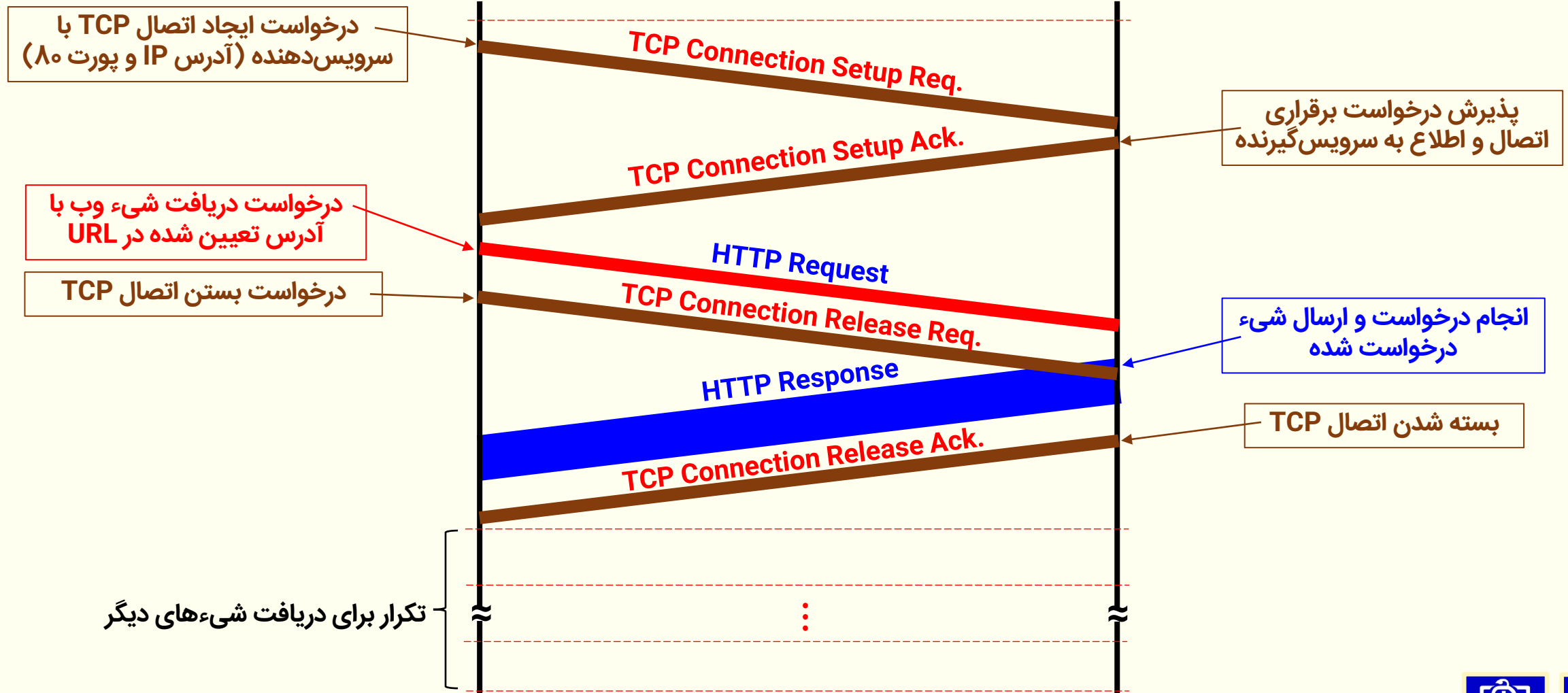
۱- باز شدن اتصال TCP با سرویس دهنده

۲- ارسال چند درخواست و دریافت شیء های مربوطه بر روی یک اتصال TCP

۳- بسته شدن اتصال TCP

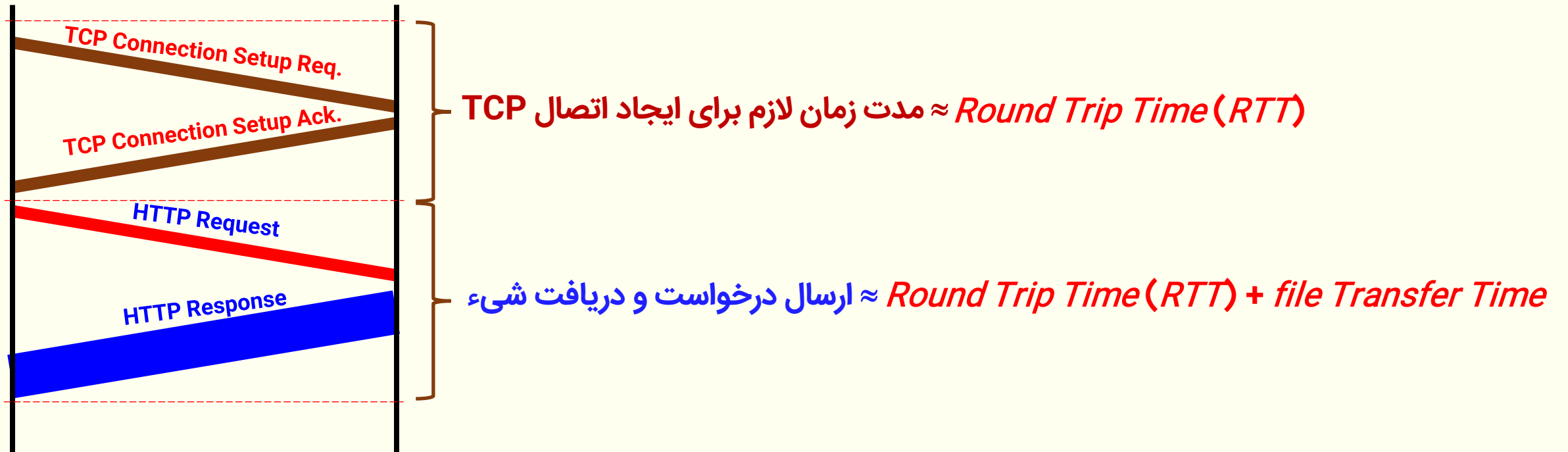
- درخواست ها می توانند پشت سرهم و قبل از دریافت پاسخ درخواست های قبلی ارسال شوند (خط لوله – Pipeline)
- استفاده از خط مشی خط لوله با برنامه کاربردی است.

اتصال HTTP غیرمداوم:



لایه کاربرد

اتصال HTTP غیرمداوم: مدت زمان دریافت یک شیء



$$t_{object} = 2RTT + t_{file}$$

لایه کاربرد

اتصال HTTP غیرمداوم: زمان پاسخ دریافت یک صفحه وب (با $N+1$ شیء)
(بدون در نظر گرفتن زمان ارسال سگمنت‌های کنترلی TCP و پیام‌های درخواست و پاسخ HTTP)

• اتصال HTTP غیرمداوم بدون اتصال موازی

$$t_{page} = (N + 1) \times 2RTT$$

• اتصال HTTP غیرمداوم با اتصال موازی (نا محدود)

$$t_{page} = 4RTT$$

• اتصال HTTP غیرمداوم با اتصال موازی محدود (K)

$$t_{page} = \left(\left\lceil \frac{N}{K} \right\rceil + 1 \right) \times 2RTT$$

اتصال HTTP مداوم (پروتکل HTTP/1.1):

اتصال HTTP مداوم (HTTP/1.1)

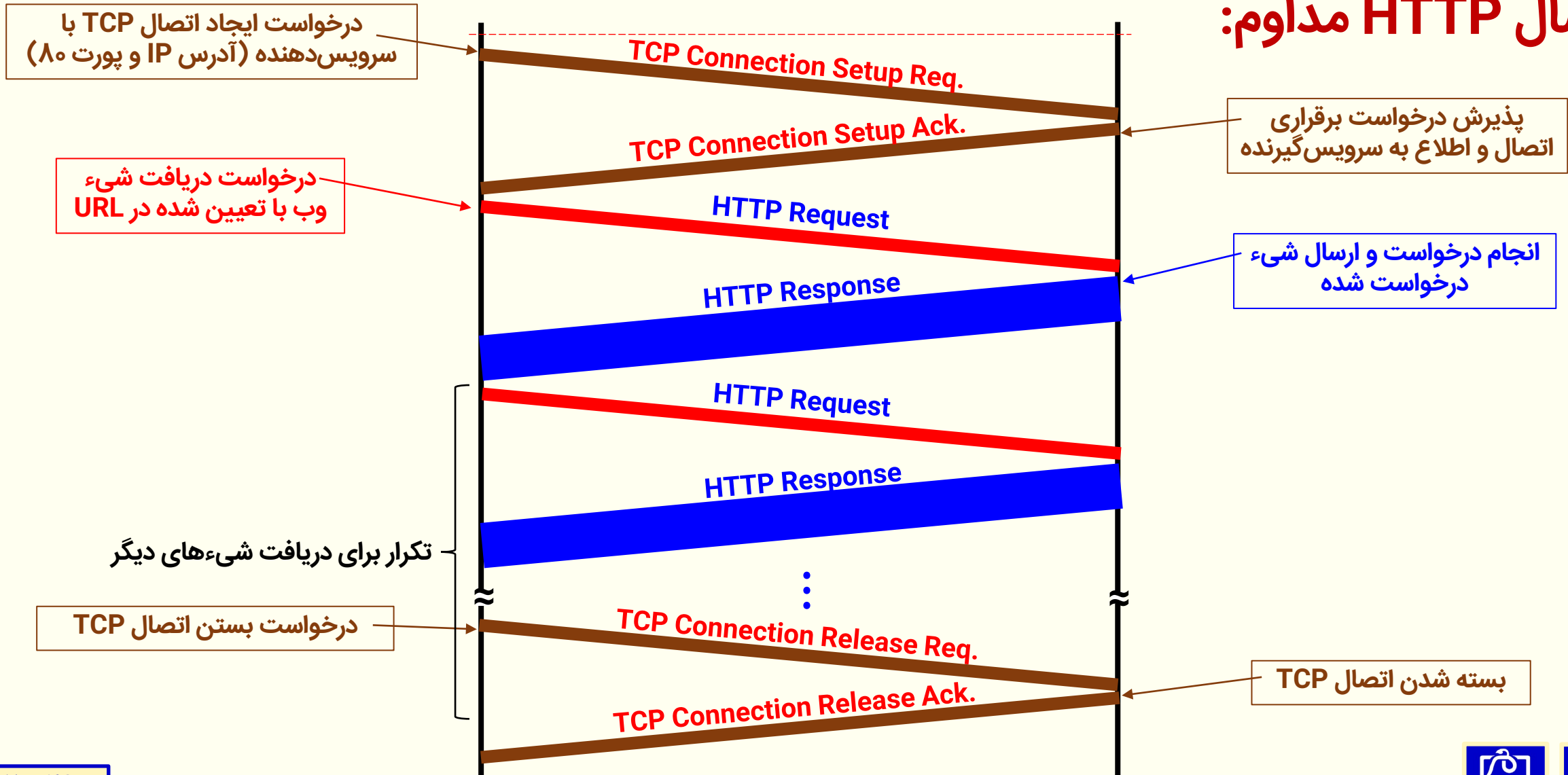
- بعد از باز شدن اتصال TCP بین سرویس گیرنده و سرویس دهنده اتصال برای ارسال و دریافت پیام های HTTP بعدی باز می ماند.
- بدلیل اینکه اتصال بسته نمی شود، ارسال درخواست ها و دریافت پاسخ های بعدی به اندازه یک زمان رفت و برگشت (RTT) سریع تر هستند.
- سرویس گیرنده می تواند درخواست های خود را پشت سرهم ارسال و پاسخ های آن را پشت سرهم دریافت نماید (به صورت خط لوله (pipelining)).

مسائل اتصال HTTP غیرمداوم

- نیاز به 2RTT زمان به ازای هر شیء
- سربار سیستم عامل به ازای هر اتصال TCP
- مرورگرها معمولاً چندین اتصال TCP همزمان برای دریافت موازی شیء ها ارجاع شده باز می کنند.

لایه کاربرد

اتصال HTTP مداوم:



لایه کاربرد

اتصال HTTP مداوم: زمان پاسخ دریافت یک صفحه وب (با $N+1$ شیء)
(بدون در نظر گرفتن زمان ارسال سگمنت‌های کنترلی TCP و پیام‌های درخواست و پاسخ HTTP)

• اتصال HTTP مداوم بدون خط لوله (Pipelining)

$$t_{page} = (N + 2)RTT$$

• اتصال HTTP مداوم با خط لوله (Pipelining)

$$t_{page} = 3RTT$$

پیام‌های پروتکل HTTP:

• انواع پیام‌ها:

- درخواست: پیامی که از طرف سرویس‌گیرنده برای سرویس‌دهنده ارسال می‌شود.
- پاسخ: پیامی که از طرف سرویس‌دهنده برای سرویس‌گیرنده ارسال می‌شود.

• متن قابل خواندن با کد اسکی (ASCII)

• مثال:

Request

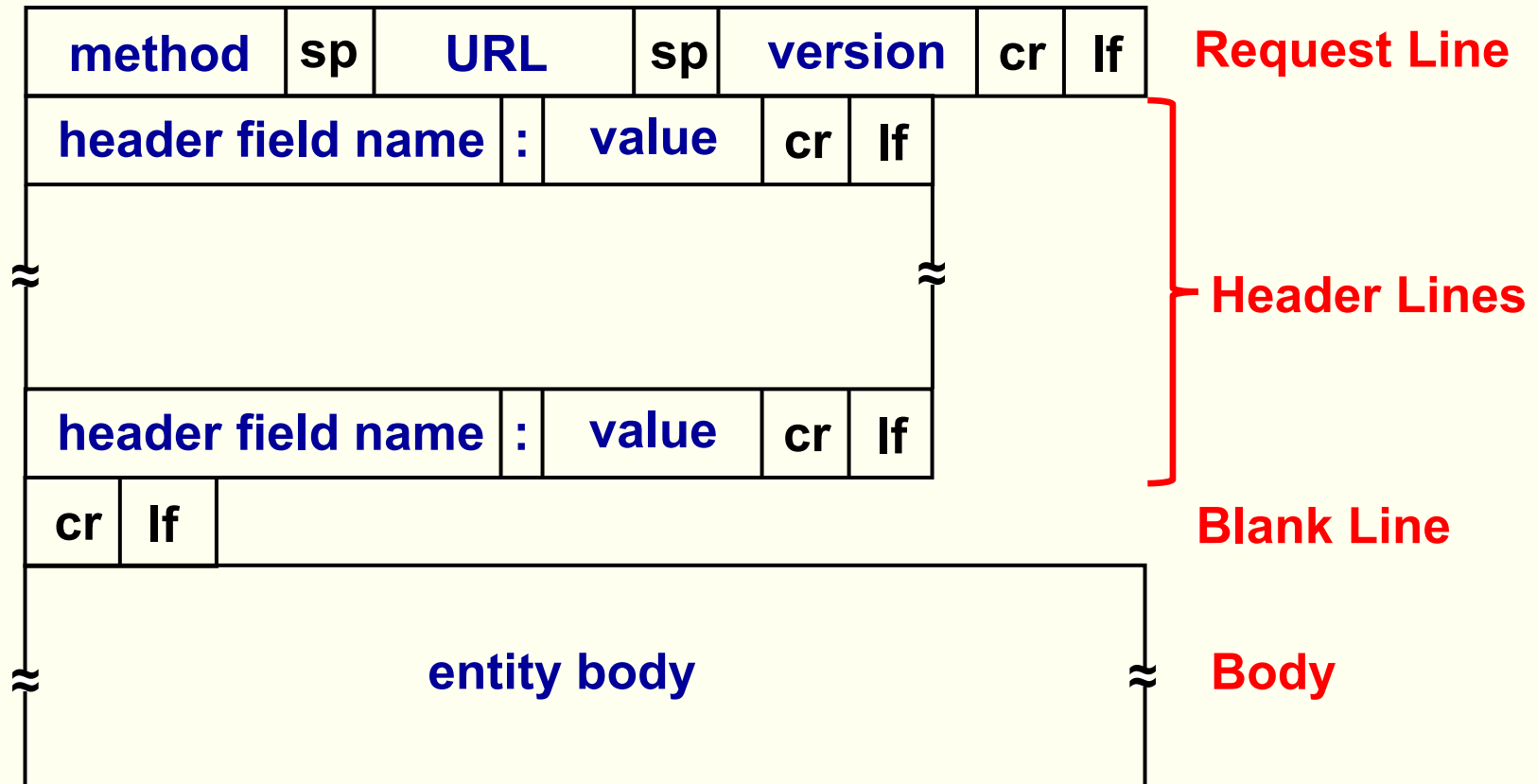
```
GET /index.html HTTP/1.1
Host: www-net.cs.umass.edu
User-Agent: Firefox/3.6.10
Accept: text/html,application/xhtml+xml
Accept-Language: en-us,en;q=0.5
Accept-Encoding: gzip,deflate
Accept-Charset: ISO-8859-1,utf-8;q=0.7
Keep-Alive: 115
Connection: keep-alive
```

Response

```
HTTP/1.1 200 OK
Date: Sun, 26 Sep 2010 20:09:20 GMT
Server: Apache/2.0.52 (CentOS)
Last-Modified: Tue, 30 Oct 2007 17:00:02 GMT
ETag: "17dc6-a5c-bf716880"
Accept-Ranges: bytes
Content-Length: 2652
Keep-Alive: timeout=10, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=ISO-8859-1

data data data data data ...
```

فرمت پیام درخواست HTTP:



sp: space
cr: carriage return
lf: line-feed

فرمت پیام درخواست HTTP: نمونه

request line (GET, POST,
HEAD commands)

header lines

carriage return, line feed at
start of line indicates end
of header lines

```
GET /index.html HTTP/1.1\r\n
Host: www-net.cs.umass.edu\r\n
User-Agent: Firefox/3.6.10\r\n
Accept: text/html,application/xhtml+xml\r\n
Accept-Language: en-us,en;q=0.5\r\n
Accept-Encoding: gzip,deflate\r\n
Accept-Charset: ISO-8859-1,utf-8;q=0.7\r\n
Keep-Alive: 115\r\n
Connection: keep-alive\r\n
\r\n
```

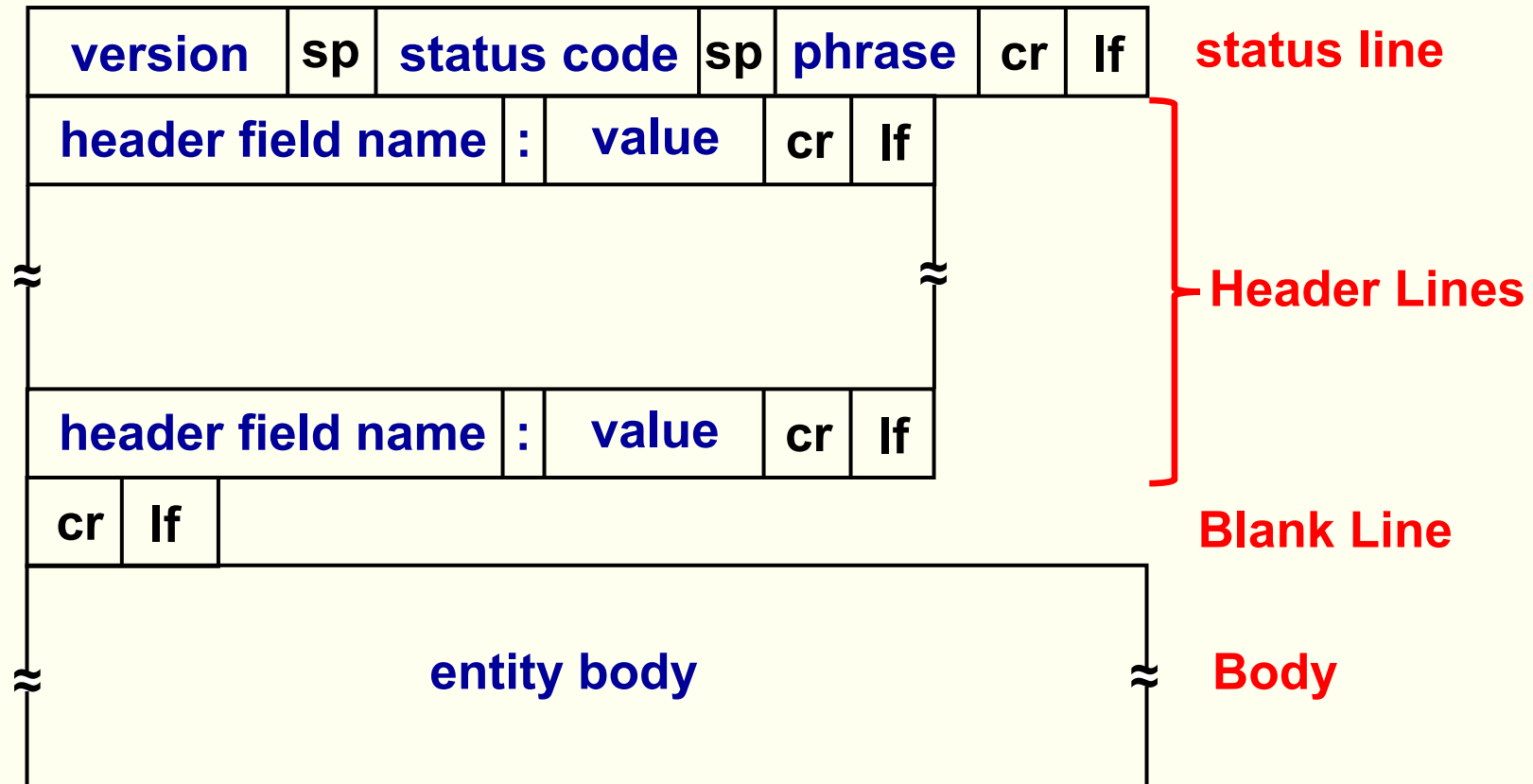
carriage return character
line-feed character

لایه کاربرد

متدهای پیام درخواست HTTP:

نام متد	شرح
GET	از طریق متد GET، سرویس گیرنده درخواست دریافت یک شیء با آدرس URL مشخص شده در پیام درخواست را می کند و سرویس دهنده در پیام پاسخ در بخش بارحمل شده (payload or entity body) شیء (فایل) مورد نظر را ارسال می کند.
POST	از طریق متد POST، سرویس گیرنده می تواند اطلاعاتی (نظیر پر کردن فرم) را برای سرویس دهنده ارسال کند. این اطلاعات در بخش بار حمل شده (payload or entity body) پیام درخواست قرار می گیرند. روش دیگر ارسال اطلاعات برای سرویس دهنده استفاده از متد GET و قرار دادن اطلاعات ارسالی در ادامه URL بعد از علامت "?" است. به عنوان مثال: www.somesite.com/animalsearch?monkeys&banana
HEAD	با استفاده از متد GET سرویس گیرنده می تواند درخواست دریافت یک شیء را بدهد. متد HEAD مشابه متد GET است با این تفاوت که در پیام پاسخ فقط سرآیند (header) پاسخ توسط سرویس دهنده ارسال می شود.
PUT	با استفاده از متد PUT سرویس گیرنده می تواند یک شیء (فایل) را در سرویس دهنده بارگذاری کند. اگر فایل با URL مشخص شده در درخواست در سرویس دهنده وجود داشته باشد، فایل جدید که در بخش بارحمل شده (payload or entity body) پیام درخواست است، جایگزین فایل موجود می شود.

فرمت پیام پاسخ HTTP:



sp: space
cr: carriage return
lf: line-feed

فرمت پیام پاسخ HTTP: نمونه

status line (protocol status code status phrase)	→	HTTP/1.1 200 OK\r\n
header lines	{	Date: Sun, 26 Sep 2010 20:09:20 GMT\r\n
		Server: Apache/2.0.52 (CentOS)\r\n
		Last-Modified: Tue, 30 Oct 2007 17:00:02 GMT\r\n
		ETag: "17dc6-a5c-bf716880"\r\n
		Accept-Ranges: bytes\r\n
		Content-Length: 2652\r\n
		Keep-Alive: timeout=10, max=100\r\n
		Connection: Keep-Alive\r\n
		Content-Type: text/html; charset=ISO-8859-1\r\n
		\r\n
data, e.g., requested HTML file	→	data data data data data ...

کدهای وضعیت پیام پاسخ HTTP:

کد وضعیت	مفهوم	شرح
1xx	اطلاعاتی (Informational)	درخواست دریافت شد، فرایند ادامه دارد.
2xx	موفقیت آمیز (Successful)	درخواست با موفقیت دریافت، درک و پذیرفته شد.
3xx	تغییر جهت (Redirection)	برای تکمیل درخواست باید اقدامات بیشتری انجام شود.
4xx	خطا سرویس گیرنده (Client Error)	فرمت پیام درخواست دارای اشکال است یا قابل انجام نیست.
5xx	خطا سرویس دهنده (Server Error)	سرویس دهنده قادر به انجام درخواست به ظاهر معتبر نیست.

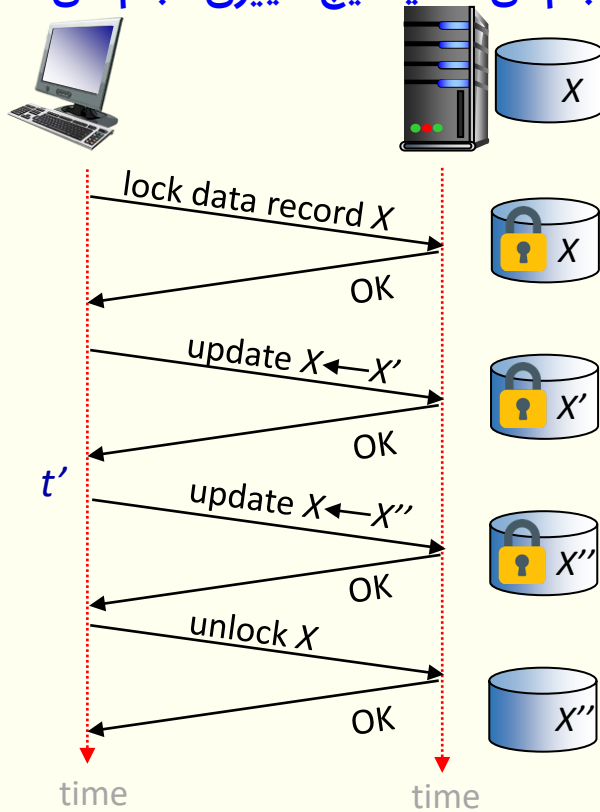
کدهای وضعیت پیام پاسخ HTTP: مثال

کد وضعیت	شرح
100 Continue	بخش اولیه درخواست دریافت شد و سرویس دهنده بعد از دریافت درخواست کامل آن را انجام داده و پاسخ نهایی را ارسال می کند.
200 OK	درخواست موفقیت آمیز انجام شد و بر اساس متد درخواست شیء درخواستی در بخش بار حمل شده (payload) ارسال شده است.
300 Moved Permanently	شیء درخواستی به طور دائمی به آدرس URL جدید منتقل شده است. سرویس گیرنده درخواستش را باید به سرویس دهنده در آدرس جدید بدهد.
404 Not Found	شیء درخواستی توسط سرویس دهنده پیدا نشد.
500 HTTP Version Not Supported	سرویس دهنده از نسخه HTTP که پیام درخواست را تولید کرده پشتیبانی نمی کند و قادر به انجام درخواست نیست.

لایه کاربرد

نگهداری حالت سرویس گیرنده و سرویس دهنده: کوکی (Cookie)

یک پروتکل با نگهداری حالت: سرویس گیرنده دو تغییر در X انجام می دهد یا هیچ تغییری انجام نمی دهد.



پروتکل HTTP یک پروتکل بدون حالت (Stateless) است:

• در پروتکل HTTP رد و بدل کردن پیام های چند مرحله ای برای انجام یک تراکنش تعریف نشده است:

• سرویس گیرنده یا سرویس دهنده نیازی به دنبال کردن وضعیت رد و بدل کردن پیام های چند مرحله ای ندارند.

• درخواست های HTTP مستقل از یکدیگرند

• نیاز به بازیابی تراکنش های ناتمام یا انجام نشده بین سرویس گیرنده و سرویس دهنده وجود ندارد.

سوال: اگر در زمان t' اتصال شبکه قطع شود یا سرویس گیرنده از کار بیفتد چه اتفاقی رخ می دهد؟

نگهداری حالت سرویس گیرنده و سرویس دهنده: کوکی (Cookie)

سرویس گیرنده و سرویس گیرنده وب برای نگهداری حالت (State) می توانند از کوکی استفاده کنند.

مثال:

- ساسان به رستوران زنگ می زند و سفارش غذا می دهد.
- کارمند رستوران نام، تلفن و آدرس ارسال غذا را از ساسان می گیرد و در سامانه رستوران ذخیره کرده و به ساسان یک کد اشتراک می دهد.
- ساسان کد اشتراک دریافتی از آن رستوران را در فایلی در تلفن همراه خود ذخیره می کند.
- در دفعات بعدی برای سفارش غذا، ساسان بعد از تماس، ابتدا کد اشتراک خود را می دهد و بعد سفارش غذا می دهد.
- کارمند رستوران بعد از دریافت کد اشتراک، اطلاعات لازم برای آدرس ارسال غذا را از سامانه دریافت کرده و سفارش ساسان را ارسال می کند.

• کوکی همانند کد اشتراک است که سرویس دهنده در پاسخ به اولین درخواست سرویس گیرنده به آن اختصاص می دهد.

• سرویس دهنده بر اساس شناسه کوکی تاریخچه تراکنش های انجام شده با سرویس گیرنده (حالت) را در پایگاه داده های خود نگهداری می کند.

• مرورگر وب سرویس گیرنده کوکی (کد اشتراک) دریافتی از سرویس دهنده را فایل کوکی ها ذخیره و مدیریت می کند.

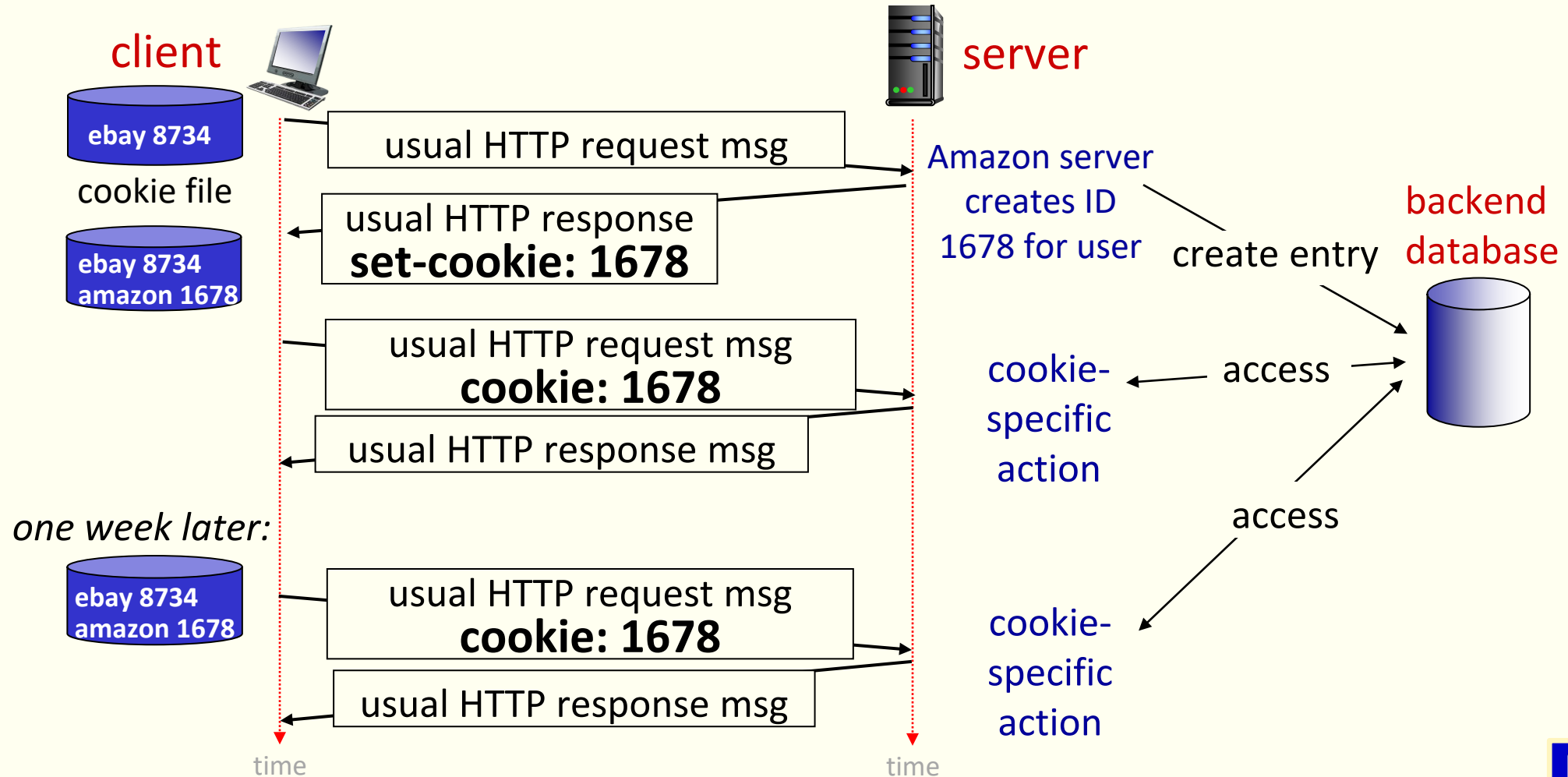
• مرورگر وب سرویس گیرنده در درخواست های بعدی به آن سرویس دهنده شناسه کوکی را در سرآیند پیام درخواستش ارسال می کند.

• سرویس دهنده بر اساس درخواست دریافتی و حالت سرویس گیرنده با شناسه کوکی موجود در سرآیند درخواست، پاسخ را ارسال می کند.

لایه کاربرد

نگهداری حالت سرویس گیرنده و سرویس دهنده: کوکی (Cookie)

مراحل تخصیص و استفاده از کوکی:



کاربردهای استفاده از کوکی (Cookie)

• کوکی برای چه منظورهایی استفاده می‌شود:

• تصدیق هویت

• سبد خرید

• توصیه کردن

• وضعیت نشست کاربر (ایمیل تحت وب)

• ...

• چالش‌ها: چگونگی نگهداری حالت

• نقاط انتهایی پروتکل: نگهداری حالت فرستنده یا گیرنده با

انجام تراکنش‌ها

• کوکی‌ها: پیام‌های HTTP حالت را حمل می‌کنند.

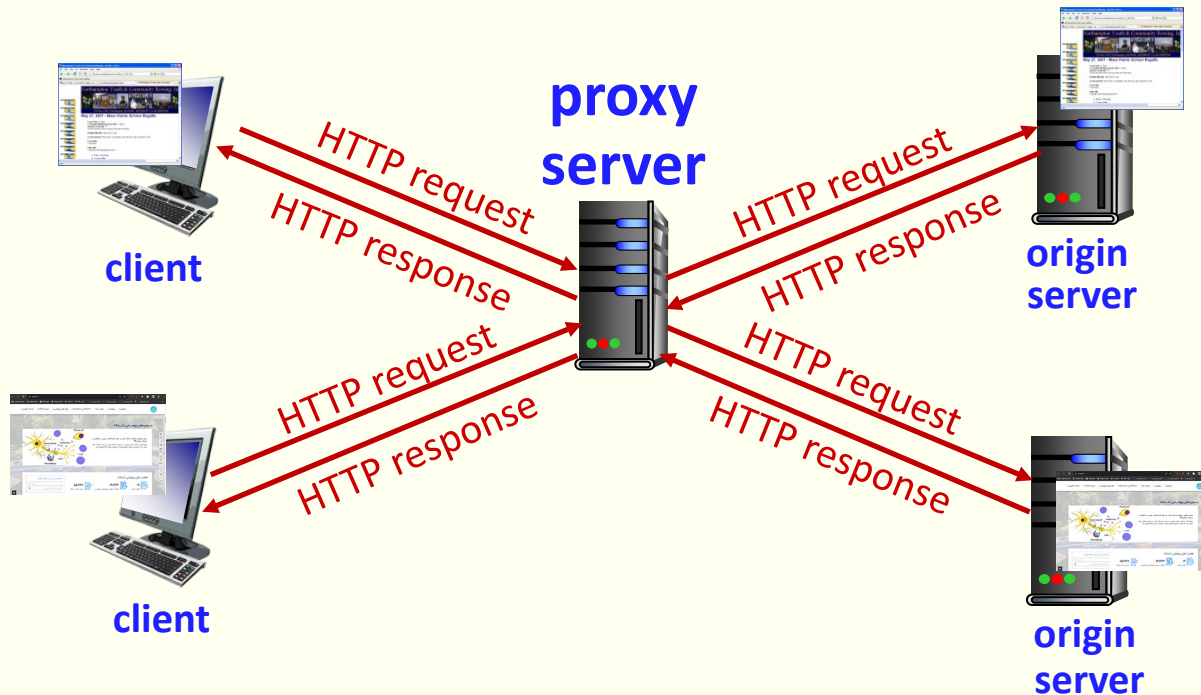
توضیح

کوکی‌ها و حریم خصوصی (Privacy)

• کوکی این اجازه را وب سایت‌ها می‌دهد تا تمام فعالیت‌هایی که شما در وب سایت آن‌ها انجام می‌دهید را بفهمند.

• کوکی‌های ثابت شخص ثالث (کوکی‌های ردیابی) اجازه می‌دهند هویت مشترک (مقدار کوکی) در چندین وب سایت ردیابی شود.

سرویس دهنده های پروکسی (Proxy Servers)



- سرویس دهنده نماینده یا واسط

- ارسال همه درخواست های وب به سرویس دهنده پروکسی

- پاسخ به همه درخواست ها از طریق سرویس دهنده پروکسی

- هدف استفاده از سرویس دهنده پروکسی:

- رضایت مندی سرویس گیرندگان

- کمک به تأمین کیفیت سرویس

- مدیریت درخواست ها

- محل قرارگیری:

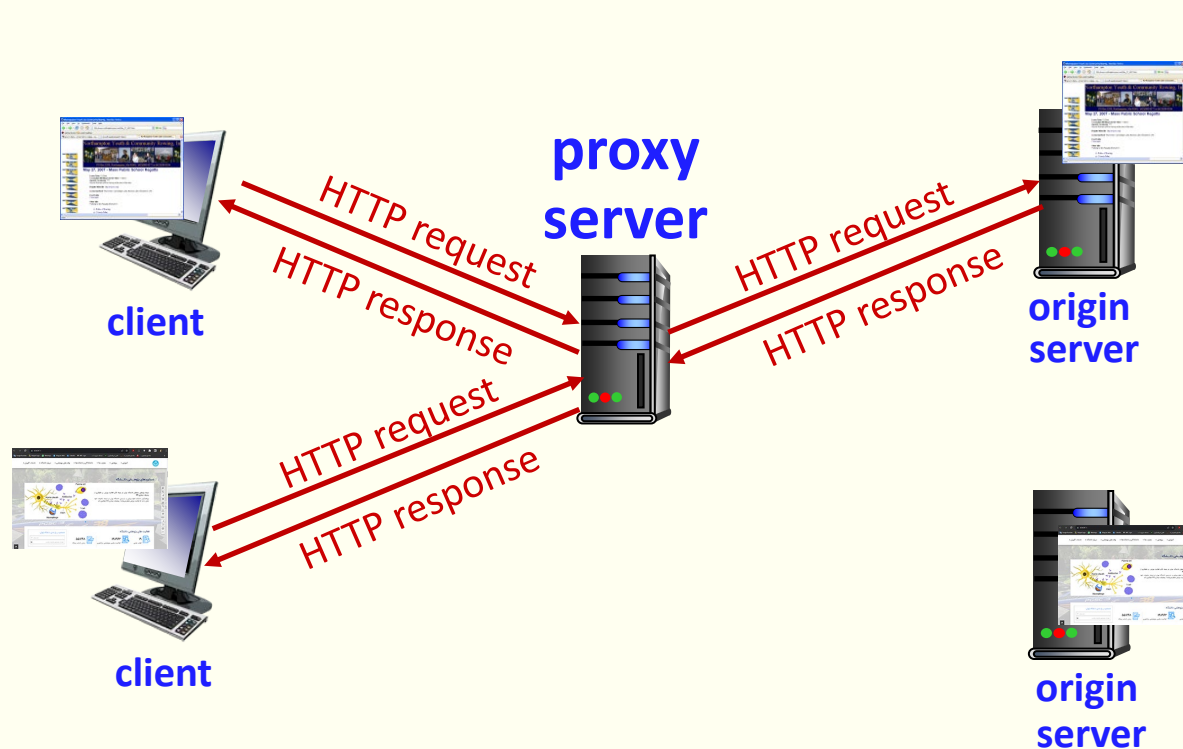
- در شبکه سازمانی (موسسه)، در شبکه دسترسی و ...

- کاربردها:

- حافظه پنهان وب (Web Caching)

- ...

حافظه پنهان وب (Web Cache) – سرویس دهنده های پروکسی (Proxy Servers)



- ذخیره (موقت) پاسخ در حافظه پنهان (Cache) و ارسال پاسخ از حافظه پنهان به درخواست های بعدی
- نحوه انجام:

- انجام پیکربندی لازم برای استفاده از سرویس دهنده پروکسی در مرورگر وب یا سیستم کاربر توسط کاربر
- ارسال همه درخواست های HTTP به سرویس دهنده پروکسی توسط مرورگر وب
- در صورت نبودن شیء درخواستی در حافظه پنهان (Cache):
 - ارسال درخواست به سرویس دهنده اصلی توسط پروکسی
 - ارسال پاسخ به سرویس گیرنده پس از دریافت پاسخ از سرویس دهنده اصلی توسط پروکسی
 - ذخیره شیء دریافتی در حافظه پنهان (Cache)
- در صورت وجود شیء درخواستی در حافظه پنهان (Cache):
 - ارسال پاسخ از حافظه پنهان (Cache) توسط پروکسی

حافظه پنهان وب (Web Cache) – سرویس‌دهنده‌های پروکسی (Proxy Servers)

- سرویس‌دهنده حافظه پنهان (پروکسی) هم به عنوان سرویس‌گیرنده و هم به عنوان سرویس‌دهنده عمل می‌کند:

- سرویس‌دهنده برای سرویس‌گیرنده اصلی درخواست‌کننده

- سرویس‌گیرنده برای سرویس‌دهنده مبدأ اصلی

- معمولاً سرویس‌دهنده حافظه پنهان (پروکسی) توسط تأمین‌کنندگان سرویس اینترنت (شرکت‌ها، دانشگاه‌ها، تأمین‌کنندگان سرویس اینترنت برای منزل‌ها) نصب می‌شود.

- مزایا:

- کاهش زمان پاسخ

- حافظه پنهان به سرویس‌گیرنده نزدیک‌تر است

- کاهش ترافیک لینک دسترسی به اینترنت شبکه مؤسسه

- پاسخ از طریق پروکسی داخل شبکه مؤسسه داده می‌شود.

- ارائه مؤثرتر محتوای ارائه‌دهندگان محتوای "ضعیف"

- تعداد زیادی حافظه پنهان (Cache) در شبکه اینترنت وجود دارد.

حافظه پنهان وب: مثال

سناریو:

- نرخ ارسال لینک دسترسی: **1.54 Mbps**
- زمان رفت و برگشت (RTT) از مسیر یاب دسترسی به سرور میزبان مبدأ: **۲ ثانیه**
- اندازه شیء وب: **۱۰۰ Kbits** (فرض کنید $1\text{ K} = 1000$)
- متوسط نرخ درخواست از مرورگرها به سرور میزبان مبدأ: **۱۵ درخواست در ثانیه**
- متوسط نرخ داده‌ها به مرورگرها: **۱.۵ Mbps**

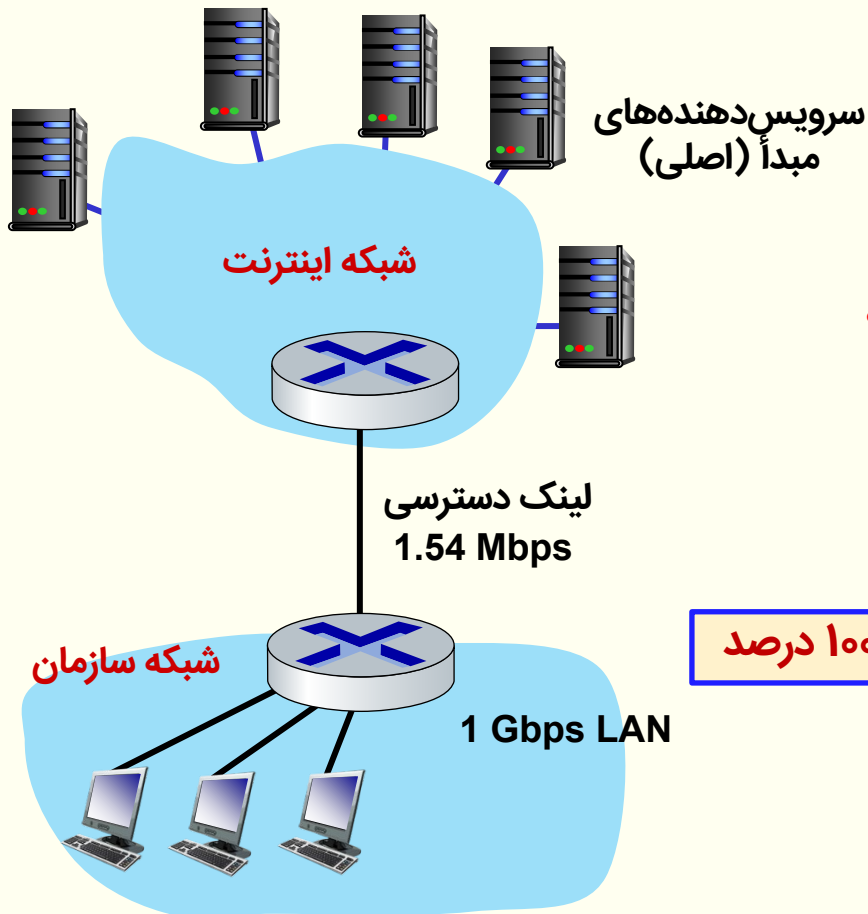
کارایی:

- بهره‌برداری (Utilization) شبکه LAN: **۰.۰۰۱۵**

- بهره‌برداری لینک دسترسی: **۰.۹۷**

مشکل: تأخیر زیاد بدلیل بهره‌برداری نزدیک به ۱۰۰ درصد

$$\begin{aligned} \text{تأخیر LAN} + \text{تأخیر لینک دسترسی} + \text{تأخیر اینترنت} &= \text{تأخیر انتها به انتها} \\ &= 2\text{ Sec} + \text{Minutes} + \mu\text{Secs} \end{aligned}$$



لایه کاربرد

حافظه پنهان وب: مثال - خرید پهنای باند بیشتر

سناریو:

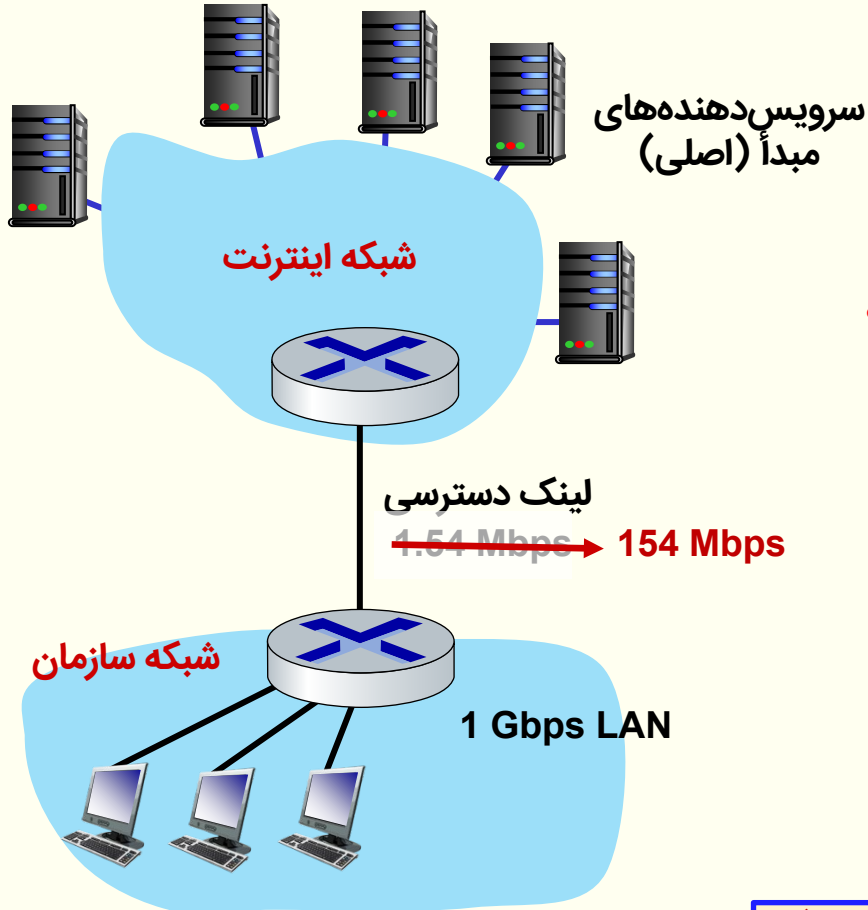
- نرخ ارسال لینک دسترسی: ~~1.54 Mbps~~ ۱۵۴
- زمان رفت و برگشت (RTT) از مسیر یاب دسترسی به سرورهای دهنده مبدأ: ۲ ثانیه
- اندازه شیء وب: ۱۰۰ Kbits (فرض کنید ۱ K = ۱۰۰۰)
- متوسط نرخ درخواست از مرورگرها به سرورهای دهنده مبدأ: ۱۵ درخواست در ثانیه
- متوسط نرخ داده‌ها به مرورگرها: ۱.۵ Mbps

کارایی:

- بهره‌برداری (Utilization) شبکه LAN: ۰.۰۰۱۵
- بهره‌برداری لینک دسترسی: ~~۰.۹۷~~ ۰.۰۰۹۷

$$\begin{aligned} \text{تأخیر LAN} + \text{تأخیر لینک دسترسی} + \text{تأخیر اینترنت} &= \text{تأخیر انتها به انتها} \\ &= 2 \text{ Sec} + \text{Minutes} + \mu\text{Secs} \\ &\quad \swarrow \searrow \\ &\quad \text{mSecs} \end{aligned}$$

هزینه: لینک دسترسی با پهنای باند بیشتر هزینه بیشتری دارد! (هزینه جاری)



حافظه پنهان وب: مثال - نصب سرویس دهنده حافظه پنهان وب (Web Cache)

سناریو:

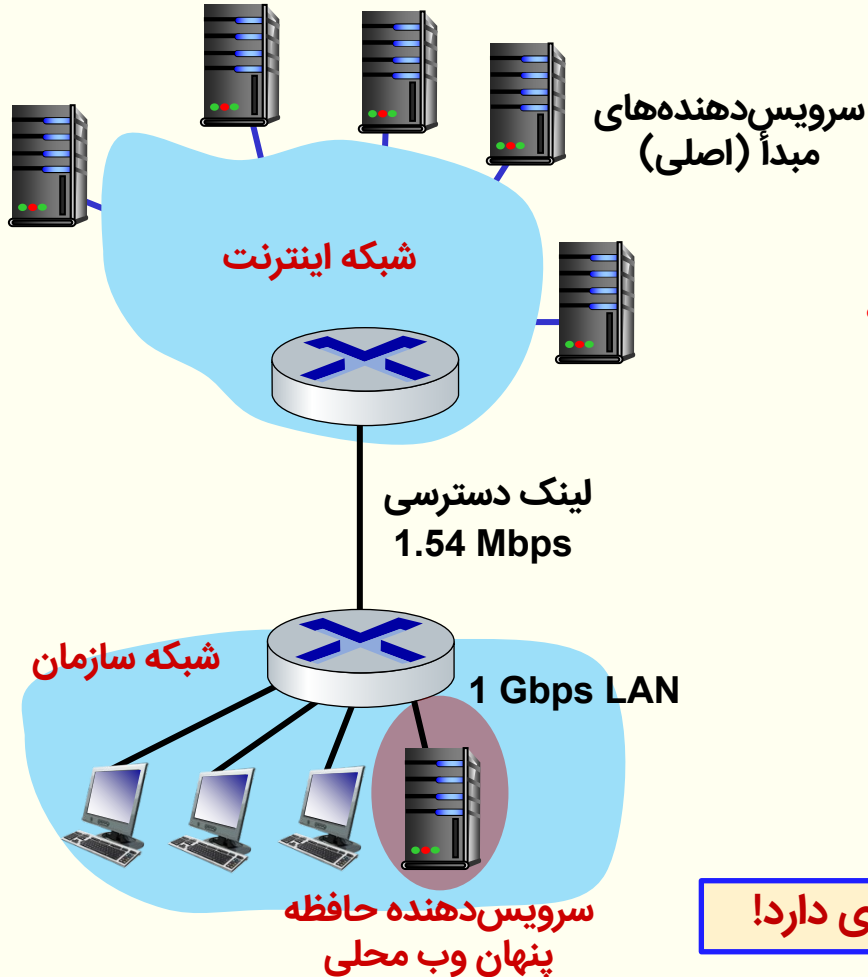
- نرخ ارسال لینک دسترسی: **1.54 Mbps**
- زمان رفت و برگشت (RTT) از مسیر یاب دسترسی به سرویس دهنده مبدأ: **۲ ثانیه**
- اندازه شیء وب: **100 Kbits** (فرض کنید $1 K = 1000$)
- متوسط نرخ درخواست از مرورگرها به سرویس دهنده های مبدأ: **۱۵ درخواست در ثانیه**
- متوسط نرخ داده ها به مرورگرها: **1.5 Mbps**

کارآیی:

- بهره برداری (Utilization) شبکه LAN: ؟
- بهره برداری لینک دسترسی: ؟
- متوسط تأخیر انتها به انتها: ؟

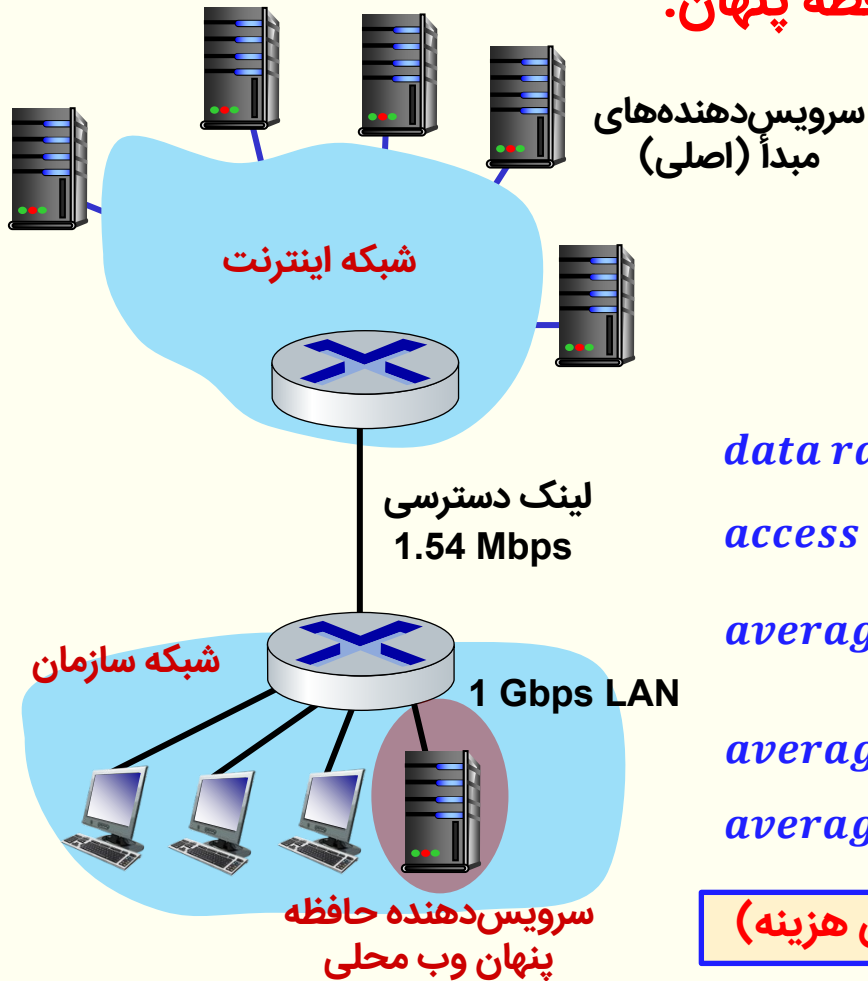
بهره برداری از لینک دسترسی و
تأخیر چگونه محاسبه می شود؟

هزینه: سرویس دهنده حافظه پنهان وب نسبت به پهنای باند هزینه کمتری دارد!



حافظه پنهان وب: مثال - نصب سرویس دهنده حافظه پنهان وب (Web Cache)

محاسبه بهره‌برداری از لینک دسترسی و متوسط تأخیر انتها به انتها با حافظه پنهان:



• فرض کنید نرخ اصابت حافظه پنهان (cache hit rate) ۰.۴ است:

• ۴۰٪ درخواست‌ها از حافظه پنهان پاسخ داده می‌شوند.

• ۶۰٪ درخواست‌ها از سرویس دهنده مبدأ پاسخ داده می‌شوند.

• بار ترافیکی روی لینک دسترسی: ۶۰٪ پاسخ‌ها

$$\text{data rate to browsers over access link} = 0.6 \times 1.54 \text{ Mbps} = 0.9 \text{ Mbps}$$

$$\text{access link utilization} = \frac{0.9}{1.54} = 0.58 \Rightarrow \text{access link delay} \approx \text{mSecs}$$

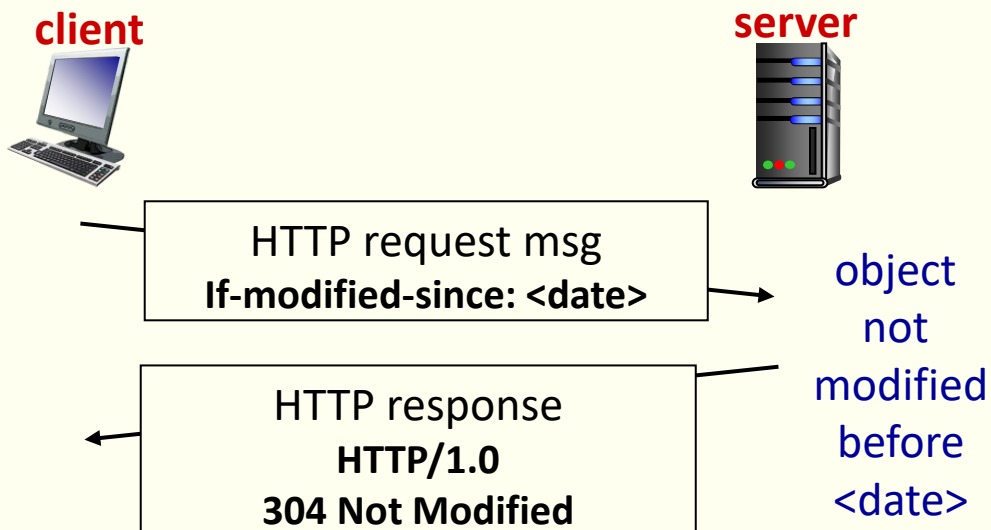
$$\text{average end to end delay} = 0.6 \times (\text{delay from origin servers}) + 0.4 \times (\text{delay when satisfied at cache})$$

$$\text{average end to end delay} = 0.6 \times (2 \text{ Sec} + \text{mSecs} + \mu\text{Secs}) + 0.4 \times (0 + 0 + \mu\text{Secs})$$

$$\text{average end to end delay} \approx 0.6 \times (2.01 \text{ Sec}) + 0.4 \times (\mu\text{Secs}) \approx 1.2 \text{ Sec}$$

متوسط تأخیر انتها به انتها کمتر با لینک دسترسی ۱.۵۴ Mbps (بدون افزایش هزینه)

دریافت مشروط (Conditional GET)



- مشکل استفاده از حافظه پنهان: عدم اطمینان از بروز بودن شیء
- مدت زمان بهره‌برداری از شیء ذخیره شده در حافظه پنهان: محدود در حد چند ساعت
- بروز رسانی شیء ذخیره شده در حافظه پنهان: با دریافت درخواست جدید
- هدر رفت پهنای باند: در صورت عدم تغییر شیء در سرویس‌دهنده مبدأ
- استفاده از دریافت مشروط:

- کاهش تأخیر ارسال
- استفاده کمتر از پهنای باند

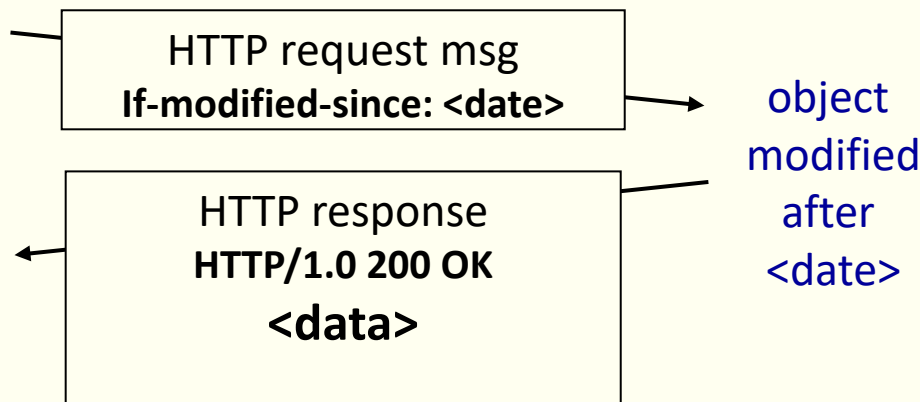
• چگونگی انجام:

- پروکسی حافظه پنهان: تاریخ ذخیره‌سازی در حافظه پنهان در درخواست HTTP مشخص می‌شود.

`If-modified-since: <date>`

- سرویس‌دهنده: اگر تاریخ ذخیره‌سازی در حافظه پنهان بروز بود، در پیام پاسخ شیء مورد درخواست نیست.

`HTTP/1.0 304 Not Modified`



پروتکل HTTP نسخه ۲ (HTTP/2)

هدف اصلی پروتکل HTTP نسخه ۲: کاهش تأخیر درخواست‌های دریافت چند شیء

HTTP نسخه ۱.۱: معرفی درخواست‌های دریافت چند شیء به صورت خط لوله بر روی یک اتصال TCP

- سرویس‌دهنده پاسخ به درخواست‌های دریافت را به ترتیب ارسال می‌کند

- (زمانبندی: FCFS : First Come First Serve)

- با زمانبندی FCFS ممکن است شیء‌های کوچک منتظر ارسال بعد از شیء‌های بزرگ شوند

- (انسداد سر صف – Head-of-line Blocking (HOL)

- بازیابی از دست دادن (باز ارسال سگمنت‌های TCP از دست‌رفته) ارسال شیء را متوقف می‌کند.

پروتکل HTTP نسخه ۲ (HTTP/2)

هدف اصلی پروتکل HTTP نسخه ۲: کاهش تأخیر درخواست‌های دریافت چند شیء

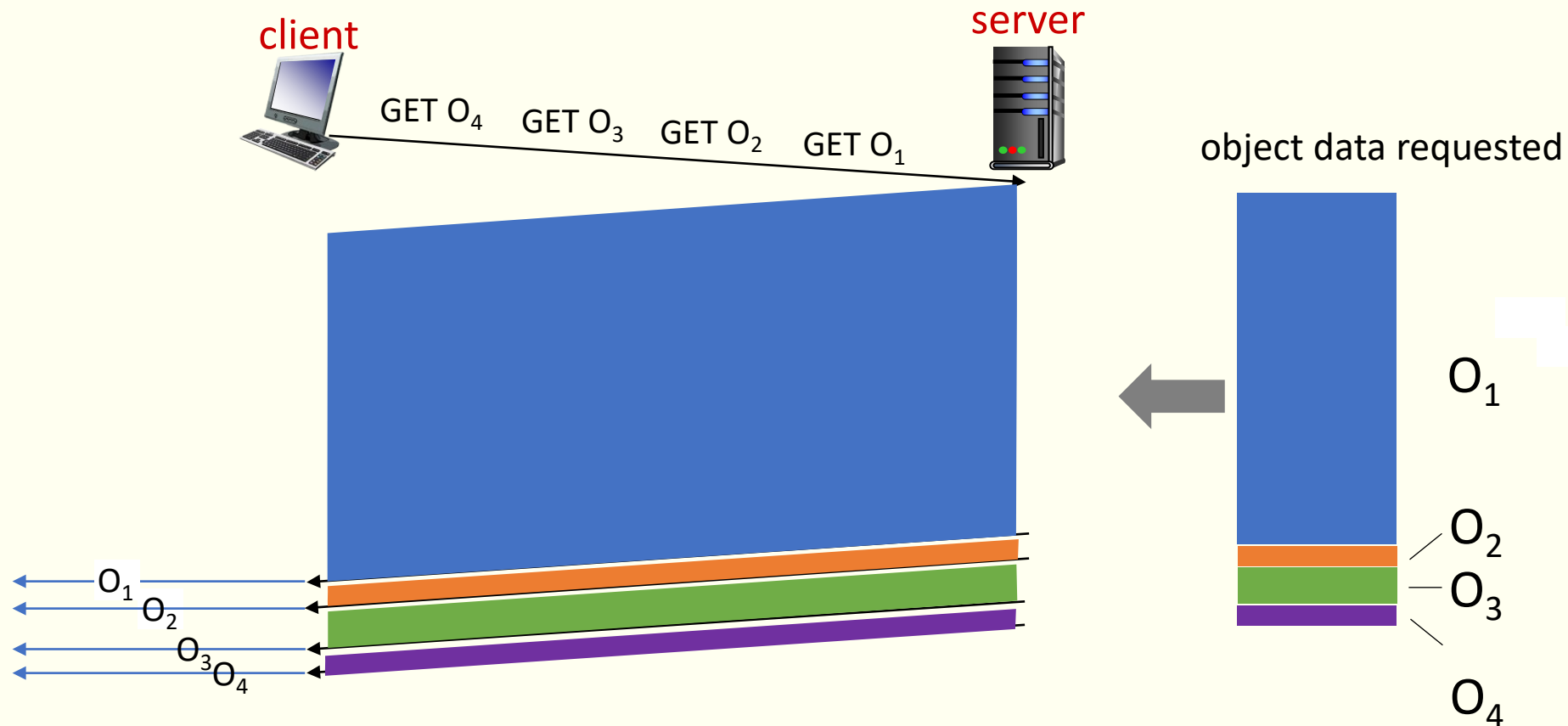
HTTP نسخه ۲: [RFC ۷۵۴۰, ۲۰۱۵] انعطاف‌پذیری برای ارسال شیء‌ها از سرویس‌دهنده به سرویس‌گیرنده را افزایش می‌دهد:

- متدها، کدهای وضعیت و اکثر فیلدهای سرآیند پیام‌ها نسبت به پروتکل HTTP ۱.۱ تغییر نکرده است.
- ترتیب ارسال شیء‌های درخواستی بر اساس اولویت شیء مشخص شده توسط سرویس‌گیرنده است (نه لزوماً FCFS)
- ارسال شیء‌های درخواست نشده به سرویس‌گیرنده (push)
- تقسیم شیء‌ها در قاب‌ها (فریم‌ها) و زمانبندی ارسال قاب‌ها برای کاهش مسئله انسداد سر صف (HOL)

لایه کاربرد

پروتکل HTTP نسخه ۲ (HTTP/2): کاهش اثر انسداد سر صف

HTTP/۱.۱: سرویس‌گیرنده درخواست دریافت یک شیء با اندازه بزرگ (مثلاً یک فایل ویدیویی) و سه شیء با اندازه کوچک را دارد.

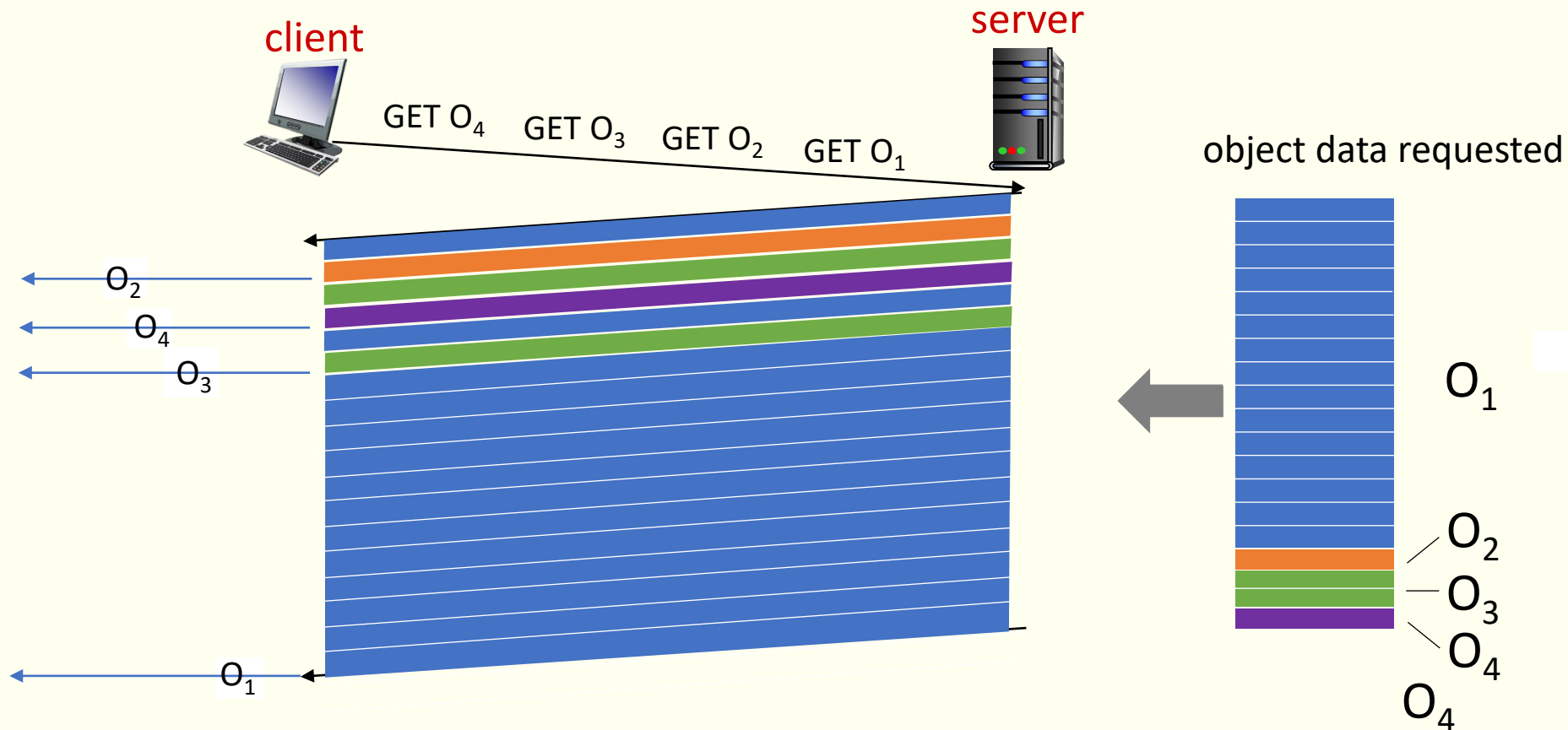


شیء‌ها به ترتیب درخواست شده ارسال می‌شوند: شیء‌های **O_۲**، **O_۳** و **O_۴** با تأخیر پس از شیء **O_۱** ارسال می‌شوند.

لایه کاربرد

پروتکل HTTP نسخه ۲ (HTTP/2): کاهش اثر انسداد سر صف

HTTP/۲: شیء‌ها درون قاب‌ها (فریم‌ها) تقسیم و قاب‌ها لابلائی هم ارسال می‌شوند.



شیء‌های O_2 ، O_3 و O_4 با سرعت ارسال می‌شوند و شیء O_1 اندکی تأخیر دارد.

پروتکل HTTP نسخه ۳ (HTTP/3)

هدف اصلی پروتکل HTTP نسخه ۲: کاهش تأخیر درخواست‌های دریافت چند شیء

HTTP/۲: دریافت شیء‌ها از طریق یک اتصال TCP:

• بازیابی از دست‌دادن بسته هنوز هم ارسال‌های شیء‌ها را متوقف می‌کند.

• مانند HTTP ۱.۱، مرورگرها برای کاهش توقف و افزایش گذردهی تمایل به باز کردن چندین اتصال TCP موازی دارند.

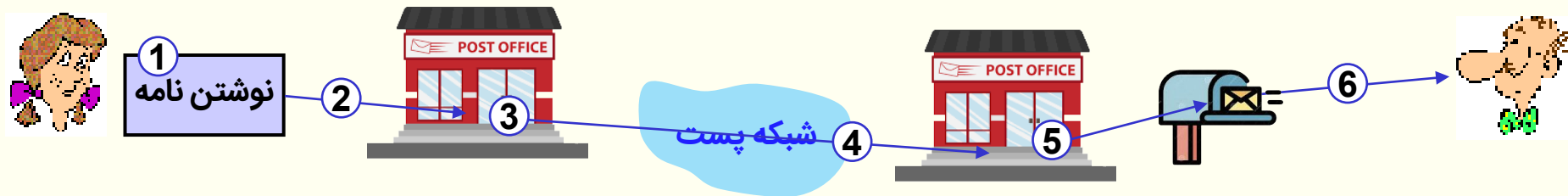
• بدون امنیت بر روی اتصال TCP

• HTTP/۳: اضافه شدن امنیت، کنترل خطا و ازدحام به ازای هر شیء (خط لوله بیشتر) روی UDP

• موارد بیشتر در مورد HTTP/۳ در لایه انتقال (پروتکل انتقال QUIC)

پست الکترونیکی (Email) – نگاهی به پست عادی (غیرالکترونیکی) سناریو: ارسال یک نامه پستی از "آلیس" به "باب"

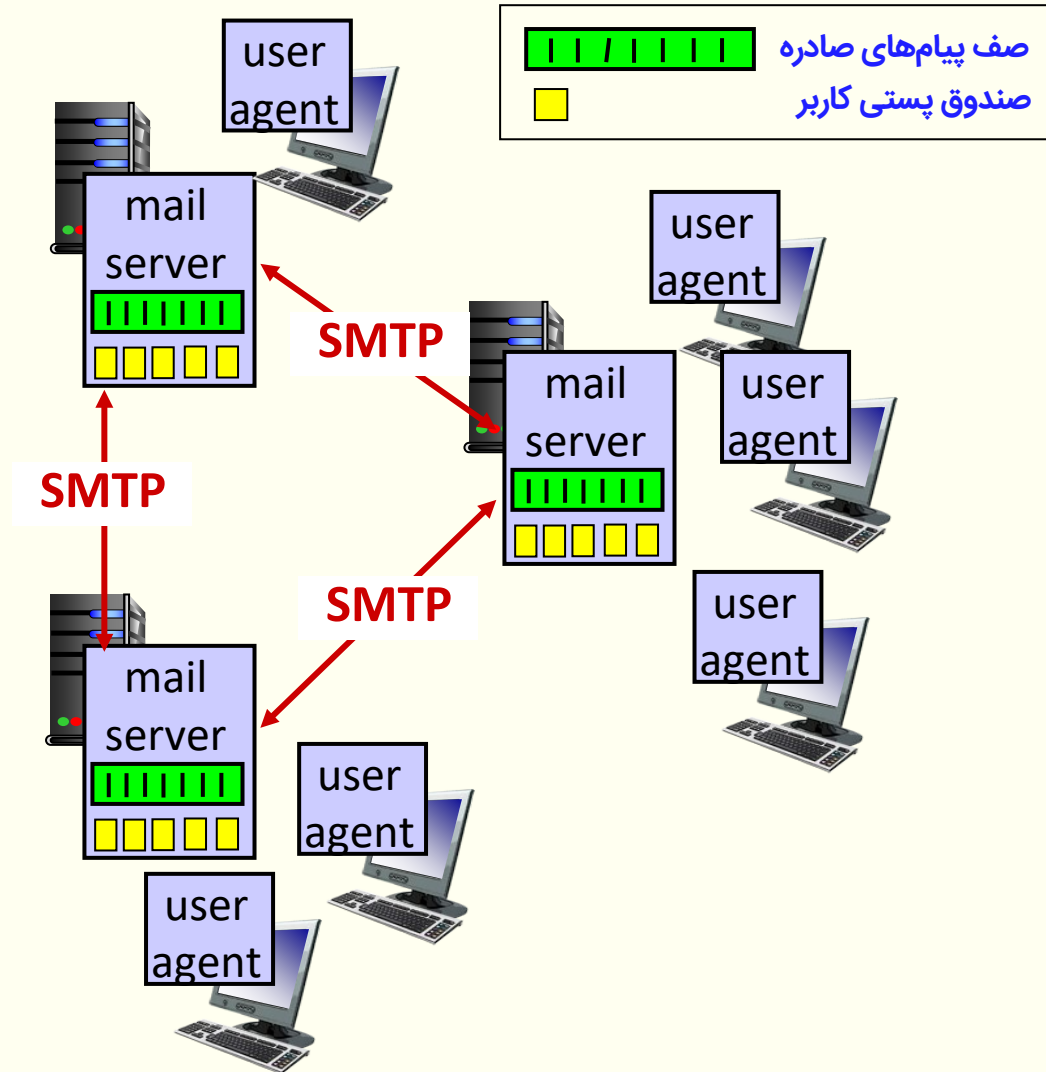
- ۱) نوشتن نامه و قراردادن آن در یک پاکت (بسته) پستی توسط آلیس
- ۲) تحویل بسته پستی به اداره پست توسط آلیس و قرارگرفتن آن در صف ارسال بسته‌های پستی
- ۳) دست‌بندی نامه‌ها براساس آدرس مقصد و تحویل آن به شبکه انتقال بسته‌های پستی
- ۴) انتقال بسته پستی و تحویل آن به اداره پست مقصد
- ۵) قراردادن بسته پستی در صندوق پستی باب
- ۶) مراجعه باب به صندوق پستی خود و خواندن نامه



پست الکترونیکی (Email)

سه بخش اصلی:

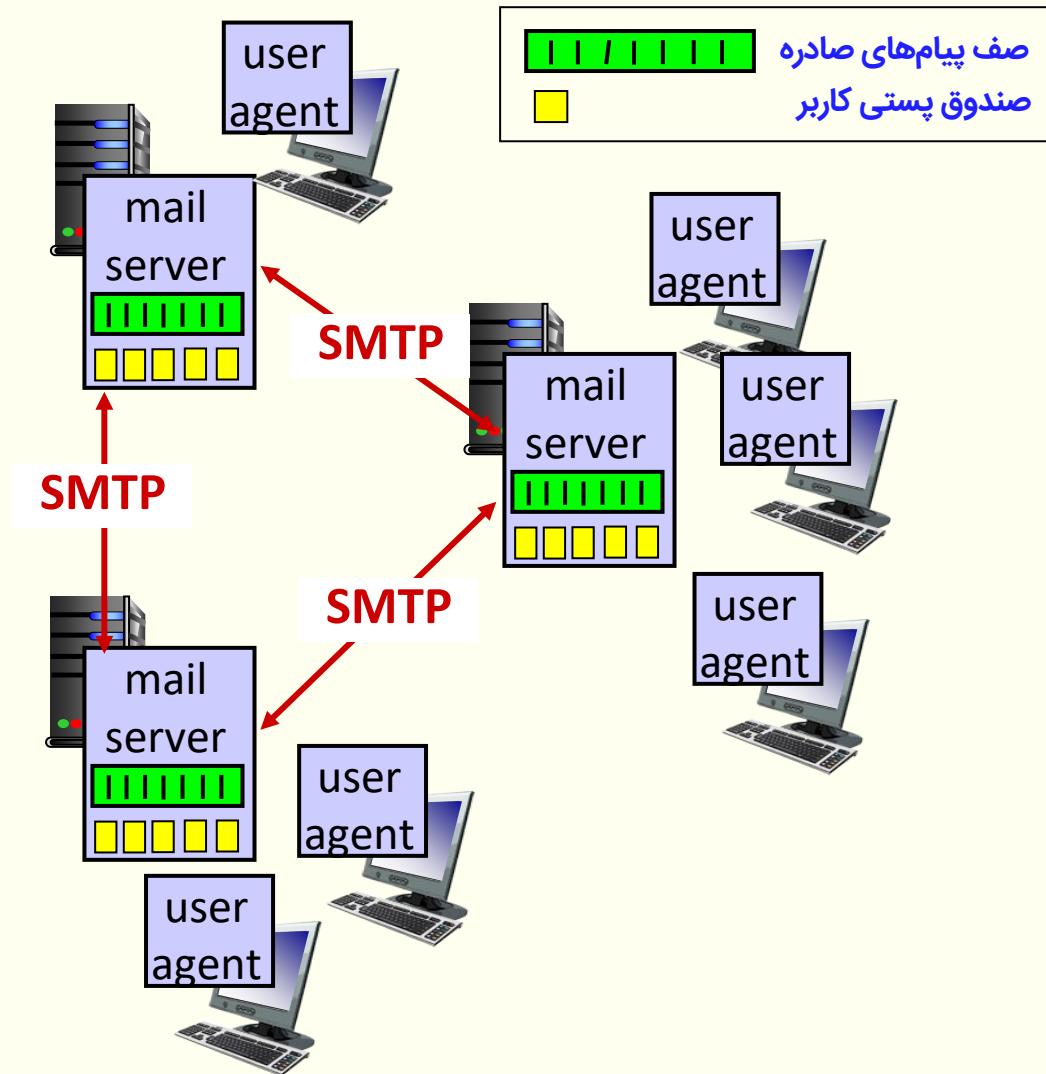
- عامل کاربر (user agents)
- سرویس‌دهنده‌های پست (mail servers)
- پروتکل انتقال پست ساده (simple mail transfer protocol - SMTP)



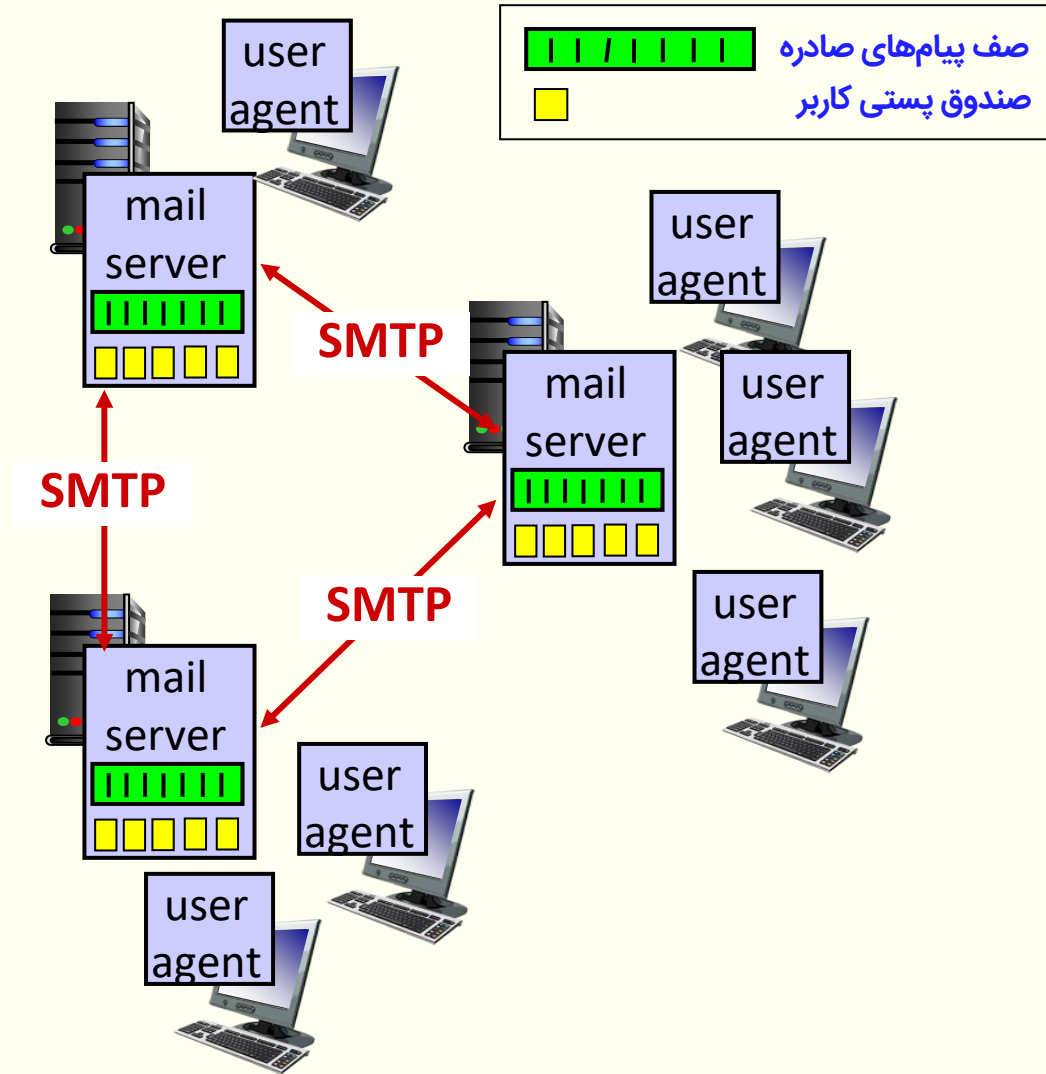
پست الکترونیکی – عامل کاربر

عامل کاربر:

- موسوم به پست خوان (mail reader)
- ایجاد، ویرایش و خواندن پیام‌های پست الکترونیکی
- به عنوان نمونه: برنامه‌های Outlook و iPhone mail Client
- پیام‌های وارده (incoming) و صادره (outgoing) در سرویس‌دهنده ذخیره می‌شوند.



پست الکترونیکی - سرویس دهنده های پست



سرویس دهنده های پست:

- نگهداری پیام های (پستی) وارده در صندوق پستی کاربر
- صف پیام های پستی صادره (برای ارسال شدن)
- استفاده از پروتکل SMTP بین سرویس دهنده های پست برای ارسال پیام های پست الکترونیکی
- **سرویس گیرنده:** سرویس دهنده پست ارسال کننده
- **سرویس دهنده:** سرویس دهنده پست دریافت کننده

پست الکترونیکی – پروتکل SMTP – RFC 5321

- استفاده از پروتکل TCP برای انتقال مطمئن پیام پست الکترونیکی از سرویس گیرنده (سرویس دهنده پست برقرارکننده اتصال) به سرویس دهنده، پورت ۲۵
- انتقال مستقیم: از سرویس دهنده ارسال کننده (عمل کننده مانند سرویس گیرنده) به سرویس دهنده دریافت کننده
- سه مرحله (فاز) انتقال پیام
 - دست دادن (خوش آمدگویی)
 - انتقال پیامها
 - خاتمه
- تعامل دستور (درخواست)/پاسخ (مانند HTTP)
 - دستورها: متن با کدگذاری اسکی
 - پاسخ: کد و عبارت وضعیت
- پیامها می بایست کدگذاری اسکی ۷ بیتی داشته باشند.

لایه کاربرد

سناریو: ارسال یک پست الکترونیکی از "آلیس" به "باب"

(۴) سرویس‌گیرنده SMTP پیام آلیس را بر روی اتصال TCP ارسال می‌کند.

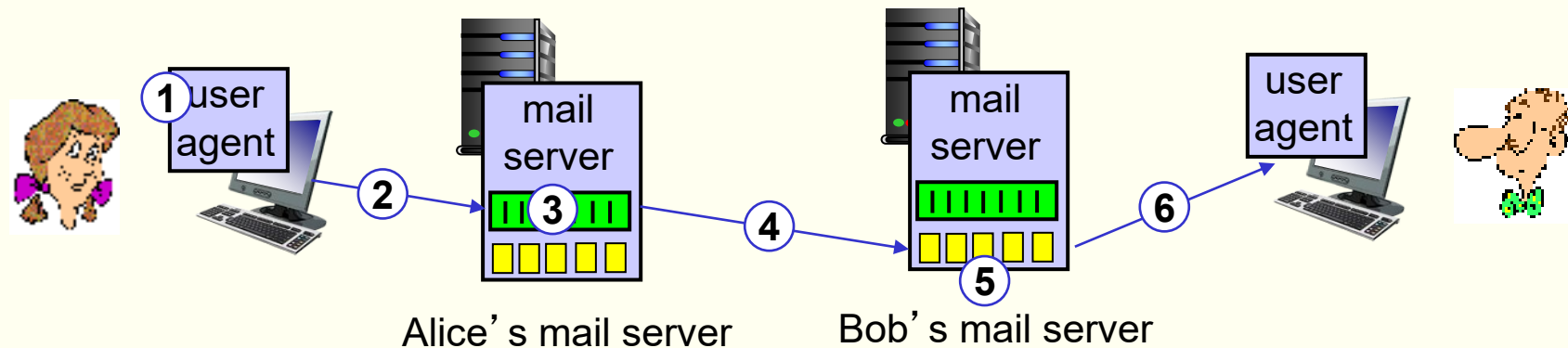
(۵) سرویس‌دهنده پست باب، پیام دریافتی را صندوق پستی باب قرار می‌دهد.

(۶) باب از طریق عامل کاربر خود پیام (رسیده) را می‌خواند.

(۱) ایجاد یک پیام پست الکترونیکی برای bob@someschool.edu توسط عامل کاربر (UA) آلیس

(۲) پیام توسط عامل کاربر آلیس به سرویس‌دهنده پست آلیس ارسال شده و در صف پیام‌های صادره قرار می‌گیرد.

(۳) سمت سرویس‌گیرنده پروتکل SMTP (سرویس‌دهنده پست آلیس) یک اتصال TCP با سرویس‌دهنده پست باب باز می‌کند.



یک نمونه از تعامل پروتکل SMTP

```
S: 220 hamburger.edu
C: HELO crepes.fr
S: 250 Hello crepes.fr, pleased to meet you
C: MAIL FROM: <alice@crepes.fr>
S: 250 alice@crepes.fr... Sender ok
C: RCPT TO: <bob@hamburger.edu>
S: 250 bob@hamburger.edu ... Recipient ok
C: DATA
S: 354 Enter mail, end with "." on a line by itself
C: Do you like ketchup?
C: How about pickles?
C: .
S: 250 Message accepted for delivery
C: QUIT
S: 221 hamburger.edu closing connection
```

انجام تعاملات SMTP توسط خودتان

به سرویس دهنده پست الکترونیکی (mail server) روی پورت ۲۵ telnet کنید.

• مشاهده پیام ۲۲۰ پاسخ سرویس دهنده

• دستورهای HELO، MAIL FROM:، RCPT TO:، DATA، QUIT را وارد کنید.

با استفاده از دستورهای فوق، بدون استفاده از یک برنامه پست الکترونیکی می‌توانید یک پست الکترونیکی ارسال کنید.

توجه: این کار فقط زمانی قابل انجام است که سرویس دهنده پست الکترونیکی اجازه ایجاد اتصال telnet بر روی پورت ۲۵ را بدهد
(بدلیل ملاحظات امنیتی، به ندرت امکان انجام این کار وجود دارد)

SMTP: مشاهدات

مقایسه با HTTP:

• HTTP: PULL

• SMTP: PUSH

• هر دو در تعاملات دستور (خواست)/پاسخ و کدهای وضعیت از کدگذاری اسکی استفاده می کنند.

• HTTP: هر شیء در پیام پاسخ خودش قرار می گیرد.

• SMTP: چند شیء در پیام چندقسمتی ارسال می شوند.

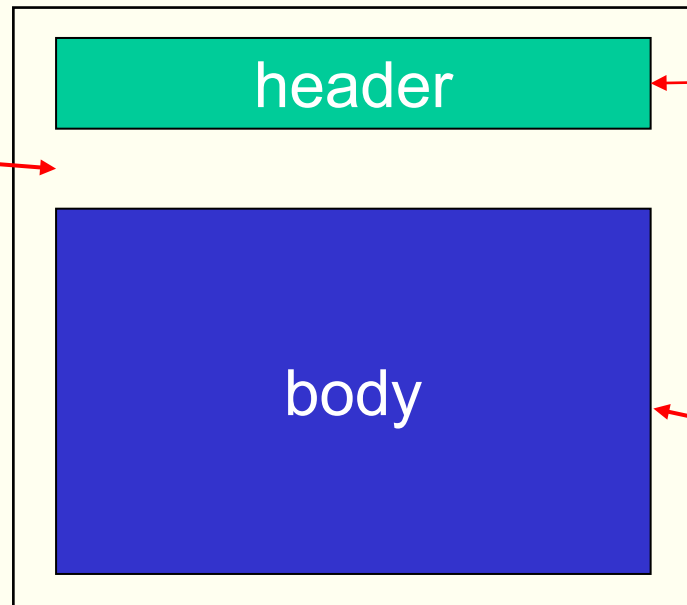
• SMTP از اتصال مداوم استفاده می کند.

• پیام (سرآیند و بدنه) در SMTP باید کدگذاری اسکی ۷ بیتی داشته باشد.

• سرویس دهنده SMTP از CRLF.CRLF برای تعیین انتهای پیام استفاده می کند.

فرمت پیام پست

SMTP: پروتکل تبادل پیام‌های پست الکترونیکی در **RFC 5321**
(مانند HTTP) تعریف شده است و در **RFC 822** فرمت پیام‌های
پست الکترونیکی (مانند HTML) تعریف شده است.



• خطوط سرآیند، به عنوان نمونه،

• به (To:)

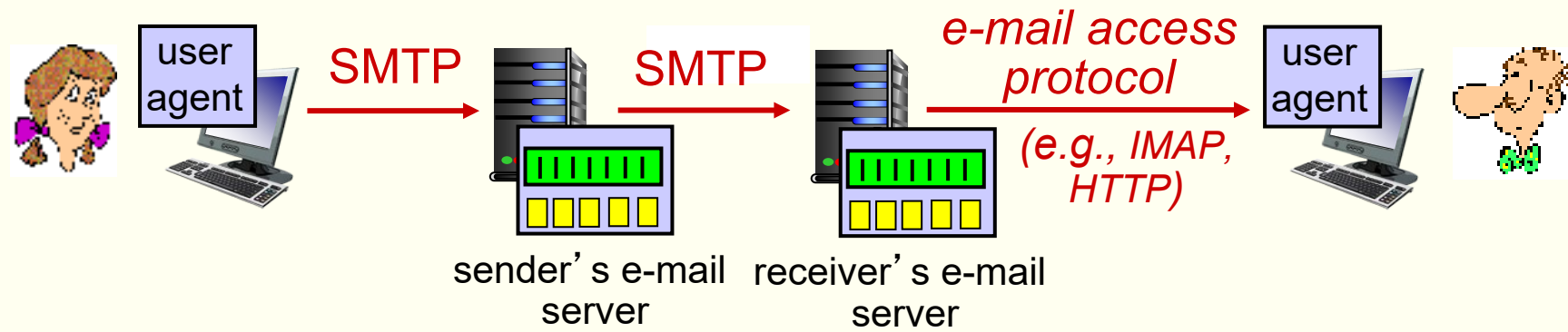
• از (From:)

• موضوع (Subject:)

این خطوط، در بدنه پیام پست الکترونیکی با دستورات SMTP MAIL FROM: و RCPT TO: تفاوت دارند.

• بدنه: "پیام"، فقط کاراکترهای اسکی

پروتکل‌های دسترسی پست الکترونیکی



• **SMTP:** برای تحویل یا ذخیره پیام‌های پست الکترونیکی در سرویس‌دهنده دریافت‌کننده

• **پروتکل دسترسی پست الکترونیکی:** بازیابی (دریافت) از سرویس‌دهنده

• **IMAP:** [RFC ۳۵۰۱] Internet Mail Access Protocol: پیام‌های در سرویس‌دهنده ذخیره می‌شوند، پروتکل IMAP بازیابی (دریافت) و حذف پیام‌ها و پوشه‌های ذخیره شده در سرویس‌دهنده را فراهم می‌کند.

• **HTTP:** Webmail ها نظیر gmail، Hotmail و Yahoo!Mail واسط تحت وب در بالای پروتکل SMTP (برای ارسال) و پروتکل IMAP (یا POP) برای بازیابی پیام‌های پست الکترونیکی را فراهم می‌کنند.

سیستم نامه دامنه (DNS: Domain Name System)

شبکه تلفن

- شماره تلفن یک عدد حداکثر ۱۵ رقمی است.
- برای تماس با هر فردی از طریق شبکه تلفن، داشتن شماره تلفن آن ضروری است.
- به خاطر سپاری شماره‌های تلفن کار دشواری است.
- هر مخاطب یک نام دارد (یا ما برای آن یک نام انتخاب می‌کنیم).
- به خاطر سپاری نام افراد (مخاطبین) کار ساده‌ای است.
- با استفاده از برنامه مخاطبین (Contact) (یا در گذشته دفترچه تلفن) نام و شماره تلفن مخاطبین را نگهداری می‌کنیم.
- برای تماس تلفنی با هر فردی با مراجعه به فهرست مخاطبین و جستجو نام مخاطب، شماره تلفن آن را بدست آوردیم.
- از طریق شماره تلفن بدست آمده با آن مخاطب تماس می‌گیریم.

سیستم نامه دامنه (DNS: Domain Name System)

شبکه اینترنت

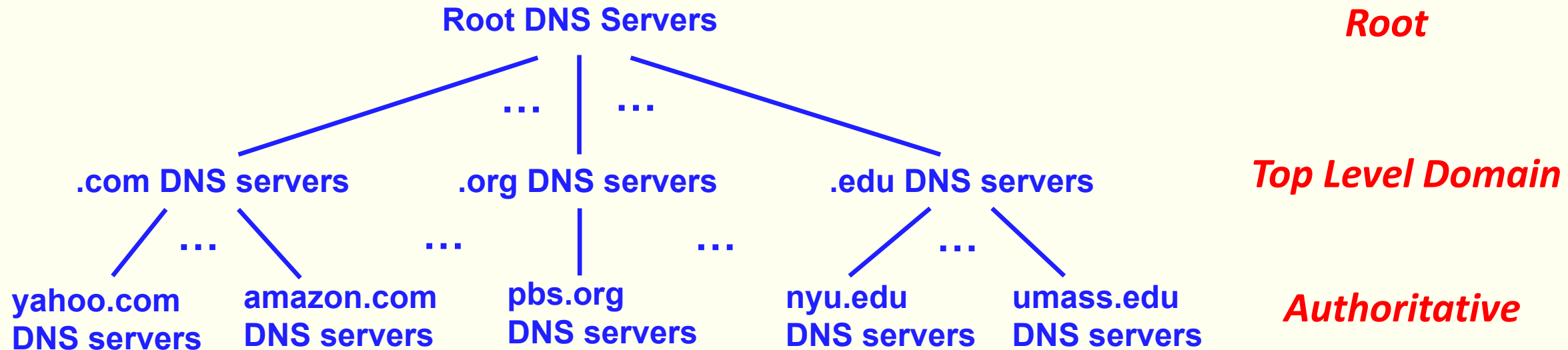
- آدرس IP یک عدد ۳۲ بیتی است (IPv4)
- برای دریافت سرویس از هر سرویس دهنده لازم است آدرس IP آن را داشته باشیم.
- به خاطر سپاری آدرس های IP سرویس دهنده ها کار دشواری است.
- هر سرویس دهنده برای خود یک نام منحصر به فرد انتخاب می کند.
- کاربران نام سرویس دهنده به راحتی می تواند در خاطر داشته باشند.
- آدرس IP سرویس دهندگان به همراه نامشان در یک پایگاه داده ها در سرویس دهنده نام DNS ذخیره می شوند.
- کاربر با استفاده پروتکل DNS درخواست دریافت آدرس IP متناظر با نام مورد نظر را می دهد (پرس و جو DNS).
- برقراری ارتباط با سرویس دهنده مورد نظر از برای دریافت سرویس از طریق آدرس IP دریافتی متناظر با نام پرس و جو شده (resolved) از سرویس دهنده نام DNS،

ساختار DNS

- پایگاه داده‌های DNS به صورت **منطقی** یک پایگاه داده‌های **متمرکز** است.
- امکان پیاده‌سازی متمرکز DNS وجود ندارد:
- مقیاس‌پذیری به دلیل حجم بسیار بالای درخواست‌ها و تأخیر دسترسی از راه‌دور
- مشکلات دیگر نظیر وابستگی به یک سرویس‌دهنده (نقطه خرابی تک)، نگهداری و ...
- DNS به صورت یک **پایگاه داده‌های سلسله‌مراتبی توزیع‌شده** پیاده‌سازی شده است (مفهومی متمرکز، پیاده‌سازی توزیع‌شده).
- تعداد بسیار زیادی سرویس‌دهنده‌های نام DNS (DNS Name Servers) در یک ساختار سلسله‌مراتبی مسئول تبدیل نام‌ها به آدرس‌های IP هستند.
- ساختار سلسله‌مراتبی DNS:
 - **ریشه:** سرویس‌دهنده‌های DNS ریشه (root)
 - **سطح اول:** سرویس‌دهنده‌های DNS **سطح بالا** (top level) نظیر .com، .edu، .org و ...
 - **سطح‌های بعدی:** سرویس‌دهنده‌های DNS **معتبر** (authoritative) نظیر google.com، amazon.com و ...

لایه کاربرد

ساختار DNS: پایگاه داده‌های توزیع شده سلسله‌مراتبی



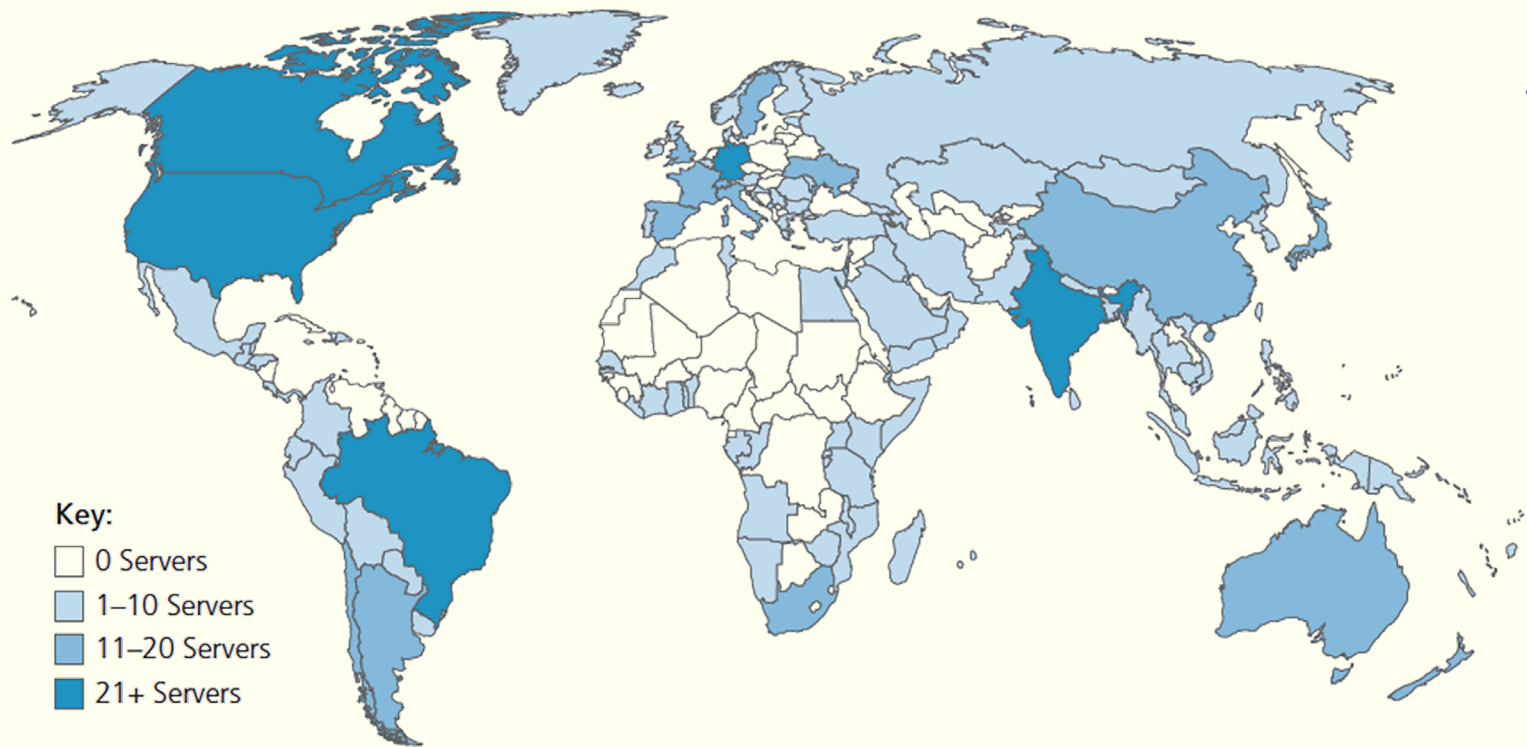
مثال: سرویس‌دهنده‌ای که آدرس IP نام دامنه `www.amazon.com` را بدست آورد: (تقریب اول)

- با پرس و جو از یک سرویس‌دهنده ریشه آدرس سرویس‌دهنده DNS سطح بالای `.com` را بدست می‌آورد.
- با پرس و جو از سرویس‌دهنده DNS سطح بالای `.com` آدرس سرویس‌دهنده DNS معتبر `amazon.com` را بدست می‌آورد.
- با پرس و جو از سرویس‌دهنده DNS معتبر `amazon.com` آدرس IP نام دامنه `www.amazon.com` را بدست می‌آورد.

ساختار DNS: سرویس دهنده های نام ریشه

۱۳ سرویس دهنده نام ریشه منطقی

هر سرویس دهنده ریشه به دفعات زیاد در سراسر جهان تکرار شده است
(حدود ۲۰۰ سرویس دهنده فقط در ایالات متحده وجود دارد)



- آخرین راه رسمی توسط سرویس دهنده های نام که نمی توانند خودشان تبدیل نام به آدرس را انجام دهند.
- وظیفه بسیار مهمی در اینترنت دارند.

- اینترنت بدون سرویس دهنده DNS ریشه نمی تواند کار کند.
- DNSSEC: فراهم کننده امنیت DNS (تصدیق هویت و صحت پیامها)

- (Internet Corporation for Assigned Names and Numbers) ICANN مدیریت می کند.

ساختار DNS: سرویس دهنده‌های سطح بالا (Top-Level)

سرویس دهنده‌های دامنه‌های سطح بالای (TLD – Top-Level Domain):

- مسئول دامنه‌های سطح بالای .com، .net، .org، .edu و و همچنین دامنه‌های کشور سطح بالا نظیر .ir، .uk، .fr و .ca.
- راه حل مناسب برای ثبت نمودن نام دامنه‌ها معتبر و اطمینان از منحصر بفرد بودن نام دامنه‌ها

سرویس دهنده‌های DNS معتبر (Authoritative)

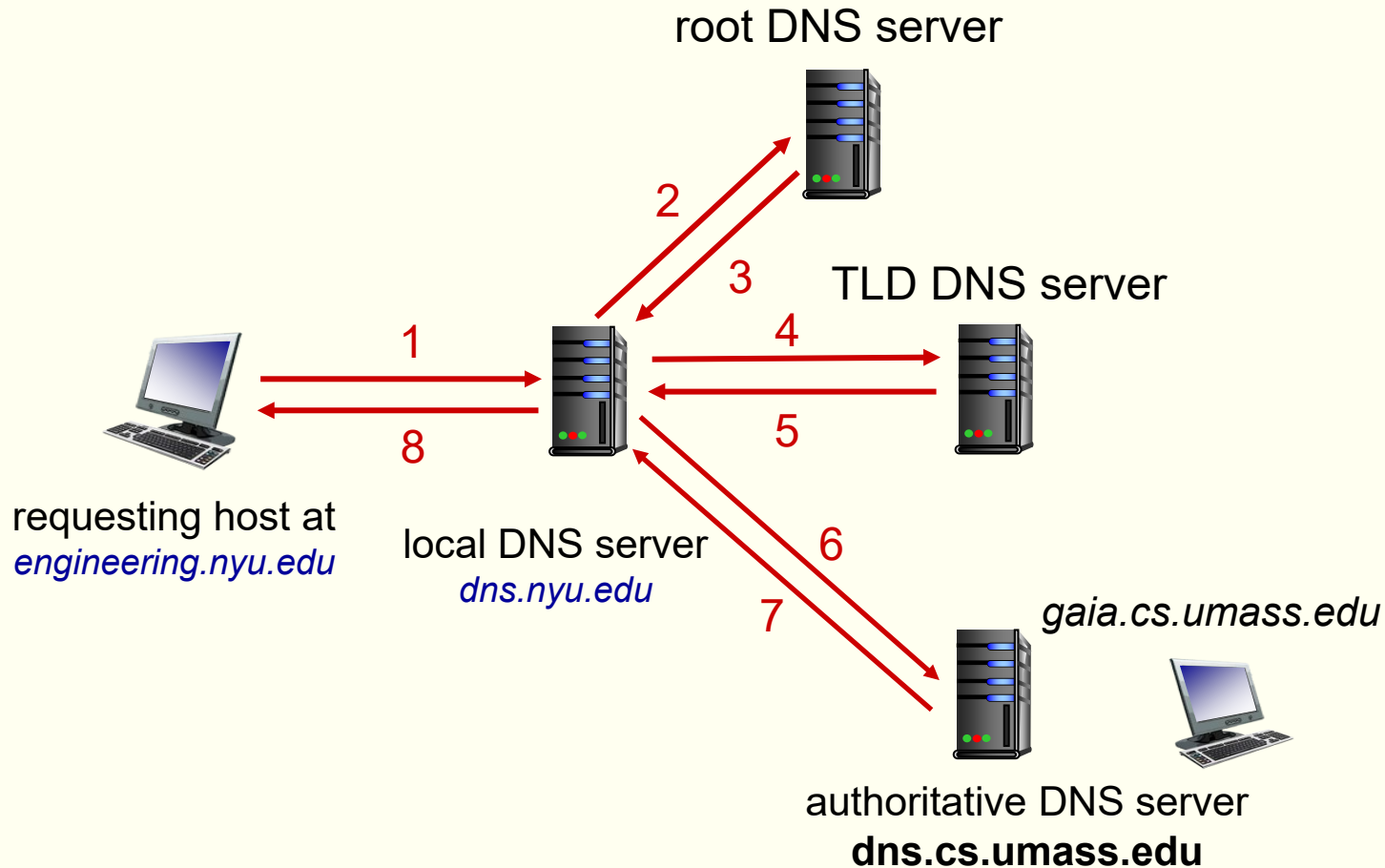
- سرویس دهنده‌های DNS سازمان‌ها (شرکت‌ها)، نگاشت نام میزبان به آدرس IP برای نام میزبان‌های سازمان را فراهم می‌کند.
- می‌تواند توسط تأمین کننده سرویس یا سازمان نگهداری شود.

ساختار DNS: سرویس دهنده های DNS محلی (Local DNS Servers)

سرویس دهنده های DNS محلی:

- به صورت مؤکد متعلق به ساختاری سلسله مراتبی نیستند.
- هر تأمین کننده سرویس اینترنت (همانند ISP های مسکونی، شرکت ها و دانشگاه ها) یک سرویس دهنده DNS محلی دارند.
- به عنوان سرویس دهنده نام پیش فرض (default) نیز شناخته می شوند.
- هر زمان که میزبان ها درخواست تبدیل نام به آدرس IP دارند، درخواست هایشان را به سرویس دهنده DNS محلی می دهند.
- سرویس دهنده DNS محلی تبدیل های اخیر نام به آدرس IP را در حافظه پنهان خود ذخیره و به طور موقتی نگهداری می کند.
- سرویس دهنده DNS محلی در صورت نبود پاسخ درخواستی در حافظه پنهان آن را در ساختار سلسله مراتبی ارسال می کند.

نحوه تبدیل نام به آدرس در DNS: پرس و جو تکرار شده (iterated query)

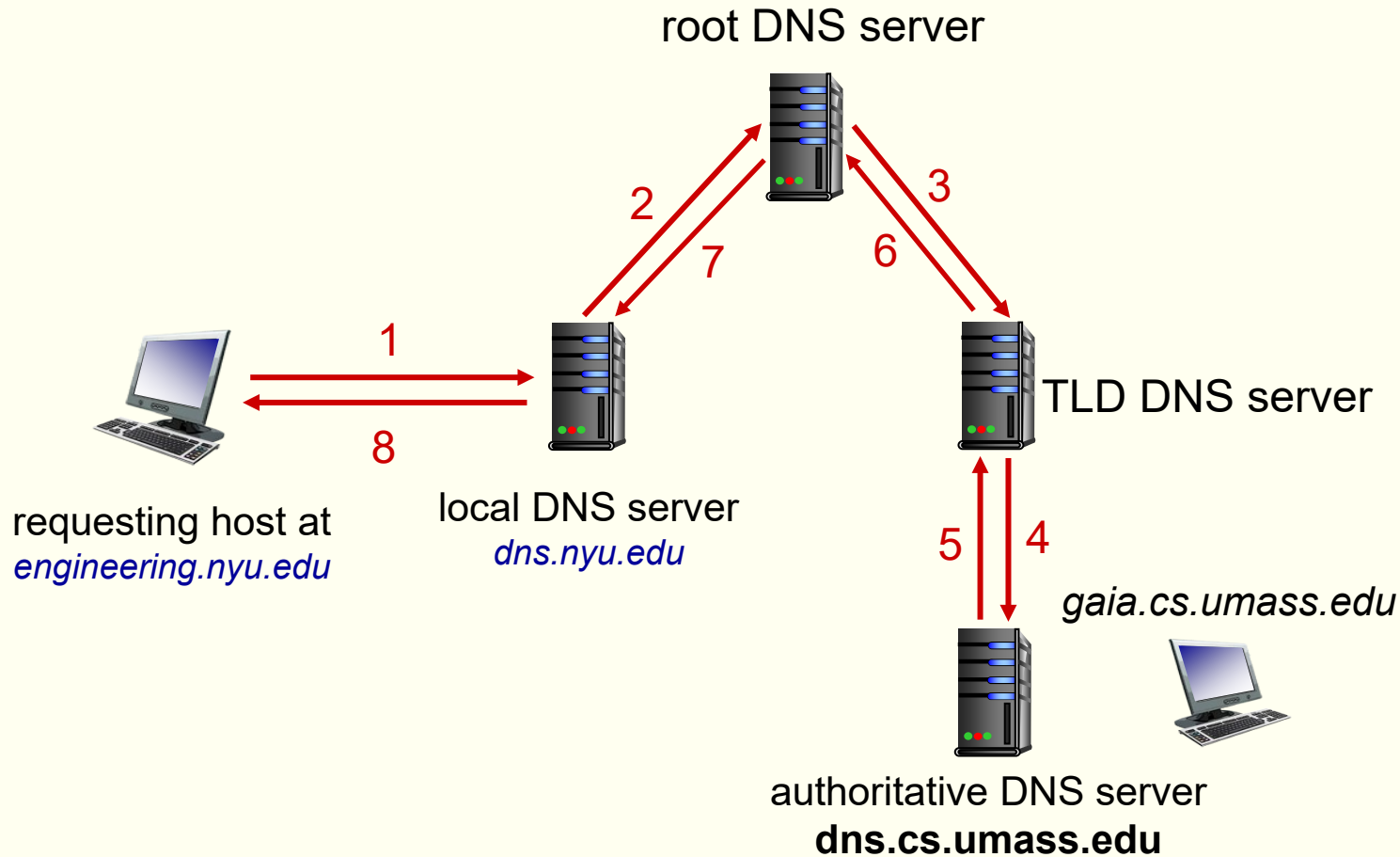


مثال: میزبانی در دامنه `engineering.nyu.edu`
آدرس IP نام `gaia.cs.umass.edu` را می‌خواهد.

پرس و جو تکرار شده (iterated)

- پاسخ سرویس‌دهنده مورد پرسش، نام سرویس‌دهنده‌ای است که باید پرسش از آن تکرار شود.
- در واقع می‌گوید "من این نام نمی‌شناسم ولی از این سرویس‌دهنده نام پرسشت را تکرار کن!"

نحوه تبدیل نام به آدرس در DNS: پرس و جو بازگشتی (recursive query)



مثال: میزبانی در دامنه `engineering.nyu.edu`
آدرس IP نام `gaia.cs.umass.edu` را می‌خواهد.

پرس و جو بازگشتی (recursive)

- تبدیل نام به آدرس IP به عهده سرویس‌دهنده مورد پرسش است.
- بار کاری سنگین بر روی سرویس‌دهنده‌های سطوح بالای ساختار سلسله‌مراتبی

ذخیره در حافظه پنهان موقت (chaching) و بروزرسانی رکوردهای DNS

- هر زمان که سرویس دهنده نام نگاشت نام را بدست می آورد، آن را در حافظه پنهان ذخیره می کند.
- رکوردهای ذخیره شده در حافظه پنهان بعد از گذشت زمان زنده بودن (Time To Live - TTL) حذف می شوند.
- سرویس دهنده نام دامنه سطوح بالا (TLD) معمولاً در حافظه پنهان سرویس دهنده محلی ذخیره می شوند.
- از این رو، غالباً سرویس دهنده های ریشه بازدید نمی شوند.
- رکوردهای ذخیره شده در حافظه پنهان ممکن است بروز نباشند (بیشترین تلاش برای تبدیل آدرس به نام!).
- اگر آدرس IP نام میزبانی تغییر کند، تا تمام زمانی که زمان های TTL در شبکه گسترده اینترنت سپری نشوند، ممکن است شناسایی نشود.
- مکانیزم های بروزرسانی و اطلاع رسانی در استاندارد IETF پیشنهاد شده است:
RFC 2316

سرویس‌های DNS

- تبدیل نام میزبان به آدرس IP
- امکان داشتن نام مستعار (aliasing) برای میزبان‌ها
- نام اصلی (رسمی) و نام مستعار
- نام مستعار برای سرویس‌دهنده‌های پست الکترونیکی
- توزیع بار ترافیکی
- استفاده از چندین سرویس‌دهنده‌های وب تکرار شده (replicated): آدرس IP همه سرویس‌دهنده‌های تکراری با یک نام مطابقت دارند.

رکوردهای DNS

ذخیره‌سازی رکوردهای منبع (Resource Record - RR) در پایگاه داده‌های توزیع شده DNS
RR Format: (**name**, **value**, **type**, **ttl**)

نوع (type)	نام (name)	مقدار (value)
A	نام میزبان	آدرس IP
NS	نام دامنه	نام سرویس دهنده نام معتبر
CNAME	نام مستعار	نام رسمی
MX	نام	نام سرویس دهنده پست

پیام‌های پروتکل DNS

پیام‌های پرس و جو (query) و پاسخ DNS یک فرمت یکسان دارند.

← 2 bytes → ← 2 bytes →	
identification	flags
# questions	# answer RRs
# authority RRs	# additional RRs
questions (variable # of questions)	
answers (variable # of RRs)	
authority (variable # of RRs)	
additional info (variable # of RRs)	

سرآیند پیام‌ها:

• شناسه پیام:

• یک عدد ۱۶ بیتی تعیین شده در پیام پرس و جو است. پیام پاسخ به هر پرس و جو از همان عدد شناسه استفاده می‌کند.

• پرچم‌ها:

- نوع پیام پرس و جو یا پاسخ است.
- مکانیزم بازگشتی مطلوب است.
- مکانیزم بازگشتی دردسترس است.
- پاسخ سرویس‌دهنده DNS معتبر است.

پیام‌های پروتکل DNS

پیام‌های پرس و جو (query) و پاسخ DNS یک فرمت یکسان دارند.

← 2 bytes → ← 2 bytes →

identification	flags
# questions	# answer RRs
# authority RRs	# additional RRs
questions (variable # of questions)	
answers (variable # of RRs)	
authority (variable # of RRs)	
additional info (variable # of RRs)	

• فیلدهای نام و نوع برای پرس و جو استفاده می‌شوند

• رکوردهای منبع (RRs) در پاسخ به پرس و جو

• رکوردها برای سرویس‌دهنده‌های معتبر

• اطلاعات "مفید" تکمیلی، استفاده از فیلد این اختیاری است

اضافه کردن رکورد در DNS

مثال: یک شرکت جدید با نام "Network Utopia"

- ثبت کردن نام دامنه "networkutopia.com" در ثبت کننده DNS
- تعریف نام و آدرس IP سرویس دهنده نام معتبر (اولیه (primary) و ثانویه (secondary))
- اضافه کردن رکورد منبع NS و A را سرویس دهنده دامنه سطح بالا (TLD) توسط ثبت کننده

(networkutopia.com, dns1.networkutopia.com, NS)
(dns1.networkutopia.com, 212.212.212.1, A)

• راه اندازی سرویس دهنده معتبر محلی با آدرس IP 212.212.212.1

• رکورد نوع A برای www.networkutopia.com

• رکورد نوع MX برای networkutopia.com

امنیت DNS

DNSSEC
[RFC 4033]

حملات تغییر مسیر (Redirect attacks)

• حملات مرد میانی (man-in-the-middle)

- پرس و جوهای DNS را به جای سرویس دهنده DNS دریافت می کند.

• مسموم سازی DNS (DNS poisoning)

- ارسال پاسخ جعلی به سرویس دهنده DNS که پاسخ ها را در حافظه پنهان نگهداری می کند.

استفاده از DNS برای حملات DDoS

- ارسال پرس و جو با آدرس منبع جعلی

حملات ممانعت از سرویس توزیع شده

Distributed Denial-Of-Service (DDoS) attacks

- بمباران ترافیکی سرویس دهنده ریشه

- تا به امروز حملات موفقی نبوده اند.

- فیلتر کردن ترافیک

- بدلیل اینکه سرویس دهنده های DNS محلی، آدرس

- سرویس دهنده های TLD (دامنه سطح بالا) را در حافظه

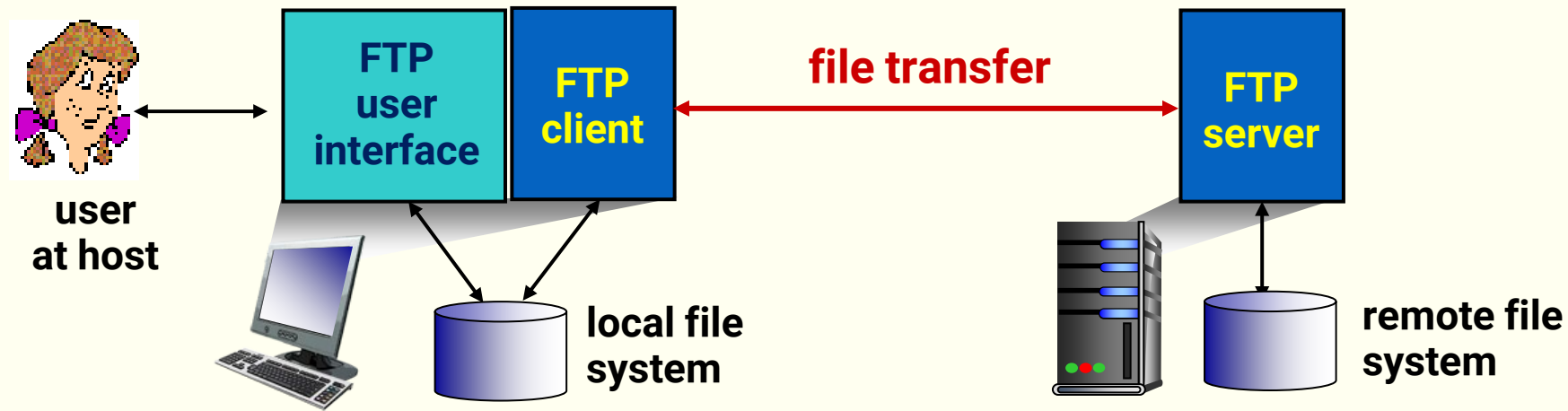
- پنهان نگهداری می کنند. اجازه می دهند

- سرویس دهنده های DNS ریشه bypass شوند.

- بمباران ترافیکی سرویس دهنده های TLD

- بالقوه حملات بسیار خطرناکی هستند.

پروتکل انتقال فایل (FTP – File Transfer Protocol)



• انتقال فایل به / از کامپیوتر میزبان راه دور

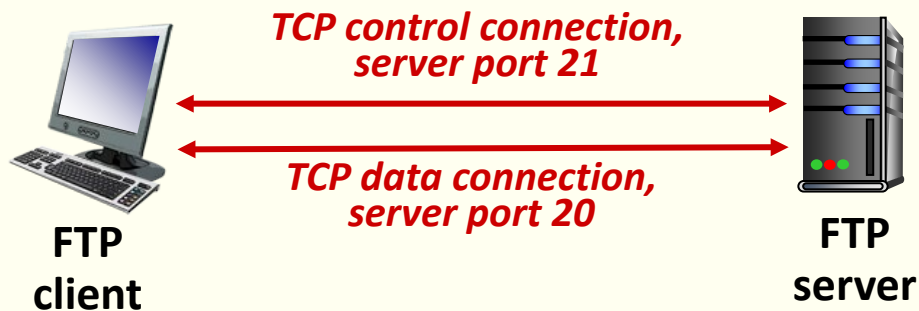
• الگو: سرویس گیرنده/سرویس دهنده

• سرویس گیرنده: سمتی است که شروع کننده انتقال است (انتقال به یا انتقال از میزبان راه دور)

• سرویس دهنده: میزبان راه دور

• RFC 959 :FTP

پروتکل انتقال فایل: اتصال‌های جداگانه کنترل و داده‌ها



- باز کردن اتصال داده TCP دیگر برای انتقال فایل دیگر
- اتصال کنترلی "خارج از باند" ("out of band")
- نگهداری حالت (state) در پروتکل FTP: دایرکتوری جاری و تصدیق هویت انجام‌شده

• برقراری اتصال سرویس‌گیرنده FTP از طریق پورت ۲۱ پروتکل TCP با سرویس‌دهنده FTP (اتصال کنترلی)

• تصدیق هویت سرویس‌گیرنده روی اتصال کنترلی

• مرور دایرکتوری‌ها از راه‌دور و ارسال دستورات روی اتصال کنترلی توسط سرویس‌گیرنده

• باز کردن اتصال داده‌ها (اتصال دوم) از طریق پورت ۲۰ پروتکل TCP به سرویس‌گیرنده توسط سرویس‌دهنده با دریافت دستور انتقال فایل

• بستن اتصال داده بعد از انتقال فایل توسط سرویس‌دهنده

پروتکل انتقال فایل: دستورات و پاسخ‌ها

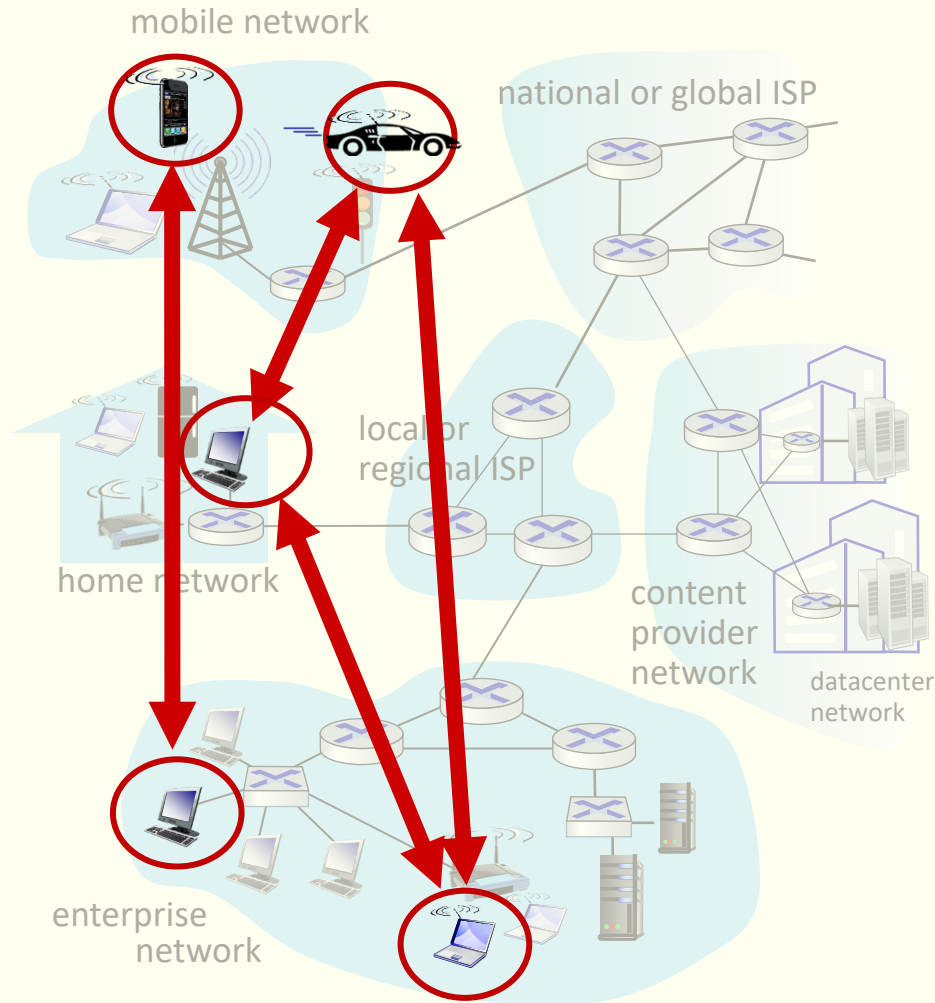
sample commands:

- sent as ASCII text over control channel
- **USER** *username*
- **PASS** *password*
- **LIST** return list of file in current directory
- **RETR filename** retrieves (gets) file
- **STOR filename** stores (puts) file onto remote host

sample return codes:

- status code and phrase (as in HTTP)
- **331 Username OK, password required**
- **125 data connection already open; transfer starting**
- **425 Can't open data connection**
- **452 Error writing file**

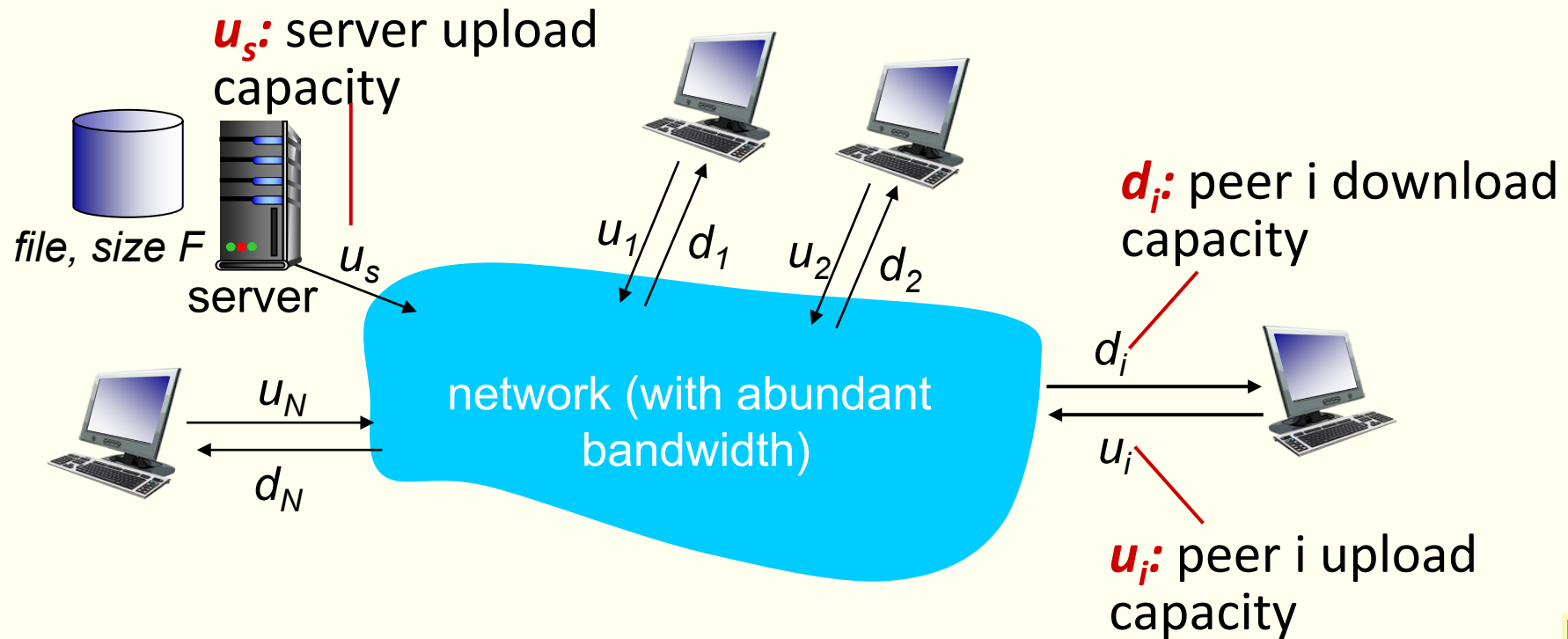
معماری نظیر به نظیر (P2P: Peer-to-Peer Architecture)



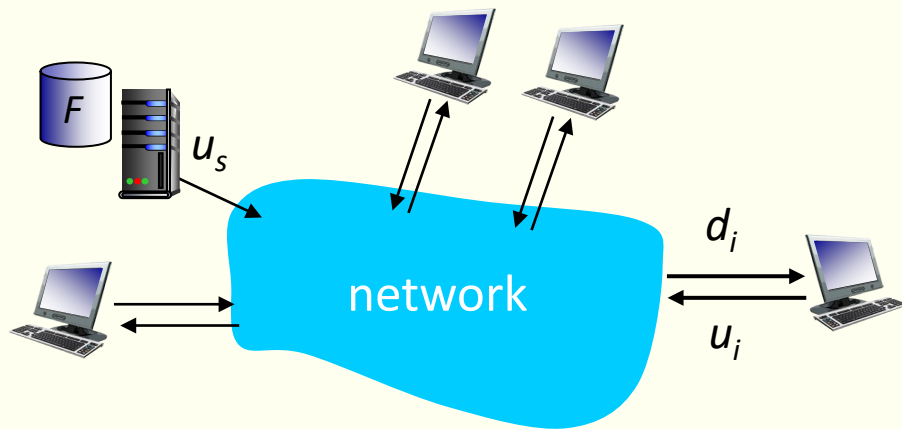
- بدون سرویس دهنده همیشه در حال کار
- سیستم‌های انتهایی به طور مستقیم با هم ارتباط دارند.
- نظیرها درخواست دریافت سرویس از نظیرهای دیگر دارند، در عوض به سایر نظیرها سرویس ارائه می‌دهند.
- مقیاس‌پذیری ذاتی - نظیر جدید ظرفیت سرویس‌دهی و در عین حال تقاضا دریافت سرویس جدید با خود می‌آورد.
- نظیرها به طور موقت به شبکه متصل هستند و آدرس IP آن‌ها تغییر می‌کند.
- مدیریت پیچیده
- مثال: شبکه P2P اشتراک فایل (BitTorrent)، پخش (KanKan) و صوت روی شبکه اینترنت (Skype)

توزیع فایل: مقایسه روش سرویس گیرنده-سرویس دهنده با روش نظیر به نظیر Client-Server VS Peer-to-Peer Architecture

سؤال: چقدر زمان طول می کشد تا یک فایل با اندازه F از یک سرویس دهنده بین N سرویس گیرنده توزیع شود؟
• ظرفیت ارسال (آپلود) و دریافت (دانلود) نظیرها محدود است.



زمان توزیع فایل: روش سرویس گیرنده-سرویس دهنده



- سرویس دهنده: سرویس دهنده باید N کپی از فایل را ارسال کند.

- ارسال هر کپی F/u_s ثانیه طول می کشد.

- ارسال N کپی NF/u_s ثانیه طول می کشد.

- سرویس گیرنده: هر سرویس گیرنده باید یک کپی از فایل را دریافت (دانلود) کند.

- نرخ دانلود سرویس گیرنده با حداقل نرخ دانلود d_{min} است.

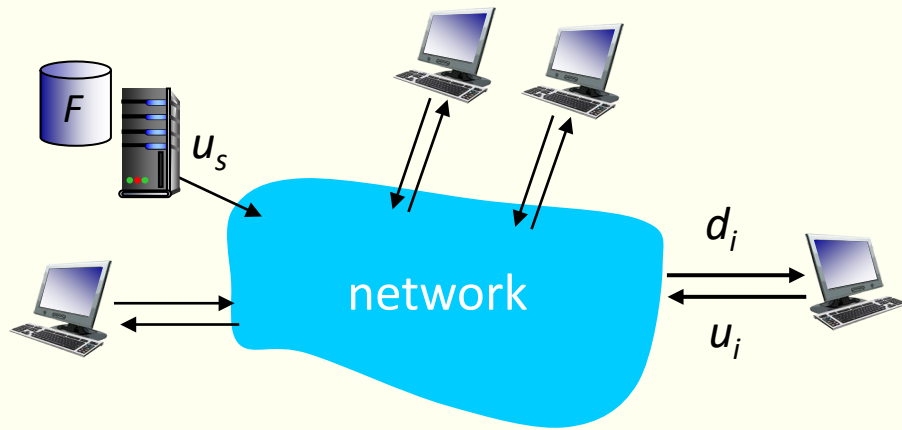
- زمان دانلود فایل سرویس گیرنده حداقل نرخ دانلود F/d_{min} است.

زمان توزیع فایل با اندازه F بین N سرویس گیرنده
با روش سرویس گیرنده-سرویس دهنده

$$D_{C-S} \geq \max\{NF/u_s, F/d_{min}\}$$

با افزایش N زمان به صورت خطی افزایش می یابد

زمان توزیع فایل: روش نظیر به نظیر



• سرویس‌دهنده: حداقل یکبار باید فایل را کپی کند.

• زمان ارسال یک کپی F/u_s ثانیه است.

• نظیر: هر نظیر باید یک کپی از فایل را دریافت (دانلود) کند.

• نرخ دانلود نظیر با حداقل نرخ دانلود d_{min} است.

• زمان دانلود فایل نظیر با حداقل نرخ دانلود F/d_{min} است.

• نظیرها: همه نظیرها باید دریافت NF کنند.

• حداکثر نرخ ارسال (محدود کننده حداکثر نرخ دریافت) برابر است با: $u_s + \sum_{i=1}^N u_i$

زمان توزیع فایل با اندازه F بین N کاربر با روش نظیر به نظیر (P2P)

$$D_{P2P} \geq \max \left\{ F/u_s, F/d_{min}, NF / \left(u_s + \sum_{i=1}^N u_i \right) \right\}$$

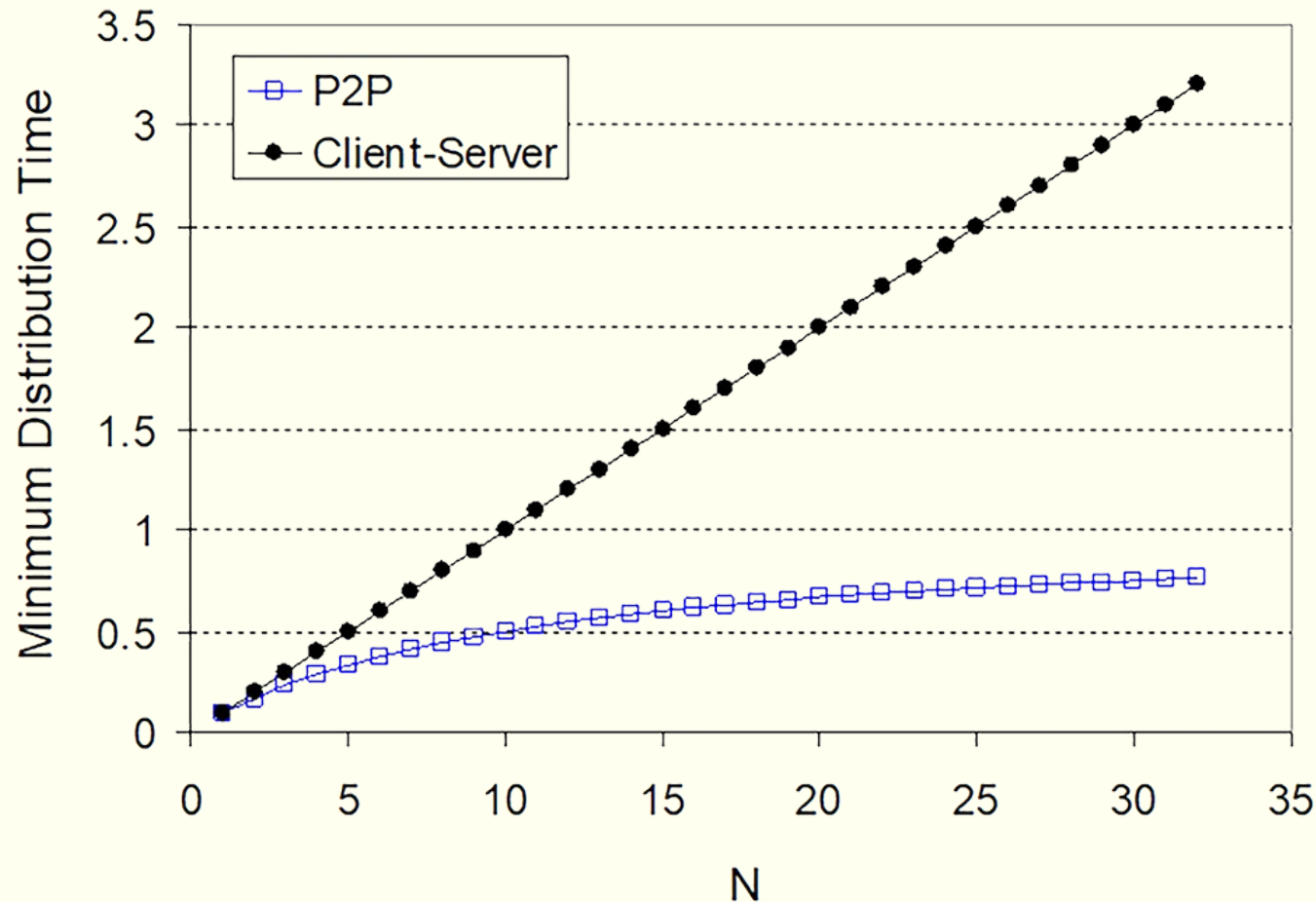
با افزایش تعداد نظیرها زمان به صورت خطی افزایش می‌یابد

با افزایش تعداد نظیرها ظرفیت سرویس‌دهی نیز افزایش می‌یابد.

لایه کاربرد

مثال: مقایسه روش سرویس گیرنده-سرویس دهنده با روش نظیر به نظیر

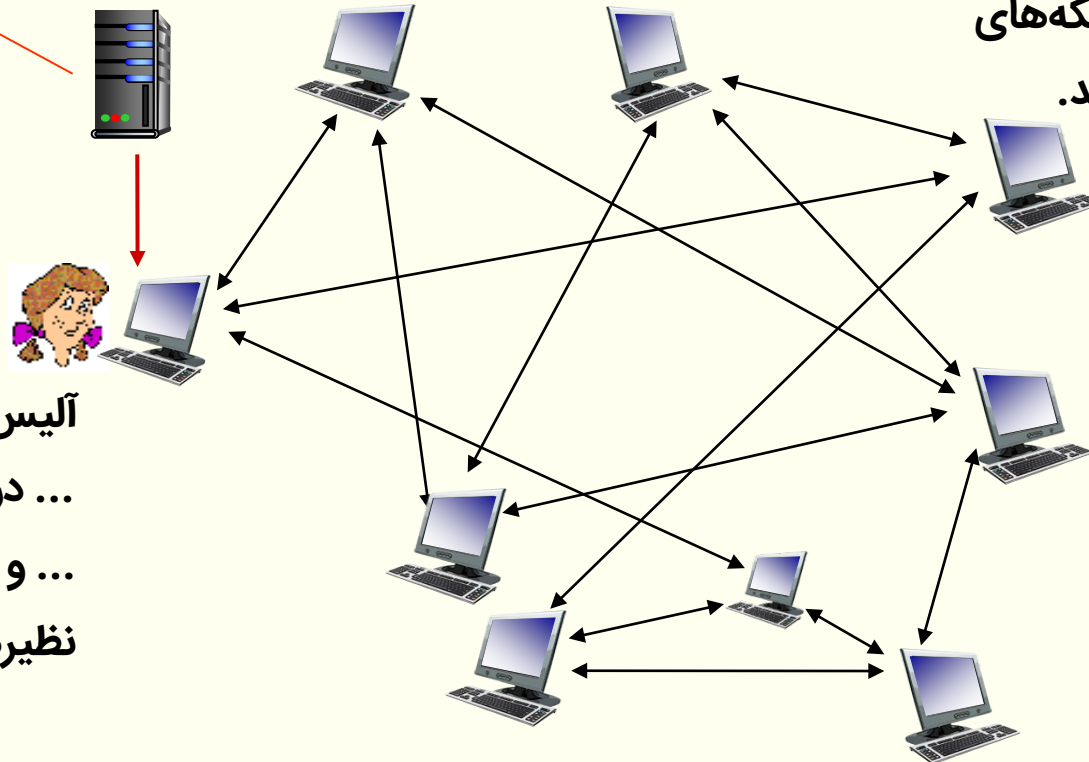
client upload rate = u , $F/u = 1$ hour, $u_s = 10u$, $d_{min} \geq u_s$



توزیع فایل با روش P2P: بیت تورنت (BitTorrent)

- یک فایل به تکه‌ها (chunks) با اندازه ۲۵۶ کیلوبایت تقسیم می‌شود.
- نظیرها در تورنت تکه‌ها را ارسال و دریافت می‌کنند.

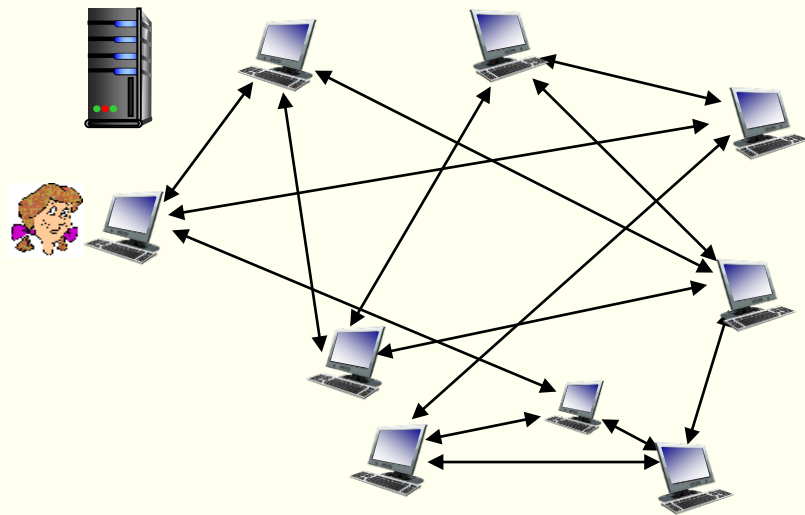
ردیاب (tracker): نظیرهای شرکت‌کننده در تورنت را ردیابی می‌کند.



تورنت: گروهی از نظیرها تکه‌های فایل را با هم مبادله می‌کنند.

آلیس وارد شبکه تورنت می‌شود...
... دریافت فهرست نظیرها از ردیاب
... و شروع به تبادل تکه‌های فایل با
نظیرها در شبکه تورنت می‌کند.

توزیع فایل با روش P2P: بیت تورنت (BitTorrent)



- نظیری که به شبکه تورنت می پیوندد:
- هیچ تکه ای ندارد، ولی در طول زمان از نظیرهای دیگر جمع می کند.
- برای دریافت فهرست نظیرها در ردیاب ثبت می شود و به زیرمجموعه ای از نظیرها ("همسایگان") متصل می شود.
- در زمان دریافت (دانلود)، تکه هایی برای نظیرهای دیگر ارسال می کند.
- ممکن است نظیرهایی که با آن تبادل تکه ها را انجام می دهند را تغییر دهد.
- ورود و خروج: نظیرهای ممکن است بروند و بیایند.
- زمانی که یک نظیر کل فایل را دریافت کرد، ممکن است (خودخواهانه) شبکه تورنت را ترک کند یا اینکه (نوع دوستانه) در شبکه تورنت باقی بماند.

بیت تورنت: درخواست و ارسال تکه‌های فایل

درخواست تکه‌ها:

- در هر زمان (مفروض)، نظیرهای مختلف زیرمجموعه‌ای از تکه‌های فایل را دارند.
- به صورت دوره‌ای، آلیس فهرست تکه‌هایی که آن‌ها دارند را سؤال می‌کند.
- آلیس تکه‌هایی که ندارد را از نظیرها درخواست می‌کند، ابتدا تکه‌های نادرتر.

ارسال تکه‌ها: این برای آن (tit-for-tat)

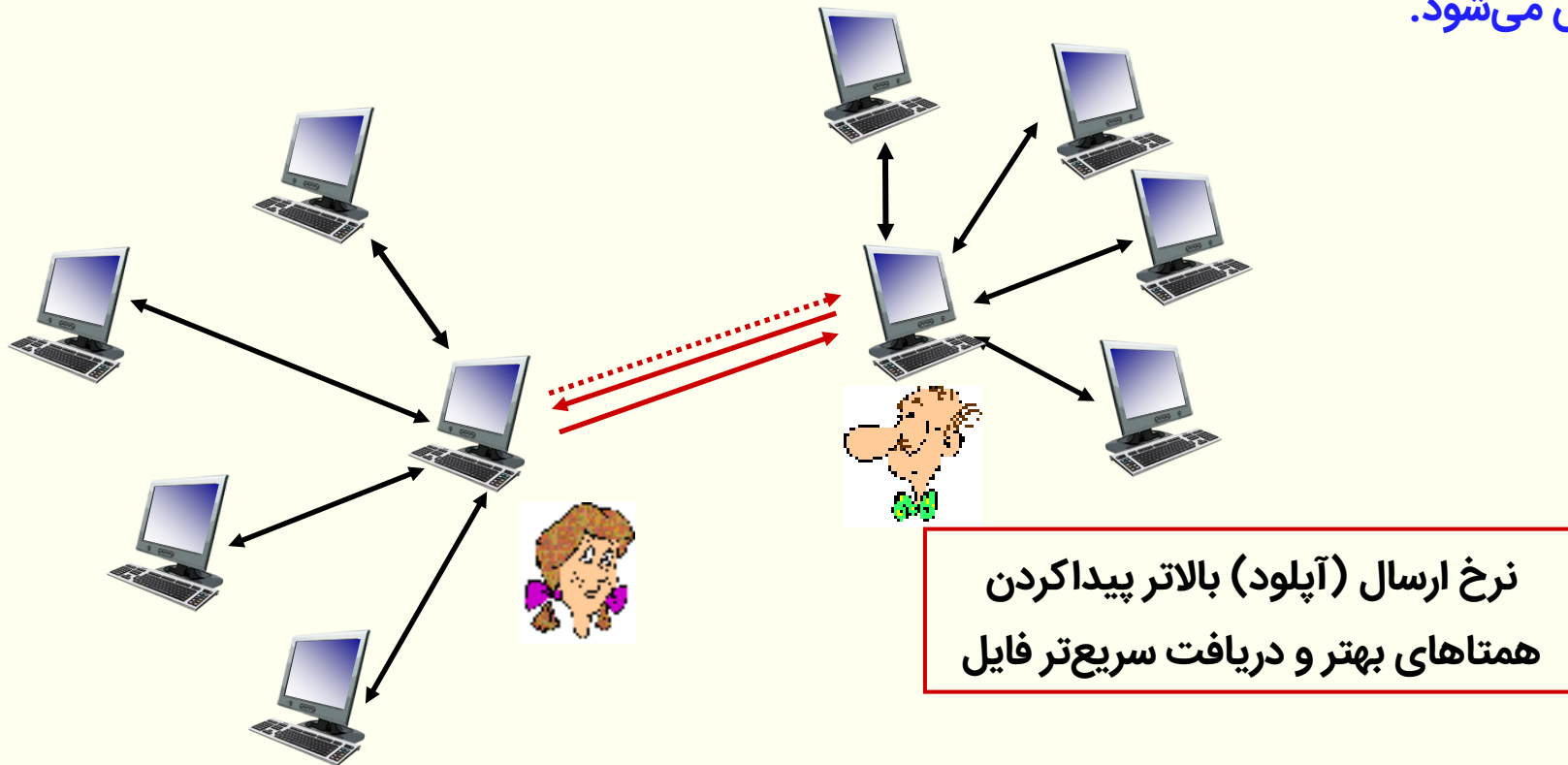
- آلیس تکه‌ها را برای چهار نظیر برتر (نظیرهایی که در حال ارسال تکه‌هایی با بیشترین نرخ به او هستند) ارسال می‌کند.
- نظیرهای دیگر توسط آلیس ساکت می‌شوند (تکه‌ای از او دریافت نمی‌کنند)
- هر ۱۰ ثانیه ۴ نظیر برتر ارزیابی مجدد می‌شوند
- هر ۳۰ ثانیه: از سایر نظیرها یک نظیر به صورت تصادفی انتخاب می‌کند و شروع به ارسال تکه‌ها با آن می‌کند.
- این نظیر "به صورت خوش‌بینانه از سکوت خارج می‌شود."
- نظیر از سکوت خارج شده ممکن است به ۴ نظیر برتر بپیوندد.

بیت تورنت: این برای آن (tit-for-tat)

(۱) آلیس به صورت خوش بینانه باب را از سکوت خارج کرد.

(۲) آلیس یکی از ۴ تأمین کننده برتر باب می شود و باب متقابلاً پاسخ می دهد.

(۳) باب یکی از ۴ تأمین کننده برتر آلیس می شود.



لایه کاربرد

جریان سازی (پخش) ویدیو و شبکه های توزیع محتوا



• ترافیک پخش ویدیو: مصرف کننده بزرگ پهنای باند شبکه اینترنت

• نتفلیکس، یوتوپ، فیلیمو، نماوا و تلویون: ۸۰ درصد ترافیک ISP های مسکونی (۲۰۲۰)



• چالش: مقیاس - چگونه تقریباً یک میلیارد کاربر برسیم؟

• یک سرویس دهنده ویدیو عظیم کار نخواهد کرد (چرا؟)



• چالش: ناهمگونی



• کاربران مختلف ظرفیت های مختلف دارند (به عنوان مثال: پهنای باند زیاد در مقابل پهنای باند کم، ثابت در مقابل موبایل)



• راه حل: زیرساخت در سطح برنامه کاربردی و توزیع شده

چند رسانه‌ای: ویدیو

- **ویدیو:** ترتیبی از تصاویر که با یک نرخ ثابت نمایش داده می‌شوند.

- به عنوان مثال ۲۴ تصویر در ثانیه

- **تصویر دیجیتال:** آرایه‌ای از پیکسل‌ها

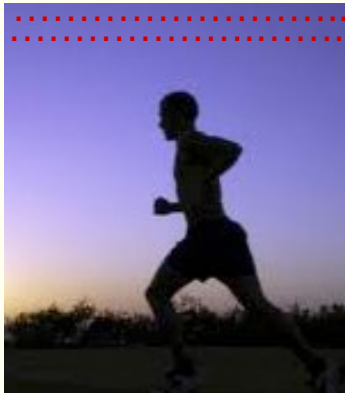
- هر پیکسل با تعداد بیت نمایش داده می‌شود.

- **کدگذاری:** از افزودن درون و بین تصاویر برای کاهش تعداد بیت‌ها در کدگذاری استفاده می‌شود.

- فضایی (spatial): در تصویر

- زمانی (temporal): از یک تصویر به تصویر بعدی

مثال کدگذاری فضایی: به جای ارسال N مقدار هم‌رنگ (تمام بنفش)، فقط دو مقدار: مقدار رنگ (بنفش) و تعداد مقادیر تکرار شده (N) ارسال می‌شود.

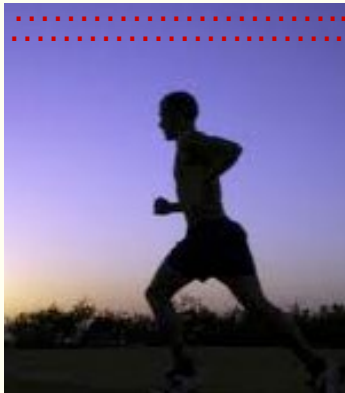


فریم i → فریم $i+1$

مثال کدگذاری زمانی: به جای ارسال کامل فریم در زمان $i+1$ فقط اختلاف آن با فریم زمان i ارسال می‌شود.

چند رسانه‌ای: ویدیو

مثال کدگذاری فضایی: به جای ارسال N مقدار هم‌رنگ (تمام بنفش)، فقط دو مقدار: مقدار رنگ (بنفش) و تعداد مقادیر تکرار شده (N) ارسال می‌شود.



فریم i → فریم $i+1$

مثال کدگذاری زمانی: به جای ارسال کامل فریم در زمان $i+1$ فقط اختلاف آن با فریم زمان i ارسال می‌شود.

• نرخ ارسال ثابت (CBR – Constant Bit Rate): ویدیو کد شده با نرخ ثابت

• نرخ ارسال متغیر (VBR – Variable Bit Rate): نرخ ویدیو کد شده با تغییرات کدگذاری فضایی و زمانی تغییر می‌کند.

• مثال:

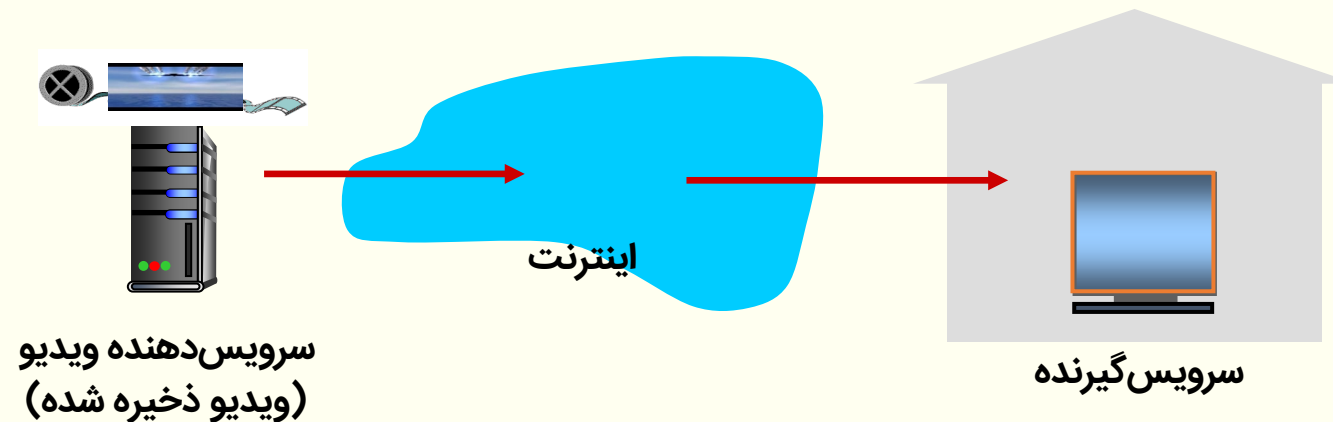
• MPEG1 (برای CD-ROM) با نرخ 1.5 Mbps

• MPEG2 (برای DVD) با نرخ 3-6 Mbps

• MPEG4 (اغلب برای استفاده در شبکه اینترنت) با نرخ 64 Kbps - 12 Mbps

جریان سازی ویدیو ذخیره شده

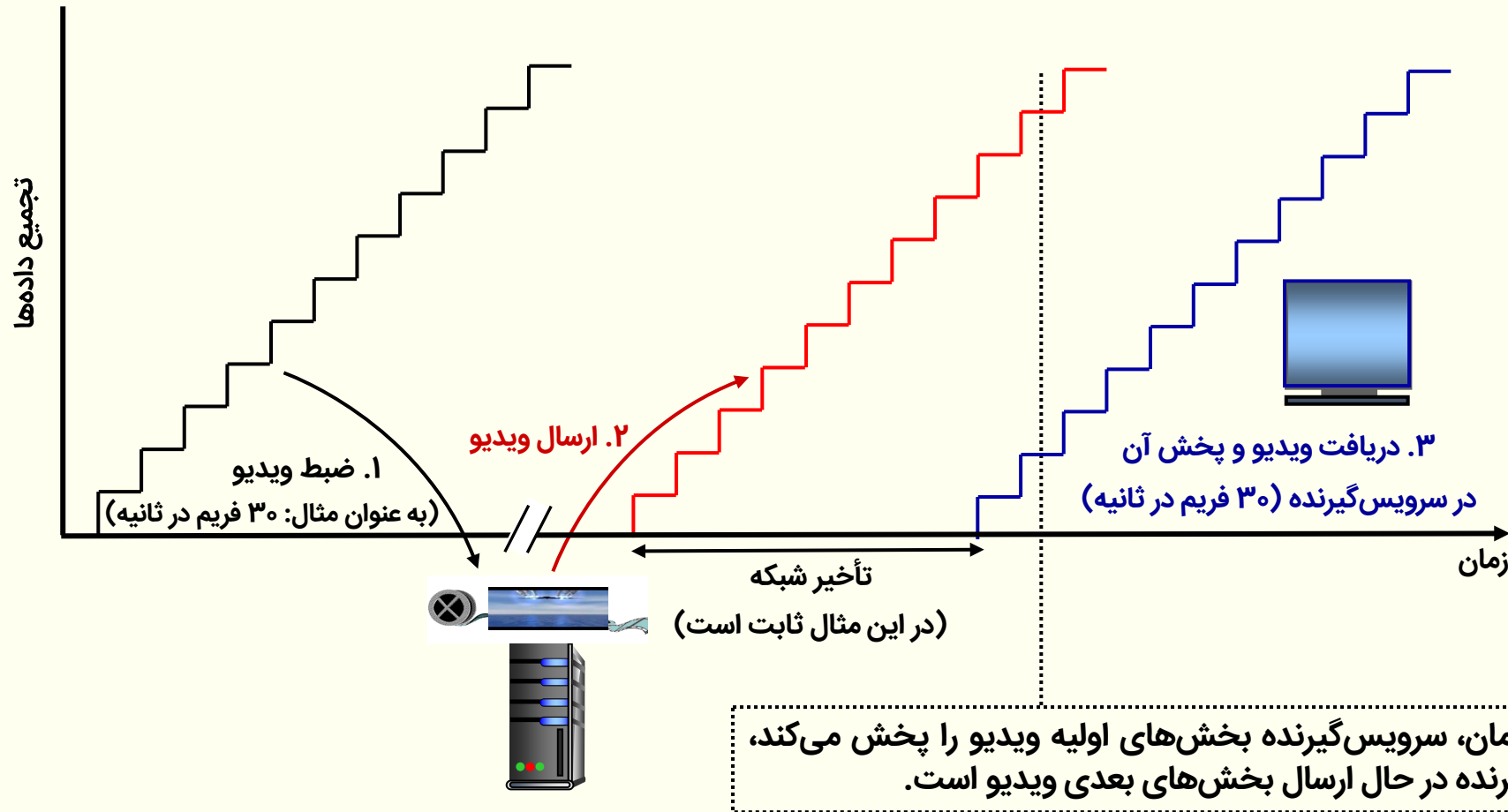
سناریو ساده:



چالش های اصلی:

- با تغییر سطوح ازدحام شبکه ممکن است پهنای باند سرویس دهنده به سرویس گیرنده در طول زمان تغییر کند (در شبکه داخل منزل، در شبکه دسترسی، در شبکه هسته، در شبکه سرویس دهنده ویدیو).
- پایین آمدن کیفیت ویدیو در حال پخش بدلیل ازدست دادن بسته ها و افزایش تأخیر ناشی از ازدحام در شبکه

جریان سازی ویدیو ذخیره شده



جریان سازی ویدیوی ذخیره شده: چالش ها

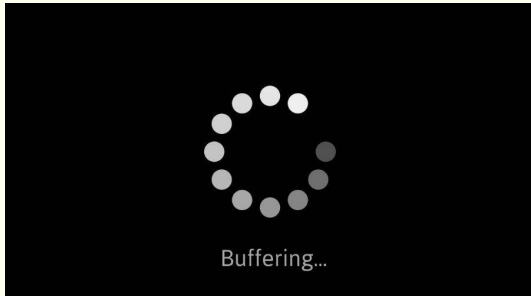
- **پخش پیوسته:** زمانی که سرویس گیرنده پخش را آغاز می کند پخش باید با زمانبندی اصلی مطابقت داشته باشد.

- با توجه به متغیر بودن تأخیر (Jitter) شبکه، لازم است سرویس گیرنده برای مطابقت با نیازمندی های پخش (زمانبندی) بافر داشته باشد.

• چالش های دیگر:

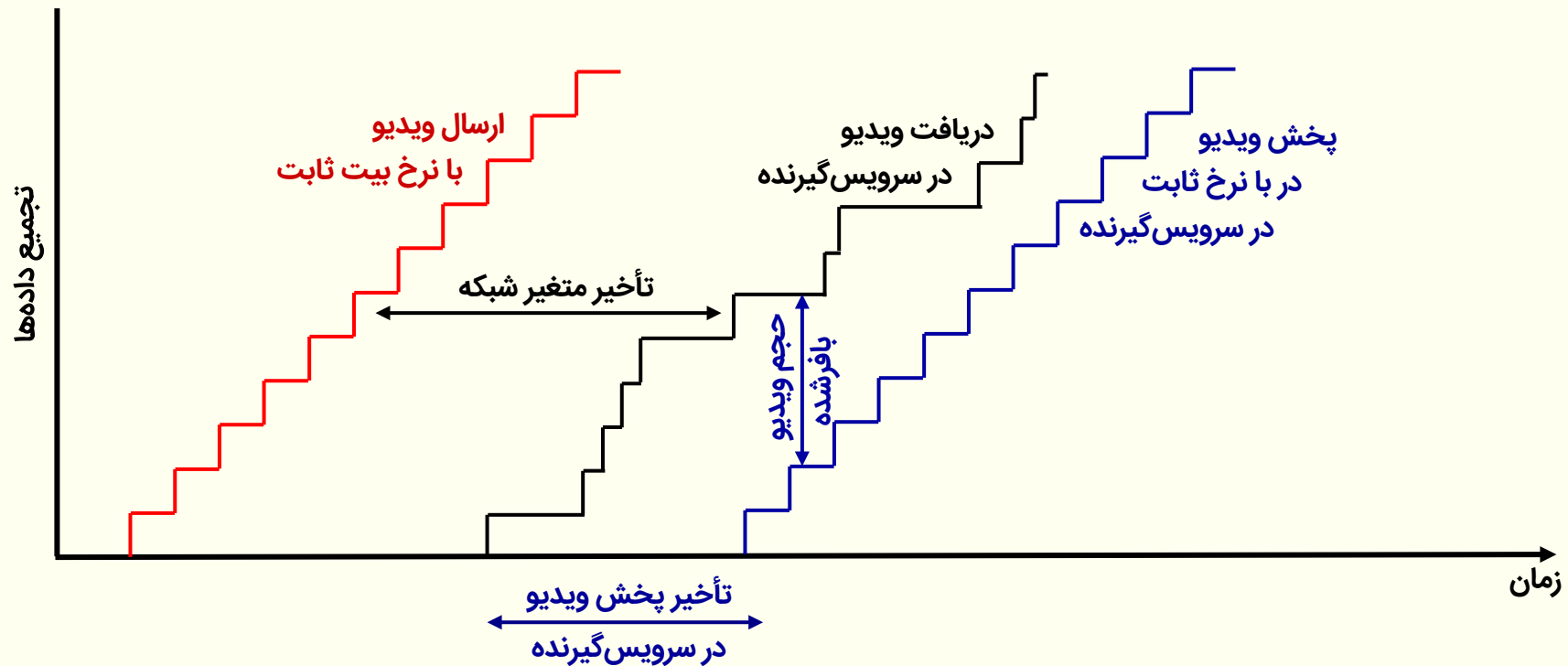
- تعامل سرویس گیرنده: ایجاد وقفه، جلو بردن سریع، بازگشت به عقب، پرش از روی ویدیو

- از دست دادن (loss) و ارسال مجدد بسته ها



لایه کاربرد

جریان سازی ویدیو ذخیره شده: بافر کردن برای پخش

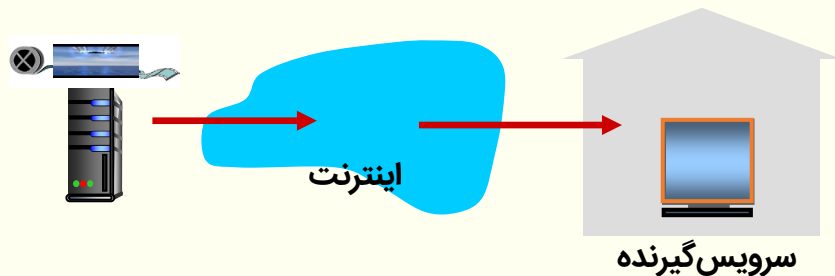


• بافر کردن و تأخیر پخش ویدیو در سرویس گیرنده: جبران تأخیر اضافه شده شبکه و تغییرات تأخیر (Jitter)

جریان سازی چند رسانه ای: DASH

• جریان سازی تطبیقی پویا روی HTTP: (DASH - Dynamic Adaptive Streaming over HTTP)

• سرویس دهنده:



• فایل ویدیویی به چندین تکه (chunk) تقسیم می شود.

• هر تکه با نرخ های مختلف کدگذاری و ذخیره می شود.

• فایل آشکارسازی (manifest): URL ها را برای تکه های مختلف را ارائه می دهد.

• سرویس گیرنده:

• پهنای باند بین سرویس دهنده و سرویس گیرنده به صورت دوره ای اندازه گیری می شود.

• مشاور آشکارسازی (consulting manifest) در هر زمان یک تکه را درخواست می کند:

• انتخاب حداکثر نرخ کدگذاری پایدار با توجه به پهنای باند فعلی

• می تواند نرخ کدگذاری مختلفی در نقاط زمانی مختلف انتخاب کند (بسته به پهنای باند در دسترس).

جریان سازی چند رسانه‌ای: DASH

• هوشمندی در سرویس گیرنده: سرویس گیرنده تعیین می کند.

• در چه زمانی تکه ها را درخواست کند.

• گرسنگی یا سرریز در بافر رخ ندهد.

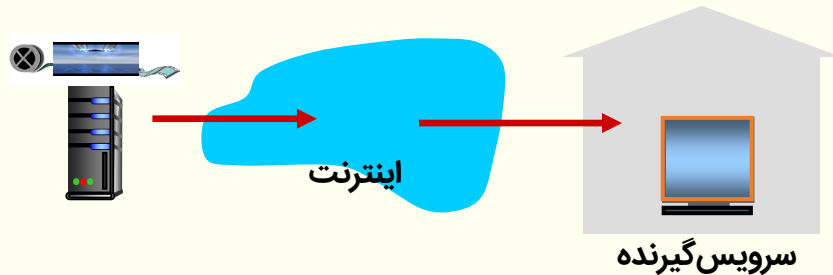
• از کدام نرخ کدگذاری درخواست کند.

• تا بیشترین کیفیت براساس پهنای باند در دستری تأمین گردد.

• از چه جایی تکه ها را درخواست کند.

• تا تأخیر کمتر دریافت را داشته باشد (سرویس دهنده نزدیک تر).

• تا بیشترین پهنای باند در دسترس را داشته باشد (بیشترین کیفیت).



جریان سازی ویدیو = کدگذاری + DASH + بافرکردن برای پخش

شبکه‌های توزیع محتوا (CDNs – Content Distribution Networks)

• **چالش:** چگونه جریان‌سازی محتوا (با انتخاب از میلیون‌ها ویدیو) به طور همزمان به صدها هزار کاربر انجام می‌شود؟

• **گزینه ۱:** استفاده از تک سرویس‌دهنده عظیم بزرگ

• تک نقطه خرابی

• نقطه ازدحام شبکه

• مسیر طولانی با سرویس‌گیرنده دور

• ارسال چندین کپی از ویدیو روی لینک خروجی

• ...

• **به سادگی:** این روش مقیاس‌پذیر نیست.

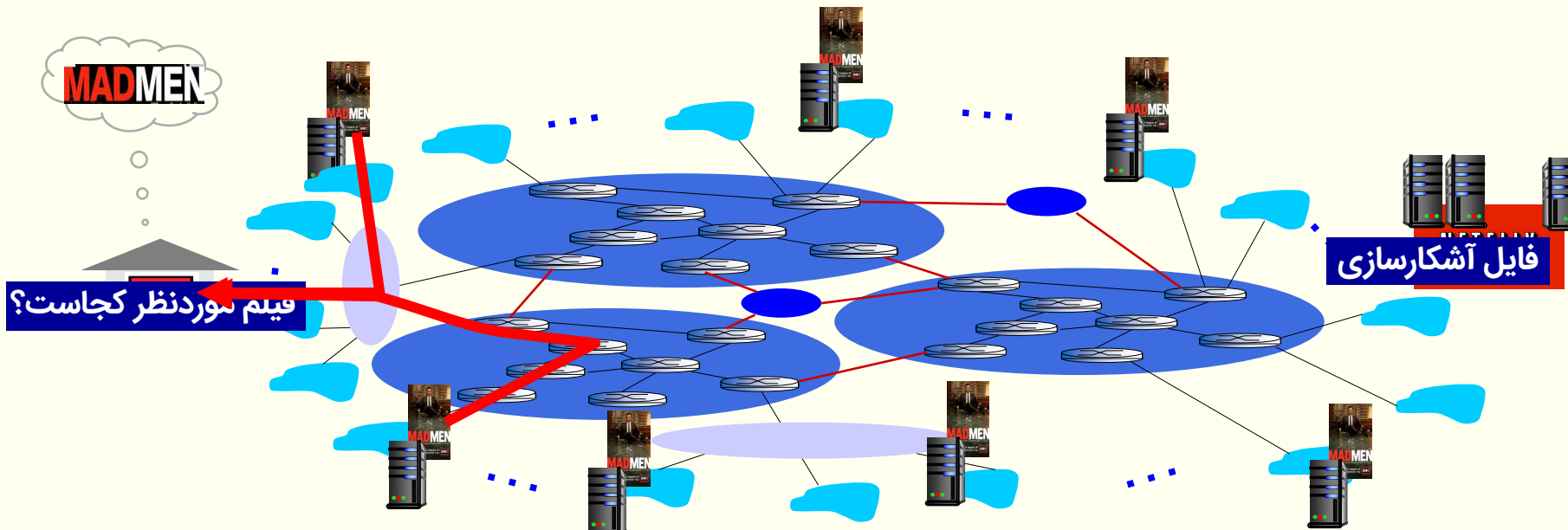
شبکه‌های توزیع محتوا (CDNs)

- **چالش:** چگونه جریان‌سازی محتوا (با انتخاب از میلیون‌ها ویدیو) به طور همزمان به صدها هزار کاربر انجام می‌شود؟
- **گزینه ۲:** ذخیره‌سازی چندین کپی از ویدیوها در چندین سایت توزیع شده جغرافیایی و ارائه محتوا از طریق آن‌ها (CDN)
- **بردن در عمق:** بردن سرویس‌دهنده‌های CDN در عمق درون شبکه‌های دسترسی نزدیک به کاربران
- Akamai: استقرار ۳۴۰,۰۰۰ سرویس‌دهنده در بیش از ۱۳۰ کشور (۲۰۲۳)
- **آوردن به خانه:** تعداد کمی (ده‌ها) خوشه‌های بزرگ در نقاط حضور (POPs) نزدیک شبکه‌های دسترسی (نه در داخل آن‌ها)
- توسط شبکه Limelight استفاده شده



شبکه‌های توزیع محتوا (CDNs)

- CDN: ذخیره‌سازی کپی‌های محتوا در گره‌های شبکه CDN
- به عنوان مثال: نتفلیکس چندین کپی از سریال‌ها و فیلم‌های پرمخاطب را در گره‌های CDN ذخیره می‌کند.
- مشترکین (subscriber) محتوا مورد نظرشان را از CDN درخواست می‌کنند.
- برای بازیابی محتوا به کپی نزدیک‌تر هدایت می‌شوند.
- ممکن است در صورت ازدحام در مسیر شبکه کپی دیگری را انتخاب کنند.



شبکه‌های توزیع محتوا (CDNs)



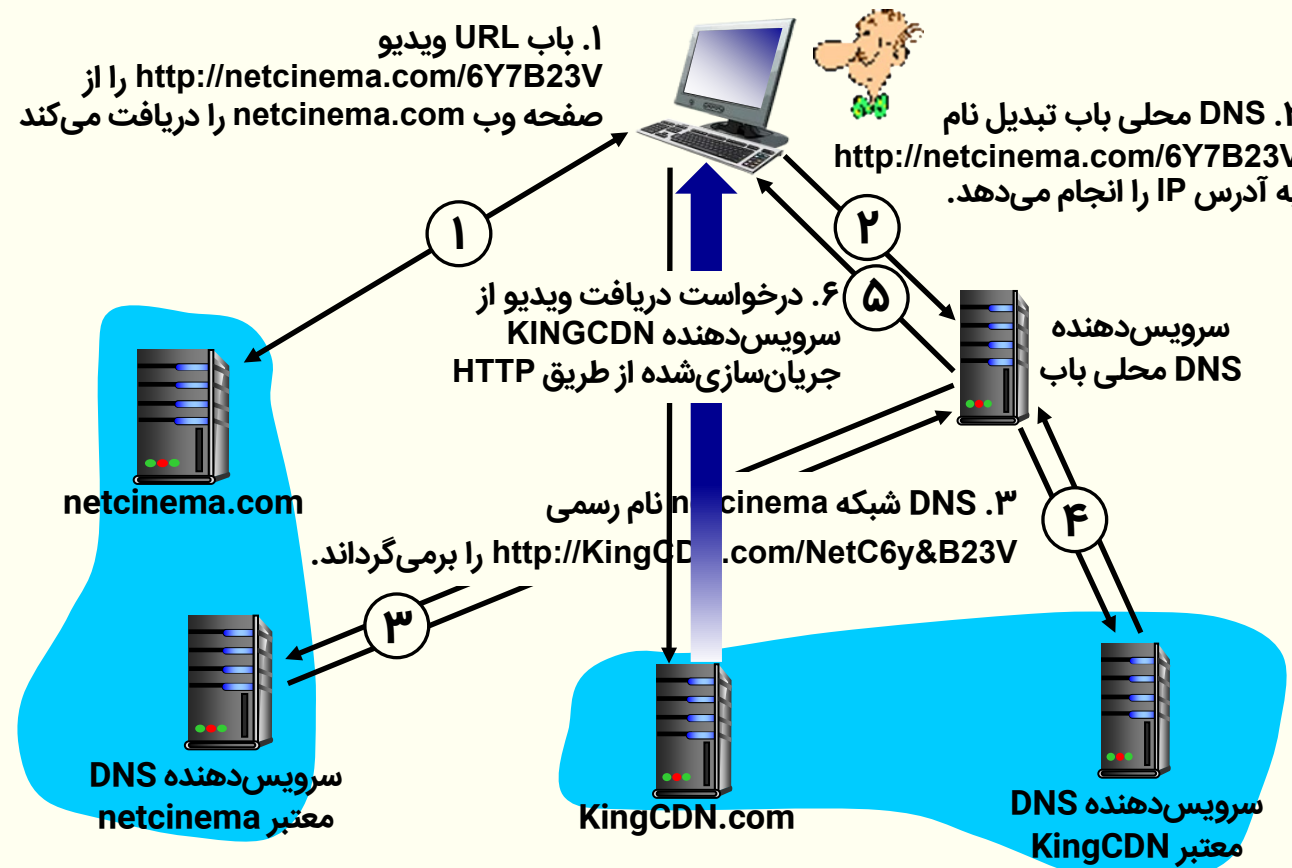
ارتباط میزبان به میزبان به عنوان سرویس

چالش‌های OTT: کپی‌کردن در شبکه اینترنت دارای ازدحام

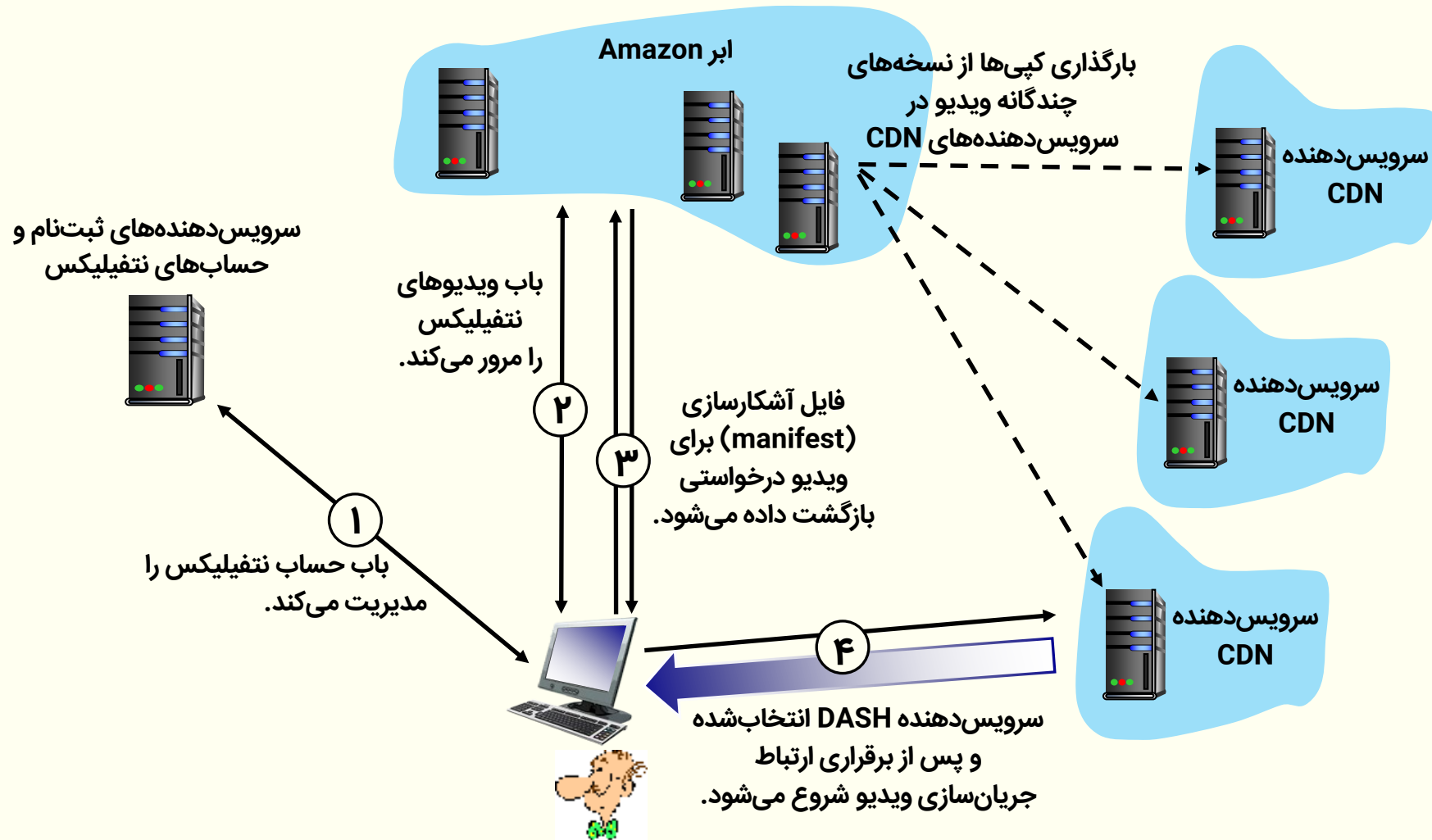
- از کدام گره CDN محتوا مورد نظر بازیابی شود؟
- رفتار بینندگان در زمان وقوع ازدحام چیست؟
- چه محتوایی در کدام گره CDN قرار گیرد؟

دسترسی محتوا CDN: نگاه نزدیکتر

- باب (سرویس گیرنده) ویدیو را از آدرس `http://netcinema.com/6Y7B23V` درخواست می کند.
- ویدیو در CDN در آدرس `http://KingCDN.com/NetC6y&B23V` ذخیره شده است.



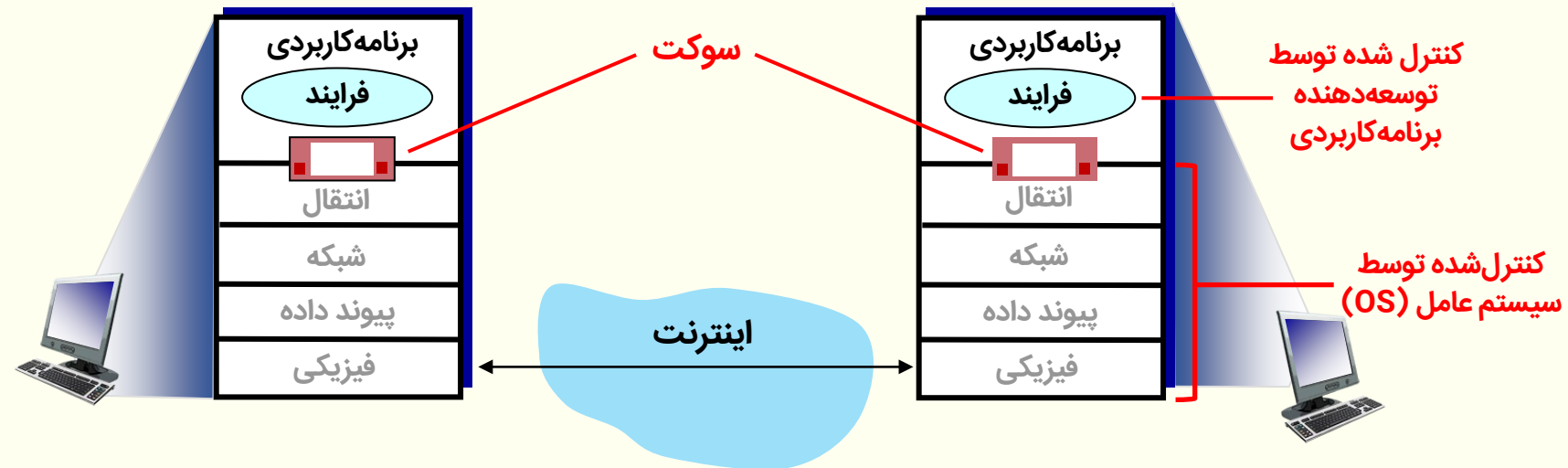
مطالعه موردی: نتفلیکس



برنامه‌نویسی سوکت (Socket Programming)

هدف: یادگیری نحوه تبادل داده‌ها بین برنامه‌های کاربردی سرویس‌گیرنده و سرویس‌دهنده از طریق سوکت

سوکت: درب (واسط) میان فرایند برنامه‌کاربردی و پروتکل انتقال انتها به انتها



برنامه‌نویسی سوکت (Socket Programming)

دو نوع سوکت برای ارائه سرویس لایه انتقال:

• سوکت UDP: غیرمطمئن (بیشترین تلاش) و بدون اتصال (دیتاگرام)

• سوکت TCP: مطمئن، انتقال رشته بایت اتصال‌گرا

• مثال برنامه‌کاربردی:

۱. دریافت یک رشته کاراکتر از طریق صفحه کلید از کاربر توسط سرویس‌گیرنده و ارسال به سرویس‌دهنده
۲. دریافت داده‌ها (رشته کاراکتر) توسط سرویس‌دهنده و تبدیل آن به حروف بزرگ
۳. ارسال داده‌های تغییر یافته به سرویس‌گیرنده توسط سرویس‌دهنده
۴. دریافت داده‌های تغییر یافته توسط سرویس‌گیرنده و نمایش آن‌ها روی صفحه نمایش برای کاربر

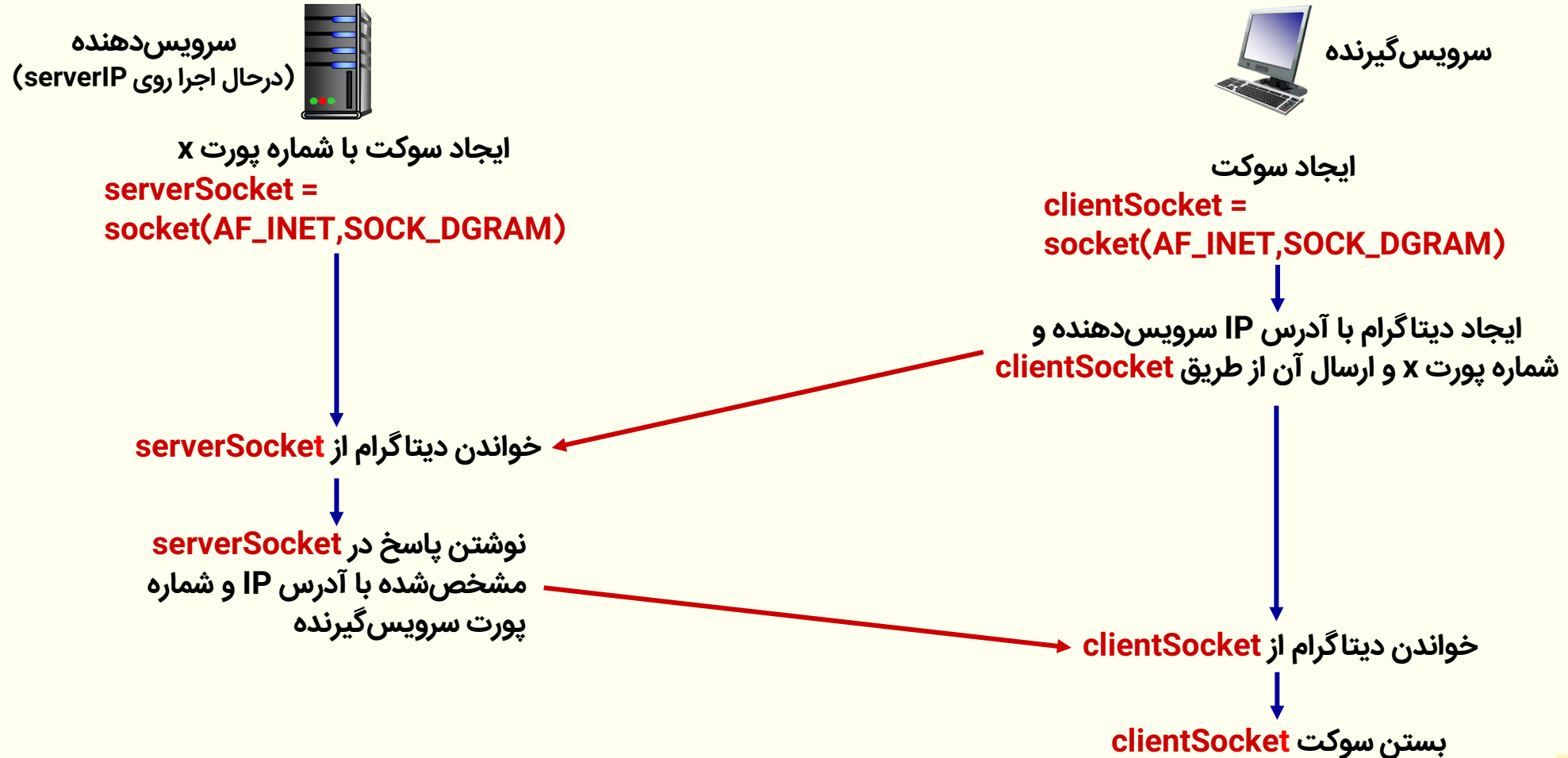
برنامه‌نویسی سوکت با UDP

- UDP یک پروتکل **بدون اتصال** است (هیچ اتصالی بین سرویس‌گیرنده و سرویس‌دهنده وجود ندارد):
- قبل از ارسال داده، هیچ‌گونه دست‌دانی وجود ندارد.
 - فرستنده صراحتاً **آدرس IP** و **شماره پورت مقصد** را به هر پیام خود الحاق می‌کند.
 - دریافت‌کننده آدرس IP و شماره پورت فرستنده را از دیتاگرام دریافتی بدست می‌آورد.
 - امکان از دست دادن یا دریافت خارج از ترتیب داده‌های ارسال شده توسط UDP

از نگاه برنامه کاربردی

UDP یک انتقال غیر مطمئن گروهی از بایت‌ها (یک دیتاگرام) بین فرایندهای برنامه‌های کاربردی سرویس‌گیرنده و سرویس‌دهنده فراهم می‌کند.

تعامل سوکت سرویس گیرنده و سرویس دهنده: UDP



مثال برنامه کاربردی: سرویس گیرنده UDP

Python UDPClient

→ اضافه کردن کتابخانه سوکت پایتون به برنامه

```
from socket import *
```

```
serverName = 'hostname'
```

```
serverPort = 12000
```

→ ایجاد سوکت UDP برای دریافت سرویس

```
clientSocket = socket(AF_INET, SOCK_DGRAM)
```

→ دریافت یک رشته کاراکتر از ورودی صفحه کلید

```
message = raw_input('Input lowercase sentence:')
```

→ الحاق کردن نام و شماره پورت سرویس دهنده و ارسال آن درون سوکت

```
clientSocket.sendto(message.encode(), (serverName, serverPort))
```

→ خواندن رشته کاراکترهای تغییر یافته از سوکت

```
modifiedMessage, serverAddress = clientSocket.recvfrom(2048)
```

→ نمایش رشته کاراکترهای تغییر یافته بر روی صفحه نمایش

```
print modifiedMessage.decode()
```

→ بستن سوکت

```
clientSocket.close()
```

لایه کاربرد

مثال برنامه کاربردی: سرویس دهنده UDP

Python UDP Server

→ اضافه کردن کتابخانه سوکت پایتون به برنامه `from socket import *`

`serverPort = 12000`

→ ایجاد سوکت UDP `serverSocket = socket(AF_INET, SOCK_DGRAM)`

→ انتساب شماره پورت ۱۲۰۰۰ به سوکت `serverSocket.bind(('', serverPort))`

`print ("The server is ready to receive")`

→ حلقه بی‌نهایت `while True:`

→ خواندن پیام و دریافت آدرس IP و شماره پورت سرویس گیرنده از سوکت `message, clientAddress = serverSocket.recvfrom(2048)`

→ تغییر رشته کاراکترهای پیام به حروف بزرگ `modifiedMessage = message.decode().upper()`

→ ارسال رشته کاراکترهای تغییر یافته (تبدیل به حروف بزرگ شده) به سرویس گیرنده `serverSocket.sendto(modifiedMessage.encode(), clientAddress)`

برنامه‌نویسی سوکت با TCP

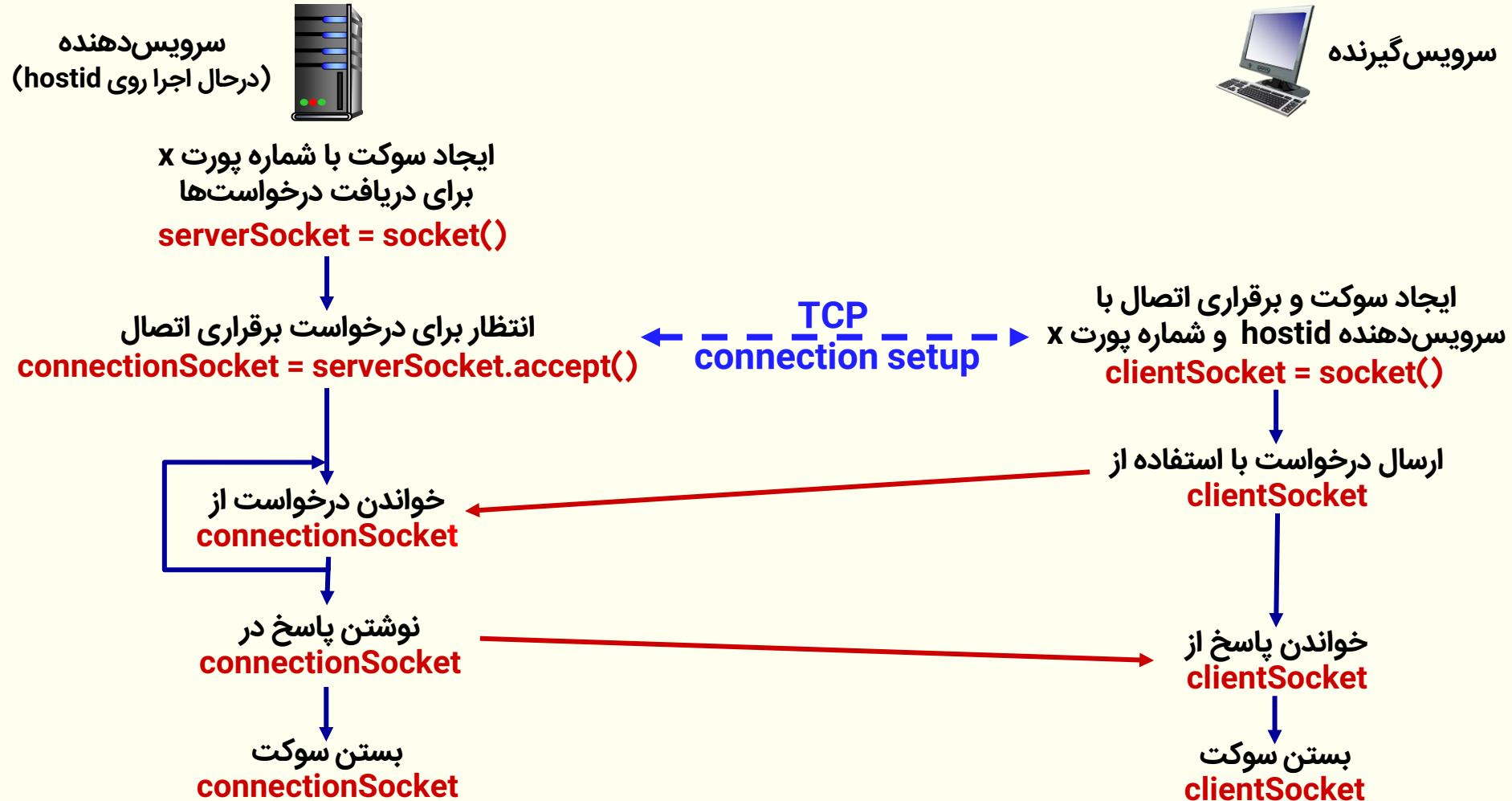
- فرایند سرویس‌دهنده، با برقراری اتصال TCP بین سرویس‌گیرنده و سرویس‌دهنده، یک سوکت جدید برای تبادل داده‌ها با این سرویس‌گیرنده ایجاد می‌کند.
- این کار به سرویس‌دهنده اجازه می‌دهد به طور همزمان با چندین سرویس‌گیرنده ارتباط داشته باشد.
- شماره پورت مبدأ برای تمایز بین سوکت‌ها استفاده می‌شود (در لایه انتقال بیشتر توضیح داده خواهد شد).

از نگاه برنامه کاربردی

TCP یک انتقال مطمئن (بدون خطا و با حفظ ترتیب ارسال) به صورت جریان رشته بایت‌ها بین فرایندهای برنامه‌های کاربردی سرویس‌گیرنده و سرویس‌دهنده فراهم می‌کند.

- سرویس‌گیرنده باید با سرویس‌دهنده اتصال برقرار کند:
- فرایند سرویس‌دهنده باید از قبل (قبل از سرویس‌گیرنده) شروع به اجرا کرده باشد.
- سرویس‌دهنده باید سوکت (درب) را ایجاد کرده و آماده دریافت درخواست برقراری اتصال باشد.
- سرویس‌گیرنده با سرویس‌دهنده اتصال برقرار می‌کند:
- ایجاد سوکت با سرویس‌دهنده‌ای که آدرس IP و شماره پورت آن مشخص است.
- زمانی که فرایند سرویس‌گیرنده سوکت را ایجاد می‌کند، TCP سرویس‌گیرنده اتصال را با TCP سرویس‌دهنده برقرار می‌کند.

تعامل سوکت سرویس گیرنده و سرویس دهنده: TCP



مثال برنامه کاربردی: سرویس گیرنده TCP

Python TCPClient

→ اضافه کردن کتابخانه سوکت پایتون به برنامه

```
from socket import *
```

```
serverName = 'servername'
```

```
serverPort = 12000
```

→ ایجاد سوکت TCP

```
clientSocket = socket(AF_INET, SOCK_STREAM)
```

→ درخواست برقراری اتصال با سرویس دهنده مشخص شده با نام و شماره پورت مشخص

```
clientSocket.connect((serverName, serverPort))
```

→ دریافت یک رشته کاراکتر از ورودی صفحه کلید

```
sentence = raw_input('Input lowercase sentence:')
```

→ ارسال رشته کاراکتر در سوکت

```
clientSocket.send(sentence.encode())
```

→ دریافت رشته کاراکترهای تغییر یافته از سوکت (نیازی به الحاق نام و شماره پورت سرویس دهنده نیست)

```
modifiedSentence = clientSocket.recv(1024)
```

→ نمایش رشته کاراکترهای تغییر یافته بر روی صفحه نمایش

```
print ('From Server:', modifiedSentence.decode())
```

→ بستن سوکت

```
clientSocket.close()
```

مثال برنامه کاربردی: سرویس دهنده TCP

Python TCP Server

→ اضافه کردن کتابخانه سوکت پایتون به برنامه

```
from socket import *
```

```
serverPort = 12000
```

→ ایجاد سوکت خوش آمدگویی TCP (بازکردن سوکت غیرفعال)

```
serverSocket = socket(AF_INET, SOCK_STREAM)
```

→ انتساب شماره پورت ۱۲۰۰۰ به سوکت

```
serverSocket.bind(('',serverPort))
```

→ شروع به گوش کردن برای دریافت درخواست‌های TCP توسط سرویس دهنده

```
serverSocket.listen(1)
```

```
print 'The server is ready to receive'
```

→ حلقه بی‌نهایت

```
while True:
```

→ انتظار برای پذیرش درخواست ورودی و ایجاد سوکت جدید برای آن

```
connectionSocket, addr = serverSocket.accept()
```

→ خواندن پیام از سوکت (ولی نیازی دریافت آدرس IP و شماره پورت شبیه UDP نیست)

```
sentence = connectionSocket.recv(1024).decode()
```

→ تغییر رشته کاراکترهای پیام به حروف بزرگ

```
capitalizedSentence = sentence.upper()
```

→ ارسال رشته کاراکترهای تغییر یافته (تبدیل به حروف بزرگ شده)

```
connectionSocket.send(capitalizedSentence.encode())
```

→ بستن سوکت با سرویس گیرنده
(اما سوکت خوش آمدگویی همچنان باز است)

```
connectionSocket.close()
```

خلاصه لایه کاربرد

بررسی و مطالعه لایه کاربرد به اتمام رسید! در این مطالعه موارد زیر بررسی شد:

- معماریهای برنامه‌های کاربردی
 - معماری سرویس‌دهنده-سرویس‌گیرنده (Client-Server)
 - معماری نظیر به نظیر (Peer-to-Peer)
- نیازمندی‌های کیفیت سرویس برنامه‌های کاربردی
 - قابلیت اطمینان، پهنای باند، تأخیر
- مدل‌های سرویس لایه انتقال شبکه اینترنت
 - اتصال‌گرا و مطمئن: TCP
 - بدون اتصال (دیتاگرام) و غیرمطمئن: UDP
- بررسی تعدادی از پروتکل‌های پرکاربرد
 - HTTP
 - SMTP و IMAP
 - DNS
 - FTP
 - BitTorrent :P2P
- جریان‌سازی ویدیو و شبکه‌های توزیع محتوا (CDNs)
- برنامه‌نویسی سوکت:
 - سوکت‌های UDP و TCP